

Noise and Uncertainty in Molecular Property Prediction: Robust Models and Adaptive Representations

Adelaide Punt

St Catherine's College

University of Oxford

A thesis submitted for the degree of

Doctor of Philosophy

October 2026

Declaration of AI Use

During this doctorate I used Claude, a large language model developed by Anthropic (https://claude.ai), [model versions and dates of use]. I used it for three purposes. First, I used it to help write, test and debug software, including parts of the code that ran the experiments reported in this thesis. Second, I used it to draft and edit parts of the text and to typeset it in LaTeX. Third, I used it to search for literature and to check citations against their sources. I reviewed, edited and verified all of its output before using it. [No data, results or figures were generated by the tool; every reported number comes from code that I ran and checked.] The research questions, experimental design, analysis and conclusions are my own. [My supervisors were aware of this use.] I take full responsibility for the content of this thesis, including any errors.

Acknowledgements

I would like to thank my supervisors, Garrett Morris at the University of Oxford and Thierry Hanser and Stéphane Werner at Lhasa Limited, for their guidance, their time and their patience throughout this project.

I also thank the EPSRC (EP/L015803/1) as well as my industrial partner Lhasa for their financial support.

Thank you to my parents and grandparents for their support, and to my cat, [name].

Abstract

Contents

Declaration of AI Use ii

Acknowledgements iii

Abstract iv

List of Abbreviations x

1 Introduction 1
- 1.1 Outline of the Thesis and Contributions . . . 3
- 1.2 Collaborations . . . 5

2 Molecular Representations, Models and Label Noise 6
- 2.1 Overview . . . 6
- 2.2 Molecular Representations in QSAR . . . 6
- 2.3 Models for Molecular Property Prediction . . . 12
  - 2.3.1 Tree Ensembles . . . 14
  - 2.3.2 Support Vector Machines . . . 15
  - 2.3.3 Gaussian Processes . . . 15
  - 2.3.4 Bayesian Neural Networks . . . 16
- 2.4 Molecular Similarity and Activity Cliffs . . . 17
- 2.5 Noise in Molecular Data . . . 19
  - 2.5.1 Artificial Noise Injection . . . 21
  - 2.5.2 Learning with Noisy Labels . . . 24
- 2.6 NoiseInject: A Benchmarking Framework . . . 25
  - 2.6.1 NoiseInject . . . 25
- 2.7 Evaluation Methodology . . . 29
- 2.8 Conclusions . . . 29

3 Noise Robustness Across Molecular Representations and Model Architectures 30
- 3.1 Introduction . . . 30
- 3.2 Methods . . . 31
  - 3.2.1 Datasets . . . 31
  - 3.2.2 Representations . . . 32
  - 3.2.3 Models . . . 33
  - 3.2.4 Noise Conditions and Levels . . . 34
  - 3.2.5 QM9 Data Pipeline . . . 35
  - 3.2.6 Uncertainty . . . 36

3.2.7 Evaluation . . . 38
3.3 Variance Decomposition by Representation and Architecture . . . 40
3.4 Probabilistic and Deterministic Model Variants . . . 45
3.5 Model Robustness Rankings across Noise Strategies . . . 46
3.6 Validation on Assay Datasets . . . 48
3.7 Uncertainty under Label Noise . . . 51
3.7.1 Per-Sample Uncertainty and Label Noise . . . 51
3.7.2 Treatment of Large Residuals . . . 52
3.7.3 Aleatoric and Epistemic Attribution . . . 52
3.7.4 Representation and Uncertainty Tracking . . . 55
3.8 Conclusions . . . 55

4 The Flaws of the Similarity Assumption 58
4.1 Introduction . . . 58
4.2 Flaws in Standard Base Representations . . . 60
4.2.1 Constant and Near-Constant Features . . . 60
4.2.2 Molecules with Identical Feature Vectors . . . 64
4.3 Flaws in Similarity and Other Pairwise Computations . . . 66
4.4 Consequences for Similarity-Based Methods . . . 72
4.4.1 Exploratory Study of Learning with Noisy Labels . . . 72
4.4.2 Noise Detection by Neighbors . . . 74
4.5 Conclusions . . . 76

5 Building Adaptive Hybrid Representations 78
5.1 Introduction . . . 78
5.1.1 Combining Representations and the Alternatives . . . 79
5.1.2 The Adaptive Hybrid Representation . . . 81
5.1.3 Blueprints and Their Chemical Interpretation . . . 83
5.1.4 Beyond Small Molecules . . . 84
5.2 Methods: The Hybrid Search . . . 84
5.2.1 From Base Representations to a Hybrid . . . 85
5.2.2 The Search Space . . . 88
5.2.3 Augmentation . . . 93
5.2.4 Fast Tier and Adaptive Pruning . . . 96
5.3 Methods: Experimental Setup . . . 99
5.3.1 Datasets and Splitting . . . 99
5.3.2 Representations by Domain . . . 102
5.3.3 Models and Metrics . . . 110
5.3.4 Comparators, Controls and Ablations . . . 112
5.3.5 Transfer, Mutational Scans and Separate Sites . . . 116
5.4 Results: Comparison with Single Representations and Ensembles . . . 119
5.5 Results: Sources of the Gain . . . 119
5.6 Results: Blueprints on Specific Datasets . . . 119
5.7 Results: Transfer across Datasets and Antibody Mutational Scans . . . 119
5.8 Results: Data Held at Separate Sites . . . 119
5.9 Discussion . . . 119
5.10 Limitations . . . 120
5.11 Conclusions . . . 121

6 Hybrid Representations for Similarity and Label Noise **122**
6.1 Introduction . . . 122
6.2 Methods . . . 123
6.2.1 Similarity in a Hybrid . . . 123
6.2.2 Noise Conditions . . . 125
6.2.3 Noise Mitigation Methods . . . 126
6.3 Results: Similarity . . . 129
6.4 Results: Noise Detection and Estimation . . . 129
6.5 Results: Noise Mitigation by Noise Type . . . 129
6.5.1 Censoring: Correction of Clipped Labels . . . 129
6.5.2 Gaussian Noise: Methods for Learning with Noisy Labels . . . 129
6.5.3 Real Labels: Cleaning without Injected Noise . . . 129
6.5.4 Comparison across Noise Types . . . 129
6.6 Conclusions . . . 129

7 Discussion and Conclusions **130**
7.1 Summary . . . 130
7.2 Dataset Dependence of the Best Method . . . 131
7.2.1 Simple Models and Small Feature Sets . . . 132
7.3 Limitations . . . 133
7.4 Future Directions . . . 134
7.5 Conclusions . . . 135

Summary of Research Contributions **137**

List of Figures

2.1 Three representations of one molecule. 7
2.2 Nodes of a deterministic and a Bayesian neural network. 16
2.3 Label distributions under the seven noise conditions. 22
2.4 Evaluation of a noise mitigation method. 24

3.1 In-memory and memory-mapped data loading. 37
3.2 ANOVA variance decomposition of performance and robustness on QM9. 42
3.3 Model–representation interaction in noise robustness on QM9. 43
3.4 Noise robustness of all models on PDV for QM9. 44
3.5 Neural network families and random forests under Gaussian noise on QM9. 46
3.6 Noise degradation slope on LogD, Caco-2 and hERG K_i. 49
3.7 Noise robustness on QM9 and three assay datasets. 50
3.8 Mean predicted uncertainty under label noise on QM9. 53

4.1 Shares of constant, near-constant and varying features per representation. 62
4.2 Two hERG K_i stereoisomers with identical ECFP4 vectors. 66
4.3 Absolute pK_i difference against pair distance on hERG K_i. 71

5.1 Schematic overview of the hybrid search. 85

List of Tables

2.1 Sources of label noise in bioactivity data. 20
2.2 The seven noise conditions and their parameters. 23

3.1 ANOVA variance decomposition by noise strategy on QM9. 41
3.2 Wilcoxon signed-rank tests of probabilistic variants on QM9. 45
3.3 Noise degradation slope by model on PDV for QM9. 47
3.4 Strongest and weakest uncertainty–noise correlations on QM9. 51

4.1 Constant and near-constant features per representation, part 1. . . . 63
4.2 Constant and near-constant features per representation, part 2. . . . 63
4.3 Molecules that share a feature vector with another molecule. 67
4.4 All-pairs correlation, five-nearest-neighbor prediction and activity cliffs per dataset. 71
4.5 Agreement between representations per dataset. 72
4.6 Recovery with neighbor-based sample weights by representation. . . . 75
4.7 Flaws of standard representations and similarity, and the requirement each sets. 77

5.1 Methods for combining or reducing representations. 82
5.2 Settings of the three importance methods searched. 91
5.3 The five allocation methods searched. 92
5.4 Worked example of the four fixed-share allocation methods. 92
5.5 The three augmentation conditions. 95
5.6 Values searched for each setting at the two search tiers. 97
5.7 The adaptive pruning rules. 98
5.8 Datasets and their best single representations. 100
5.9 Representations used for small molecules. 104
5.10 Representations used for chemical reactions. 105
5.11 Representations used for antibodies. 106
5.12 Representations used for nucleotide sequences. 107
5.13 Representations used for cancer patients. 107
5.14 Base and augmentation representations per dataset. 108

6.1 Noise magnitude estimators. 127
6.2 Label smoothing and sample weighting rules. 128

List of Abbreviations

AC Activity cliff
ADME Absorption, distribution, metabolism, and excretion
AI Artificial intelligence
ANOVA Analysis of variance
ASCII American Standard Code for Information Interchange
AT Adenine–thymine
AUC_{norm} Normalized area under the retention curve
AUROC Area under the receiver operating characteristic curve
BLOSUM Blocks substitution matrix
BNN Bayesian neural network
BRCA Breast invasive carcinoma
CASH Combined algorithm selection and hyperparameter optimization
CDR Complementarity-determining region
CGR Condensed graph of reaction
CIMP CpG island methylator phenotype
CLS Classification token
CpG Cytosine–phosphate–guanine
CPU Central processing unit
CSV Comma-separated values
DeepLIFT Deep Learning Important FeaTures
DNA Deoxyribonucleic acid
DNN Deep neural network
$\mathbf{EC_{50}}$ Half-maximal effective concentration
ECE Expected calibration error
ECFP Extended-connectivity fingerprint
ECFP4 Extended-connectivity fingerprint, diameter 4
ESM Evolutionary Scale Modeling
FcRn Neonatal Fc receptor
FLuID Federated learning using information distillation
GC Guanine–cytosine
GCN Graph convolutional network
GIN Graph isomorphism network
GNN Graph neural network
GP Gaussian process
GPL GNU General Public License
GPU Graphics processing unit
H3K4me3 Trimethylation of lysine 4 on histone H3

HER2 Human epidermal growth factor receptor 2
hERG Human ether-à-go-go-related gene
HLM Human liver microsomes
HOMO Highest occupied molecular orbital
HRD Homologous recombination deficiency
IC_{50} Half-maximal inhibitory concentration
ICC Intraclass correlation coefficient
IMGT International ImMunoGeneTics information system
InChIKey Hashed International Chemical Identifier
JSON JavaScript Object Notation
KL Kullback–Leibler
kNN k-nearest neighbors
LightGBM Light Gradient Boosting Machine
LM Language model
LNL Learning with noisy labels
LOF Local outlier factor
LogD Logarithm of the distribution coefficient
LOH Loss of heterozygosity
LST Large-scale state transitions
LUAD Lung adenocarcinoma
LUMO Lowest unoccupied molecular orbital
LUSC Lung squamous cell carcinoma
MACCS Molecular ACCess System
MCC Matthews correlation coefficient
MCS Maximum common substructure
MDR1–MDCK Multidrug resistance protein 1 in Madin–Darby canine kidney cells
MHG-GNN Molecular Hypergraph Grammar Graph Neural Network
miRNA MicroRNA
ML Machine learning
MLP Multi-layer perceptron
mRMR Minimum redundancy, maximum relevance
NDS Noise degradation slope
NGBoost Natural gradient boosting
NLP Natural language processing
NN Neural network
OPIG Oxford Protein Informatics Group
PAM50 Prediction Analysis of Microarray 50
PCA Principal component analysis
PDV Physicochemical descriptor vector
PLS Partial least squares
QRF Quantile regression forest
QSAR Quantitative structure–activity relationship
RAM Random-access memory
RBF Radial basis function
ReLU Rectified linear unit
RF Random forest
RLM Rat liver microsomes

RMSE Root mean squared error
RNA Ribonucleic acid
RNA-seq RNA sequencing
RPPA Reverse-phase protein array
SELFIES Self-referencing embedded strings
SHAP SHapley Additive exPlanations
SMARTS SMILES arbitrary target specification
SMILES Simplified molecular-input line-entry system
SNS Sort & Slice
SVM Support vector machine
t-SNE t-distributed stochastic neighbor embedding
TAI Telomeric allelic imbalance
TCGA The Cancer Genome Atlas
Th1 T helper type 1
Th2 T helper type 2
TNF Tetranucleotide frequency
UCSC University of California, Santa Cruz
UMAP Uniform manifold approximation and projection
USR Ultrafast shape recognition
USRCAT Ultrafast shape recognition with CREDO atom types
UTF-8 8-bit Unicode Transformation Format
VBLL Variational Bayesian Last Layer
VEGF Vascular endothelial growth factor
WL Weisfeiler–Lehman
XGBoost eXtreme Gradient Boosting

Chapter 1

Introduction

New avenues in the drug discovery pipeline have opened up thanks to advances in large dataset collection, analysis, and computation. Therapeutic targets are identified, potential drug candidates are screened and optimized, and the most promising of these go on to pre-clinical and clinical trials [1]. There are various methods by which potential drug candidates can be screened. Virtual screening allows for the rapid and large-scale analysis of libraries of compounds, and is a particularly useful approach used to speed up the drug discovery process [2]. One standard screening method is the prediction of bioactivities and pharmacological properties of such candidates. There is a long history of using Quantitative Structure-Activity Relationship (QSAR) models to make these predictions, to act as a filter for compounds with undesirable properties, and thus reduce the time and cost associated with experimental screening [3]. QSAR models aim to predict the relationship between a molecule's molecular features as encoded within a particular molecular representation and a desired molecular property [3] of interest. A QSAR model typically consists of a molecular representation paired with a machine learning (ML) model. An ideal molecular representation will allow a model to learn only the relevant information about each molecule with respect to the target property. Representations are either 'static' hand-crafted descriptors or circular fingerprints that encode predefined structural and physicochemical features [4–7]; or learned representations, where the encoding is derived from a predictive model, often taken from the activations of the final layer of a neural network, or a latent space embedding [3, 8].

QSAR models are of course, sensitive to the quality of the input data on which they are trained. Experimental noise can arise from measurement errors, variability under experimental conditions, varying provenance of the data, and intrinsic biolog-

ical and chemical variability. Even the very measures of model performance may be flawed if test and validation sets are laden with noise. Kolmar and Grulke [9] show that the variance associated with experimental data can contribute more to prediction error than the model's own error. There is also a limit to how much noise the model can tolerate; Jorner et al. [10] found that once the model error approaches the magnitude of the experimental measurement error on labels, adding more training data yields diminishing returns in predictive performance. In the context of activity prediction, the inherent variability within experimental setups and assays, batch effects, measurement errors, and biological variability introduce additional layers of complexity [9, 11]. All of this increases the likelihood of encountering noise in the data [12]. Even the molecular representations in the training data themselves may be incorrect or misleading [13], and chemical datasets contain too few repeat measurements of each compound to characterize the distribution the label is drawn from. Thus, models treat a single label as the population mean [9]. Repeat measurements of the same compound-target pair disagree by about 0.68 log units for pIC_{50}, and by about 0.54 log units for $\mathrm{p}K_i$ [14, 15]. Measurements from a single assay lowers the mean disagreement from 0.50 to 0.27 log units [16].

This thesis addresses the challenge of conducting molecular property prediction when the underlying experimental data contain noisy labels. Most work on noise in QSAR modeling assumes Gaussian label noise and evaluates either representations or model architectures in isolation. Cortes-Ciriano et al. [17] examined 12 algorithms across multiple noise levels but used fixed representations, and studies that compare representations typically use clean data. No work has systematically compared the relative importance of representation versus architecture choice for noise robustness, asked whether probabilistic methods justify their computational overhead under noise, or tested whether findings generalize across corruption patterns and property types. The first half of this thesis therefore asks four questions. First, how much of the variance in predictive performance do representation and model architecture each explain, and does that division change for the performance retained under increasing label noise? Second, what does the choice of model provide under label noise, and how do deterministic and probabilistic models compare? Third, does the type of label noise matter, or only its quantity? To answer this, we generate and add label noise in several ways, both to clean data and to data with inherent experimental noise. Fourth, what is the impact of artificial label noise on a model's uncertainty estimates? In particular, does the estimated uncertainty rise with increasing label

noise, and can it identify which labels are corrupted? Together, these questions aim to identify what, if anything, makes a QSAR model robust to noise. To answer them we introduce NoiseInject, an open-source Python package for benchmarking noise robustness and uncertainty quantification.

Learning with noisy labels (LNL) covers methods for training models when some labels are wrong; noise mitigation, which removes, reweights or corrects labels, is one part of it. Of 221 configurations tested in an exploratory study carried out earlier in this project, the 20 that retained the least R^2 under noise all used a distance between molecules (Section 4.4.1).

We hypothesized that the fault lay in this distance. On a given dataset, most features of a general-purpose representation either do not vary or do not relate to the property, yet each adds a term to the distance, so together they dominate it (Chapter 4). We propose instead to compute distances over a hybrid representation: a fixed number of task-relevant features selected from several representations. A first version showed that the selected features also work as ordinary model inputs, so the question became how to choose them (Chapter 5).

The second half asks whether the representation itself can be adapted to the task:

1. How can the best set of features be selected for a given prediction task?
2. Do hybrid representations achieve higher noise robustness, measured as the percentage of performance retained, than their base representations?
3. Do the features selected for a hybrid representation help stabilize distance-based noise mitigation?
4. Does weighting features by their importance improve the construction of hybrid representations, model training and noise mitigation?
5. When sample removal is suboptimal, can sample weighting or label smoothing handle Gaussian label noise better?

1.1 Outline of the Thesis and Contributions

Chapter 2 introduces molecular representations, model families, molecular similarity and activity cliffs, noise in molecular data, the evaluation protocol, the seven noise conditions used in Chapter 3, and NoiseInject.

Chapter 3 studies the noise robustness of QSAR models, mainly on the gap between the highest occupied and lowest unoccupied molecular orbitals (HOMO–LUMO gap) in QM9, whose computed labels carry negligible measurement noise, and on three assay datasets: LogD, Caco-2 and hERG K_i. It addresses the four questions above, comparing seven noise conditions at a matched amount of noise and each model with its probabilistic counterpart. Model architecture matters more than representation for noise robustness, whereas the two matter about equally for R^2 under noise. The shape of the noise distribution makes little difference; noise shared within a scaffold family and labels clipped at an assay limit (censoring) do the most damage. Predicted uncertainty rises with noise on average but does not identify which labels were corrupted.

Chapter 4 motivates the later methods. It introduces no method of its own, and its measurements are on a smaller scale than those of Chapter 3. On a given dataset, standard representations carry many features that do not vary, and some map different molecules to the same vector. Similarity computed over these representations inherits both problems, and methods that rely on it fail as a result. These include a noise mitigation method, tested in the exploratory study, that cleans each label by comparing it with its nearest neighbors' labels. The chapter ends with what a representation would need to fix these problems, which Chapter 5 builds.

Chapter 5, planned as the first of two method papers, presents the adaptive hybrid representation. From each of several base representations, the method selects the features most important for the task, divides a fixed budget of features between them, and combines the selected features into one representation. The chapter also describes the datasets and setup. It will compare the hybrid with the best single representation, an ensemble of one model per representation, principal component analysis, minimum-redundancy maximum-relevance selection, recursive feature elimination and random features of the same size. It will also test whether the selected features are interpretable and stable, whether search settings transfer between datasets, and whether the method extends to other domains and to data held at separate sites. **[RESULT NEEDED: one sentence of findings for Chapter 5.]**

Chapter 6, planned as the second method paper, asks whether a hybrid representation repairs the failures described in Chapter 4. It compares standard and hybrid representations selected for predicting each molecule's activity from its five nearest neighbors, asks whether neighbors in a hybrid representation identify corrupted

labels better than neighbors in a standard one, and tests LNL methods, including noise mitigation, one noise type at a time. **[RESULT NEEDED: one sentence of findings for Chapter 6.]**

Chapter 7 summarizes each chapter and argues that the noise in a dataset should be modeled before a method is chosen, since no single method fits every dataset and simple methods are often as good as complex ones. It then gives limitations, future directions and conclusions.

The software contributions are NoiseInject and the KIRBy repository, which implements the search for hybrid representations and the experiments (Additional file S.6).

1.2 Collaborations

This work was supervised by Garrett Morris and co-supervised by Thierry Hanser and Stephane Werner at Lhasa Limited, which runs the federated learning platform Effiris [18]. Lhasa also supplied the hERG data used in Chapter 5, which we treat as data held at separate sites, as in federated learning [18, 19].

The Oxford Protein Informatics Group (OPIG) pointed us to the antibody and protein datasets used in Chapters 5 and 6; all datasets are listed in Additional file S.5. The collaboration also produced five posts on OPIG's blog, Blopig, listed under Blogposts in the Summary of Research Contributions.

The adaptive hybrid representation, its software and the experiments in Chapters 4 to 6 are the author's own work.

Chapter 2

Molecular Representations, Models and Label Noise

2.1 Overview

This chapter introduces the concepts to which every later chapter refers. It covers molecular representations, which turn a molecule into a fixed-length vector of features, the model families that learn from them, and the assumption that similar molecules have similar properties, together with the cases in which this assumption breaks down. It then turns to label noise, describing its sources in real assay data, the seven artificial noise conditions used in this thesis and the methods for LNL. The chapter closes with NoiseInject, the package that injects the noise and reports robustness and uncertainty metrics, and with the group-aware splits and metrics used throughout.

2.2 Molecular Representations in QSAR

The success of machine learning hinges on the quality of the input data. Although quality can refer to noise, on which most of this work centers, here we mean the type of input that the model receives. Molecular representations provide different features from which QSAR models and noise detection techniques can learn, and Figure 2.1 shows three of them for a single molecule. A potential bottleneck on larger datasets should nonetheless be noted, since the more complex the data structure, the greater the computational cost. On smaller datasets, the number of parameters is limited by the size of the training set in order to prevent overfitting.

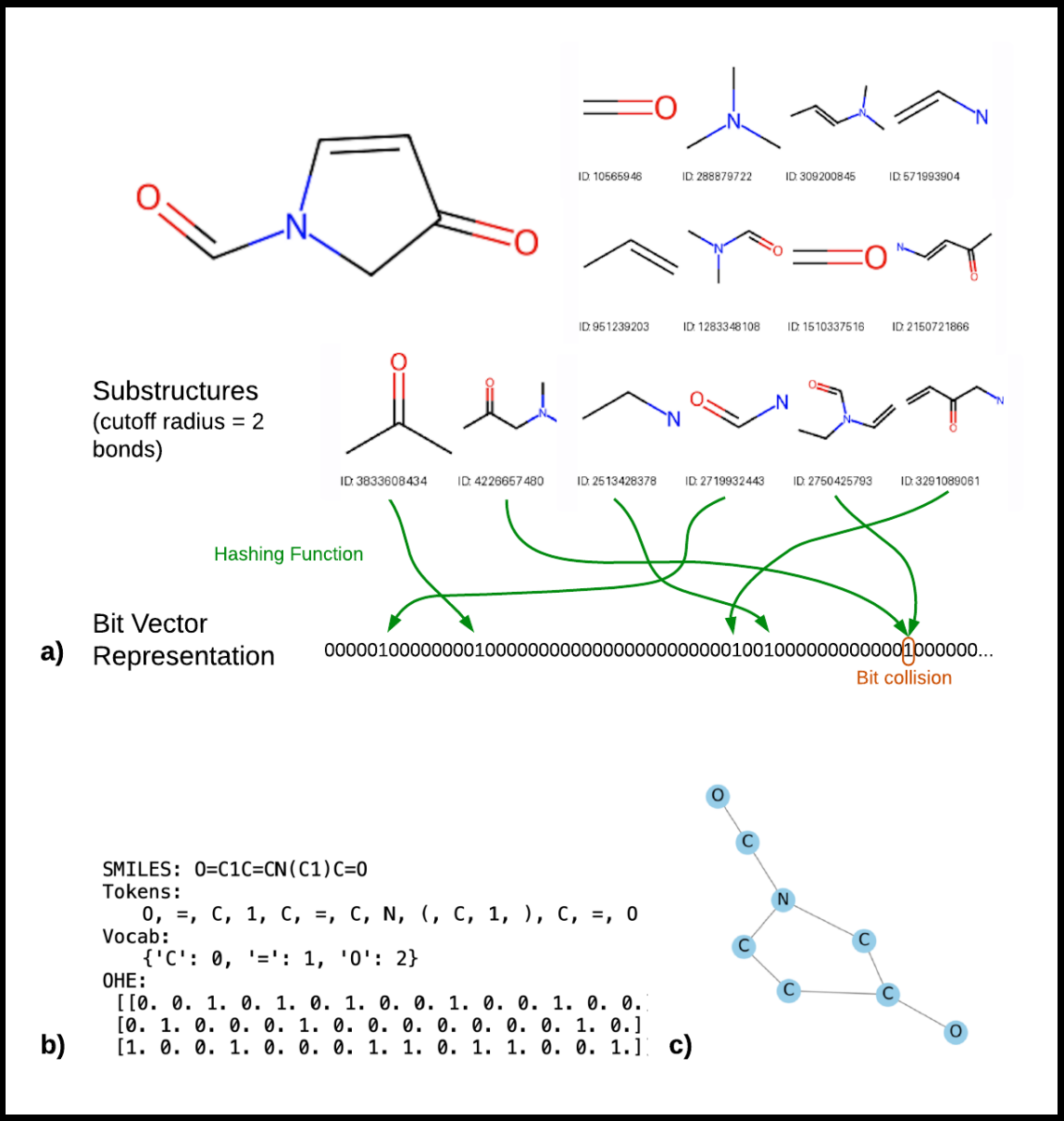


Figure 2.1: Three representations of the molecule O=C1C=CN(C1)C=O. a) ECFP4, in which circular substructures up to a radius of two bonds are hashed (green arrows) to positions in a bit vector, and two substructures can share a position (bit collision, orange). b) Tokenization and one-hot encoding of the SMILES string. c) Molecular graph, with atoms as nodes and bonds as edges.

One of the most common and intuitive molecular representations are Simplified Molecular-Input Line-Entry System (SMILES) strings which consist of a sequence of ASCII characters generated by a fixed set of rules to specify a given chemical identity [20]. A common way to use SMILES strings as inputs to QSAR models is to tokenize and one-hot-encode them rather than to use the strings themselves [21]. The strings are tokenized into atomic branches and symbols and mapped to indices of a vocabulary, and finally converted into binary vectors using one-hot encodings [21]. This representation, referred to as one-hot-encoded SMILES, allows models

to process molecular structures as bit-vector-based fingerprints or as sequences of vectors. Sequence-based models such as recurrent neural networks and transformers have proven to be particularly successful in capturing text semantics within a sequence. When applied to raw or one hot encoded SMILES strings, these models can effectively capture the structure of the molecule in its textual representation [21].

Another common representation is the molecular fingerprint, a vector containing information about the properties of the molecule in question. In a typical bit vector-based fingerprint, each bit accounts for the presence (1) or absence (0) of a given chemical feature or substructure [22]. One common fingerprint is the Molecular ACCess System (MACCS) key, a substructure key-based fingerprint that encodes predefined atom symbols and properties, as well as bond types [23]. Another is the Extended Connectivity Fingerprint (ECFP), which encodes local neighborhoods around each atom and bond connectivity in molecules [23]. ECFPs are typically constructed with a radius of 2, a length of 2048 bits, and active chirality flags. These existing molecular representations are well established and have been quite successful when coupled with QSAR models [3, 24]. They are relatively simple, and as the bits can be mapped back to one or more substructures, they are human-interpretable, and furthermore easy-to-compute. However, they are subject to information loss due to bit collisions, which happens when two different substructures are encoded at the same bit position [22]. They are also based on a fixed procedure and are not trainable or adaptable to a particular dataset. Alternative representations that capture different structural, chemical, or physical features are able to overcome these limitations.

Recent advances have led to the development of new molecular fingerprints. One such method is the Sort & Slice (SNS) technique, an alternative to hash-based folding for pooling ECFP substructures [6]. Sort & Slice begins by sorting all circular substructures in the training set according to their frequency of occurrence. It then "slices" away all but the N most frequent substructures, which are subsequently used to generate an N-dimensional, bit-collision-free binary fingerprint. This approach solves the ambiguity problem of hashed fingerprints, where multiple substructures might be encoded at the same bit position, making it very interpretable [6]. Sort & Slice fingerprints outperform ECFP of diameter four (ECFP4) with standard hash-based folding methods across various molecular property prediction tasks. These experiments consisted of different data splitting techniques, machine learning models, fingerprint diameters, fingerprint dimensions, and atomic invari-

ants [6]. Furthermore, Sort & Slice outperformed advanced substructure selection schemes such as filtered fingerprints and mutual information maximization [6].

Neural fingerprints are a category of learned representations that overcome the fixed, hard-coded nature of traditional fingerprints. Unlike ECFP or MACCS keys, which rely on predefined hashing schemes or substructure catalogs, neural fingerprints train on molecular data to capture chemical information [25, 26]. These representations are the result of deep learning architectures that have learned dense, continuous embeddings which can be used for downstream tasks.

Mol2vec, a common neural fingerprint, adapts the Word2vec algorithm from natural language processing (NLP) to molecular substructures [25]. It treats molecules as "sentences" and the circular substructures from which ECFPs are built as "words", and it learns an embedding for each substructure through unsupervised training, such that chemically similar substructures are mapped to nearby points in the embedding space [25]. The embeddings of all substructures present in a molecule are summed to give a 300-dimensional dense vector that represents the whole molecule [25]. Mol2vec has performed strongly on various QSAR benchmarks, matching or exceeding ECFPs on solubility and bioactivity prediction tasks [25].

Transformer-based architectures have also been applied to molecular representation learning, similarly stemming from NLP ideas. MolBERT applies a bidirectional transformer to learn molecular representations from SMILES strings through self-supervised pre-training [27]. The model has domain-relevant auxiliary adaptations, such as the prediction of calculated physicochemical properties alongside the masked token prediction, to improve the chemical fidelity of the learned embeddings [27]. MolBERT demonstrated improvements over traditional fingerprints and graph-based methods on Virtual Screening and QSAR benchmarks [27]. ChemBERTa is another transformer-based approach which pre-trains masked language models on SMILES data from PubChem [26]. ChemBERTa uses a tokenization strategy adapted for the SMILES notation, which allows the model to learn representations of atoms and functional groups through self-supervised learning [26]. Attention visualization reveals that ChemBERTa learns chemically meaningful patterns, with different attention heads specializing in functional groups, ring structures, or SMILES syntax [26]. ChemBERTa-2, an optimized variant pre-trained in 77 million compounds, achieved even stronger performance than the original ChemBERTa in MoleculeNet benchmarks [28].

Neural fingerprints provide several advantages over traditional fingerprints. They

produce dense continuous vectors that avoid sparseness and information loss from bit collisions [25]. They can also be pre-trained once on large unlabeled datasets and then fine-tuned for specific downstream tasks, opening the door to transfer learning [26, 27]. However, these methods are often computationally expensive [25] and due to their black-box nature, they are not as interpretable. [26]. However, attention mechanisms can have some degree of interpretability through visualization [26].

Beyond fingerprints, physicochemical-descriptor vectors (PDVs) are a class of continuous-value molecular descriptors that encode structural and chemical properties of molecules [29]. Unlike binary fingerprints or graph-based representations, PDVs aggregate atom and bond-level quantities into fixed-length feature vectors that capture a wide variety of information about a molecule and its structure [30]. PDVs typically consist of hundreds of properties including constitutional descriptors such as molecular weight, atom and bond counts, and heteroatom ratios; topological indices encoding molecular connectivity patterns; physicochemical properties such as logP (lipophilicity), molar refractivity, and topological polar surface area; and electronic descriptors derived from partial charges and polarizabilities [29, 31]. The `RDKit` cheminformatics toolkit provides a comprehensive suite of more than 200 molecular descriptors that cover these categories [32]. PDVs are very interpretable; each descriptor has a clear physicochemical meaning [31]. They are also less computationally expensive than some 3D methods or neural fingerprints, with many descriptors simply requiring a 2D molecular structure without conformer generation or extensive pre-training. This efficiency makes PDVs particularly appropriate for high-throughput applications. However, it is also possible to use descriptors based on 3D components [30].

However, PDVs have limitations. The dimensionality of PDV feature spaces can lead to problems with multicollinearity [29]. This can be mitigated by feature selection or dimensionality reduction techniques [31]. Additionally, PDVs encode molecular properties as aggregated global features, which may not capture fine-grained structural details such as specific substructure arrangements or local chemical environments, information important for particular endpoints [30].

One of the more intuitive representations are molecular graphs. These graphs can be categorized into 2D and 3D graphs. 2D graphs rely on adjacency matrices that encode topological atom- and bond-level features but leave out geometric properties such as bond angles or torsion, meaning they may fail to capture stereochemistry. 3D graphs incorporate these spatial features, but are often limited by remaining in

a single conformation [33]. A standard 3D molecular graph is constructed such that each molecule is represented as $G = (V, E)$ with node features $\mathbf{x}_i$ and edge features $\mathbf{e}_{ij}$. Nodes encode atom type, degree, hybridization, aromaticity, and formal charge; edges encode bond order and conjugation [33].

Another common molecular graph is the standard distance compatibility graph, in which atoms and bonds are mapped to sets of nodes and edges [5, 34]. These graphs are typically 2D objects that encode 3D information such as atomic coordinates, bond angles and chirality [5]. Alternative graph representations that incorporate 3D structure and physical interactions include graphs with quantum 3D embeddings, weight alignment graphs, association graphs containing pairs of molecules, and geometric interaction graph neural networks [34–36]. Du et al. [37] combined 2D and 3D graphs in a zipper-like arrangement, in which the 3D data guarantee precision and completeness by providing unabridged information and the 2D data provide chemical intuition for guidance. These representations pose various challenges, for example atoms localized at single voxels leave the points too sparse, which leads to poor performance. Kuzminykh et al. [38] addressed this problem by extending each atom to fill nearby voxels with a wave transformation.

These graph representations do not highlight specific substructures or specific arrangements of atoms and bonds within each molecule. The substructures contain functional groups that are related to a range of chemical properties and are potentially involved in downstream tasks related to the function of molecules. Bouritsas et al. [39] developed graph substructure networks, a topologically aware message passing scheme based on substructure encoding. The method involves encoding structural roles based on the appearance of specific substructures and using them in message passing updates. Since message-passing graph neural networks (GNNs) are at most as powerful as the Weisfeiler–Lehman test, they cannot isolate sufficient features from the graph structure [39]. Graph substructure networks, on the other hand, capture these substructure-related features. And recent research has highlighted the abilities of pretrained GNN encoders produce embeddings on unlabeled datasets that capture structural features [8, 40].

Kolmar and Grulke [9] examined the predictive limits of QSAR in the presence of noise and showed that, by learning the underlying trends rather than fitting the noise, models can make predictions that are more accurate than their own training labels. They assumed that experimental noise is Gaussian, arguing from the central limit theorem that measurements aggregated over multiple independent error

sources follow a Gaussian distribution. Heid et al. [41], however, used mean-variance estimation and bias-variance decomposition to show that noise can be tied to specific properties of the molecules, such as their structure. For example, nitrogen-containing functional groups introduce measurement deviations, which implies the presence of systematic noise.

2.3 Models for Molecular Property Prediction

One key issue with QSAR models is the standard assumption that labels represent true values. Most ML models rely on the assumption that the training data is, in fact, an accurate reflection of the population. Therefore, they treat these potentially noisy labels as discrete quantities rather than distributions, which opens the door to overfitting to noise and giving unduly optimistic assessments of the models.

There are a wide variety of model architectures that can be used for QSAR modeling. Starting off with simpler and more explainable models, we have tree-based methods. Decision trees are non-parametric supervised learning algorithms that perform recursive binary partitioning of the feature space [42]. A decision tree builds a predictive model by learning a series of if-then rules that recursively split the data based on given feature values, creating a tree where internal nodes represent decision rules, branches represent the results of those decisions, and leaf nodes contain predicted values or class labels [42]. Each recursive split is chosen to maximize homogeneity within the resulting partitions. For classification, Gini impurity is commonly used and for regression, mean squared error reduction [42].

There are several properties that make decision trees a good foundation for QSAR modeling. First, they handle non-linear relationships between molecular descriptors and target properties without requiring explicit feature engineering or transformation [42]. They also do not make assumptions about the underlying distribution of the data, making them robust to violations of parametric assumptions that can limit linear regression or other classical methods [43].

Decision trees handle data in a "contributory" rather than "consensus" manner. In consensus methods such as linear regression or neural networks, every data point influences the prediction at every other point in the feature space. Tree-based methods make predictions based on the subset of training samples that fall into the same leaf node as the test point [42]. This mechanism essentially quarantines outliers in their respective leaf nodes, limiting their impact on predictions for samples in other

regions or chemical domains.

Individual decision trees tend to have high variance and often overfit, particularly at large depths [42]. Small modifications made to the training data can result in large differences in the structure of the tree, resulting in unstable predictions [44]. Ensemble methods are used to counteract this instability, aggregating predictions from many trees. Random forests (RFs) aggregate predictions from trees trained on bootstrap samples of data with random subsets of features at each split [44]. Gradient boosting methods such as eXtreme Gradient Boosting (XGBoost) sequentially fit trees to the residuals of previous iterations, progressively reducing the prediction error [45]. Bootstrapping ensures that each tree is trained on a slightly different subset of data, which effectively reduce the influence of any individual sample [44]. This is especially beneficial with noisy samples. The selection of random features at every node decorrelates the trees, preventing systematic biases from propagating through the ensemble [44]. The aggregation of trees further prevents overfitting to noise by smoothing out prediction errors from individual noisy labels.

Neural networks (NNs), a very popular family of models, can also be used for QSAR. A NN is a system of interconnected nodes which learn by passing information through it's layers. There are weights and biases attached to the nodes and various other parameters which the model optimizes to minimize the difference between the predicted output and the actual target value. During training, the network parameters are updated using backpropagation, in which gradients of the loss function with respect to each weight are propagated backward through the network [46]. The structure of these networks and the hyperparameters used depend on the type of NN. In QSAR, one of the most commonly used architectures is the multilayer perceptron (MLP), a NN containing fully-connected layers with non-linear activation functions which transforms molecular feature vectors (i.e., fingerprints, descriptors) into a property or activity prediction [47].

For molecular property prediction, it is recommended that architectures use 2-3 hidden layers with 128-512 units each, rectified linear unit (ReLU) activations, and dropout regularization to prevent overfitting. Training is performed using backpropagation with gradient descent optimizers such as Adam, to minimize a loss function (mean squared error for regression, cross-entropy for classification) on batches of training data [48].

However, NN performance on QSAR-related tasks has been very mixed. They are often outperformed by more traditional methods: decision trees and related en-

semble models tend to perform stronger on smaller datasets, overfit less, and don't rely on a static 3D molecular structure like NNs do [49, 50]. Their complexity and use of gradients make them prone to label memorization [51], which in the context of noise is problematic, as it means that models may memorize the noise themselves. Although models that are memorizing labels may initially appear to have strong predictive performance, they cannot generalize out of distribution samples properly and often provide inaccurate uncertainty estimates. Various approaches have been proposed to improve neural network robustness to label noise, including noise-robust loss functions [51], confident learning to identify mislabeled examples [52], and regularization techniques such as mixup or label smoothing [51].

All the models up to this point have assumed a vector-based input. Molecular graphs, on the other hand, require different types of predict modeling. Two standard ML methods on molecular graphs include Graph Convolutional Networks (GCNs) and Graph Isomorphism Networks (GINs). GCNs apply the concept of convolution to graph-structured data. Instead of operating on grid-based data, such as images, GCNs aggregate information from neighbors of a node to update its representation [21, 53]. In contrast, GINs address the limitations in GCNs' ability to differentiate between non-isomorphic graphs. Unlike GCNs, which normalize and weight the sum of neighboring node features, GINs use a simple sum followed by a trainable MLP, giving the model flexibility to learn complex transformations of node features. The GIN model's design is informed by the Weisfeiler–Lehman test for graph isomorphism, enabling it to distinguish between molecular graphs that are structurally similar but differ in subtle ways [21, 53]. This makes GINs particularly suited to activity cliffs (ACs), where small variations in molecular structure can lead to significant changes in chemical or biological activity [21].

2.3.1 Tree Ensembles

RFs are among the most common choices for QSAR modeling because of their robustness and interpretability [43]. Quantile regression forests (QRFs) keep the distribution of training labels in each leaf and compute quantiles over all trees [54].

Gradient-boosted tree ensembles include XGBoost [55, 56], Light Gradient-Boosting Machine (LightGBM) [57], and NGBoost [58]. NGBoost extends deterministic gradient boosting by treating the parameters of a chosen parametric distribution as regression targets, which it learns through boosting with a natural

gradient update rule [58].

2.3.2 Support Vector Machines

Support vector machines (SVMs) are a well-established baseline in QSAR modeling [43, 59].

Support vector regression fits a function that is as flat as possible while keeping training errors small [59, 60]. It uses the ϵ-insensitive loss, under which errors smaller than ϵ cost nothing and larger errors cost their size minus ϵ. A constant C trades flatness against tolerance of large errors, and classification uses the same machinery with a maximum-margin boundary [61]. The features enter the model only through the kernel. With the radial basis function (RBF) kernel, the kernel is a function of the Euclidean distance, so every feature enters every distance, and we therefore standardize the features before fitting. Three properties of the method suggest robustness to label noise. Errors inside the ϵ tube change nothing, the loss outside the tube grows linearly rather than quadratically, and the weight of each molecule is capped at C [60]. Under simulated experimental error, the RBF-kernel SVM degraded smoothly and linearly [17], and Chapter 3 reports how it holds up under each of our noise conditions. Its main limitation is computational cost, because training operates on a kernel matrix that grows with the square of the number of training molecules [62].

2.3.3 Gaussian Processes

Gaussian processes (GPs) a non-parametric class of models that use a kernel to produce a Gaussian predictive distribution over every data point [63, 64]. One of the major limitations of using GPs in practice is computational scaling; inference has $O(N^3)$ time complexity and $O(N^2)$ memory, where N is the number of training points [64]. This can be mitigated with sparse GP approximations, but these introduce approximation error and can be more difficult to tune [65].

The `Gauche` framework [66] defines kernels for strings, fingerprints, and graphs that operate on a range of widely used molecular representations. The Tanimoto kernel [67, 68] is defined for binary vectors $\mathbf{x}, \mathbf{x}' \in \{0,1\}^d$ for $d \geq 1$ as:

$$k_{\text{Tanimoto}}(\mathbf{x}, \mathbf{x}') := \sigma_f^2 \cdot \frac{\langle \mathbf{x}, \mathbf{x}' \rangle}{\|\mathbf{x}\|^2 + \|\mathbf{x}'\|^2 - \langle \mathbf{x}, \mathbf{x}' \rangle}, \tag{2.1}$$

where $\|\cdot\|$ represents the Euclidean norm [66].

The RBF kernel is defined for $\mathbf{x}, \mathbf{x}' \in \mathbb{R}^d$ as:

$$k_{\text{RBF}}(\mathbf{x}, \mathbf{x}') := \sigma_f^2 \cdot \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\ell^2}\right), \tag{2.2}$$

where ℓ is the lengthscale.

2.3.4 Bayesian Neural Networks

NNs are often deterministic; however, they can be transformed into probabilistic models (Figure 2.2).

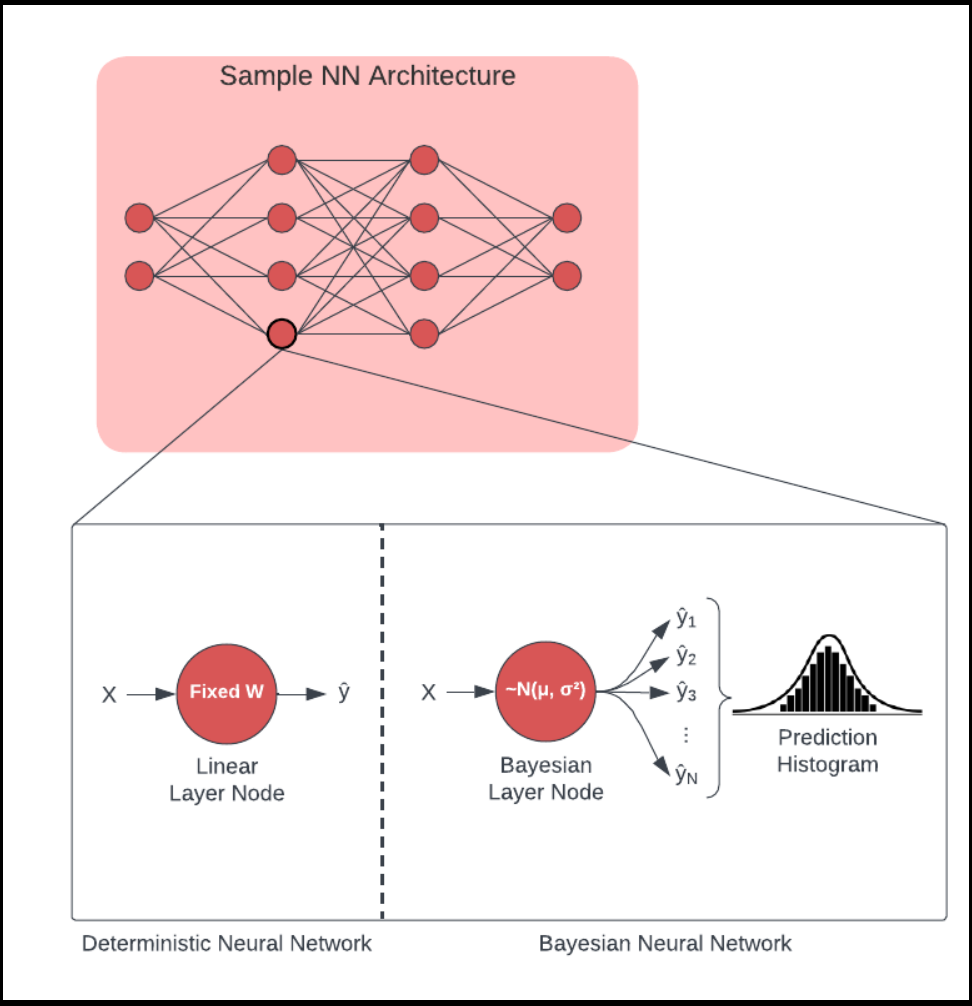


Figure 2.2: One node of a deterministic and of a Bayesian neural network. The deterministic node (bottom left) applies a fixed weight W and returns one prediction $\hat{y}$. The Bayesian node (bottom right) draws its weight from $\mathcal{N}(\mu, \sigma^2)$, so repeated predictions $\hat{y}_1, \ldots, \hat{y}_N$ form a histogram.

Unlike standard deterministic ML methods, Bayesian methods model distributions over predictions using Bayes' rule, allowing for the quantification of uncertainty

within parameters and labels. Bayesian methods treat the model parameters as random variables with distributions. Uncertainty can also be estimated from other non-deterministic methods, including deep ensembles [69], bootstrap modeling [70], and federated learning [19].

Epistemic uncertainty is model-driven and can be reduced by collecting either more or higher quality training data. Aleatoric is data-driven and reflects the intrinsic noise in the labels.

2.4 Molecular Similarity and Activity Cliffs

Distance-based reasoning rely on the assumption that the data embeddings contain quality information. When the labels are corrupted or the important features are scarce, similarity scores over-rely on compounding false or useless information, and the assumption that similar molecules have similar properties breaks down. Bajusz et al. [71] found that while the Tanimoto coefficient can be effective with binary fingerprints, changing the similarity formula yields minimal improvement if the underlying representation is weak. This problem is further complicated by ACs, which are defined as structurally similar molecules with large property differences, often due to a subtle chemical change [72, 73]. These molecules, which potentially contain the most important information for model prediction, can be confused with noise. Another barrier to distance-based reasoning is persistent noise across labels. The task of trying to determine if a molecule's value matches that of its neighbors becomes a lot harder when all of the neighbors have some form of noise as well.

Much of cheminformatics rests on the similar property principle, which states that structurally similar molecules tend to have similar properties [74, 75]. One of its clearest applications is nearest-neighbor prediction, which appears across cheminformatics workflows and, in this thesis, in the kernels used by GPs and at the basis of many noise identification algorithms.

The difficulty with this principle is that similarity is defined only relative to a similarity measure, which is fixed by the choice of representation and coefficient [76]. There is no standard definition of similarity between molecules, especially when different molecular embeddings are handled in a computational setting. Two molecules that are near-identical under a substructure fingerprint may be far apart under a PDV, and Maggiora et al. [76] caution that quantified similarity can be over-interpreted in this way.

The most common measure for binary fingerprints is the Tanimoto coefficient [71]. For two molecules with fingerprints $\mathbf{x}, \mathbf{x}' \in \{0,1\}^d$,

$$T(\mathbf{x}, \mathbf{x}') = \frac{\sum_{j=1}^{d} x_j x'_j}{\sum_{j=1}^{d} x_j + \sum_{j=1}^{d} x'_j - \sum_{j=1}^{d} x_j x'_j}, \tag{2.3}$$

which is the number of bits set in both molecules divided by the number of bits set in either. Continuous representations are instead compared with a distance, most often the Euclidean distance $\sqrt{\sum_{j=1}^{d}(x_j - x'_j)^2}$. Both measures are sums over the features, and neither weighs a feature by its relation to the property. In the Tanimoto coefficient, a bit that is off in both molecules contributes nothing, whereas every bit that is set in either molecule counts. In the Euclidean distance, every feature that differs between the two molecules counts. Chapter 4 returns to this point (Equation 4.3).

There are many downstream tasks which rely on the concept of similarity. The second half of this thesis relies on many such tasks, and as such focuses on identifying representations that agree with this principle.

The sharpest failure of the similar property principle is the activity cliff, a pair of structurally similar molecules with a large difference in potency that is produced by a small chemical change [72, 77]. Activity cliffs inherit the measure dependence of the principle they violate, and there is no widely accepted definition of one [75]. The MoleculeACE benchmark used in this thesis makes this choice explicit. A pair is considered highly similar if its similarity exceeds 90% under at least one of three measures, namely the Tanimoto coefficient on ECFPs, the Tanimoto coefficient on scaffold fingerprints and a scaled Levenshtein distance on SMILES strings. Such a pair forms a cliff if the two molecules also differ tenfold or more in K_i or EC_{50} [75]. This soft consensus over three measures is used because each measure captures only part of the underlying chemistry. Across the thirty MoleculeACE targets, cliff compounds account for between 7% and 52% of a dataset.

Dimensionality itself poses a further difficulty. Aggarwal et al. [78] show that as the number of dimensions grows, the distances to the nearest and to the furthest point become increasingly similar, so that pairwise distances become progressively less discriminative. Molecular representations are routinely high-dimensional and sparse, and when a large proportion of their dimensions carry no information relevant to the property being modeled, distances computed over them are dominated by

those uninformative dimensions.

2.5 Noise in Molecular Data

Real-world cheminformatics datasets contain label noise arising from multiple sources across the drug discovery pipeline, and modeling experimental noise properly requires first understanding where it comes from.

Studies involving noise, and specifically the quantification of noise robustness, typically approach label noise by adding independent, zero-mean, Gaussian error to the labels. However, there's evidence to suggest that noise from experimental assays takes on different forms. Krüger and Overington [79] rejected normality of bioactivity differences by Anderson-Darling test and fitted a Laplace distribution instead. Kalliokoski et al. [14] examined the widest disagreements among 16,844 pairs of repeat pIC_{50} measurements and found that in almost all cases, the wide gap between repeat values was due to annotation error rather than to an imprecise assay.

Beyond just the shape, the assumptions of independence and zero mean may fail as well. Bentz et al. [80] examined P-glycoprotein transport measurements across 23 laboratories and found that 62% of the variance in log efflux ratio was due to differences between laboratories. When you pool data from different sources, each source has the potential to carry a shared offset. These sources usually contain families of related compounds, breaking the independence assumption. The zero-mean assumption is not assured either, especially when true values fall above or below an assay's detection limit, called censoring. Svensson et al. [81] count censored labels in fifteen industrial assays, of which thirteen carry censored labels and eight sit between a quarter and two thirds censored. In this study, we tested noise conditions that vary the shape, the correlation, and the bias of injected error separately at a matched quantity.

Label noise enters bioactivity data at every step from the assay plate to the database. Table 2.1 lists these sources, with examples of each and the kind of error each tends to produce. The kind of error describes how the error is distributed across molecules, which is the property that the noise conditions in Section 2.5.1 vary.

These sources of noise can interact with one another and produce errors that a simple Gaussian does not describe. Unlike artificially injected noise, real-world

Source	Example	Kind of Error
Assay	Edge effects at plate boundaries from evaporation or temperature gradients [82]; carryover between wells, incomplete liquid transfers and air bubbles [83]; reagents or compounds degrading during a run [84]	Error that varies with plate position or read time
Instrument	Higher relative error near a detection limit [85]; fluorescence quenching by the test compound [86]; background subtraction in radiometric assays [85]	Larger error for weak binders; a systematic bias; labels clipped at the limit
Biology	Seeding density, growth rate, cell cycle and serum properties [87]; passage speed [88]	Variation between repeats of the same experiment
Aggregation across sources	ChEMBL, BindingDB and PubChem pool hundreds of laboratories [89, 90]; K_i or IC_{50} values for one compound–target pair spanning two orders of magnitude [15]	An offset shared by each laboratory or source, hard to separate from the true value without metadata [91]
Federated learning	Teacher models label a shared public set, and a student model learns from those labels [18]	The teachers' prediction errors on top of the measurement error
Annotation	Mis-registered structures, wrong stereochemistry and salt forms [92]; 1–5% structure errors in public databases [93]; ambiguous protonation states and counterions [94]	A label attached to the wrong molecule; distinct molecules merged

Table 2.1: Sources of label noise in bioactivity data, with reported examples and the kind of error each source produces.

noise is neither independent, identically distributed, nor symmetric [41]. We address this complexity by examining several ways of both modeling noise and adding it to datasets.

Although the general assumption in many of the noise analysis and mitigation literature is that noise, specifically label noise, will have a negative effect on the model, this is not necessarily the case. There are multiple instances of using noise as a method of regularization. Although not the optimal approach to counteracting overfitting in linear regression, adding artificial noise to labels is considered a form of regularization [95]. Furthermore, adding artificial noise drawn from a Gaussian distribution to the activation layers of a neural network is considered another form of regularization [96]. These studies indicate that the presence of noise may act as a form of regularization in some contexts. It is important to address this potential of regularization from label noise in QSAR and quantitative structure–property

relationship models.

2.5.1 Artificial Noise Injection

Our objective is to evaluate the robustness of QSAR models against label noise. To do so, we "inject" artificial noise into the labels during training while validation and test data remain free of noise. Validation labels are given noise of the same type and quantity as the training labels, drawn independently. This artificial noise simulates real-world experimental noise. The clean target label y_i is replaced by a noisy label $\tilde{y}_i = y_i + \epsilon_i$, where ϵ_i is the artificial noise injected at the index i, the value of which is determined by a specified noise strategy. As we increased the quantity of noise, we measured how much predictive performance fell.

We explored seven ways of adding noise, which we refer to as noise conditions and which are illustrated in Figure 2.3 and summarized in Table 2.2. Three conditions vary the shape of the distribution from which the error is drawn, and three concentrate the error on some molecules or share it within Bemis–Murcko scaffold families. The seventh condition, censoring, clips labels at a limit.

For a given label, every condition except censoring delivers the same expected quantity of noise at a set noise level. These levels are defined as a fraction of the standard deviation of clean training labels. The noise added to the label of molecule i is

$$\epsilon_i = \frac{\tau}{G}\, s_i z_i, \qquad G = \sqrt{\frac{1}{n}\sum_{j=1}^{n} s_j^2}\;\; \sigma_z, \tag{2.4}$$

Here τ is the target standard deviation of the noise in the units of the label, and z_i is drawn from the distribution of the condition at unit scale. The multiplier s_i equals 1 for most molecules and $\lambda = 3$ for the molecules that the condition affects, which are a random 10% of molecules under the Outlier condition and whole scaffold families covering 20% of the split under the Grouped-wider condition. The constant σ_z is the standard deviation of z, which equals 1 for the Gaussian, $\sqrt{\nu/(\nu-2)}$ for the Student-t and $\sqrt{2}$ for the Laplace distribution. The factor G divides out both of these quantities, so that ϵ has a root-mean-square value of τ in expectation, whatever the distribution and whichever molecules are affected. This matching is the new element of our design. Most noise-robustness studies vary only the amount of Gaussian noise, whereas here every condition except censoring delivers the same root-mean-square noise at each level, so that the conditions differ only in how the

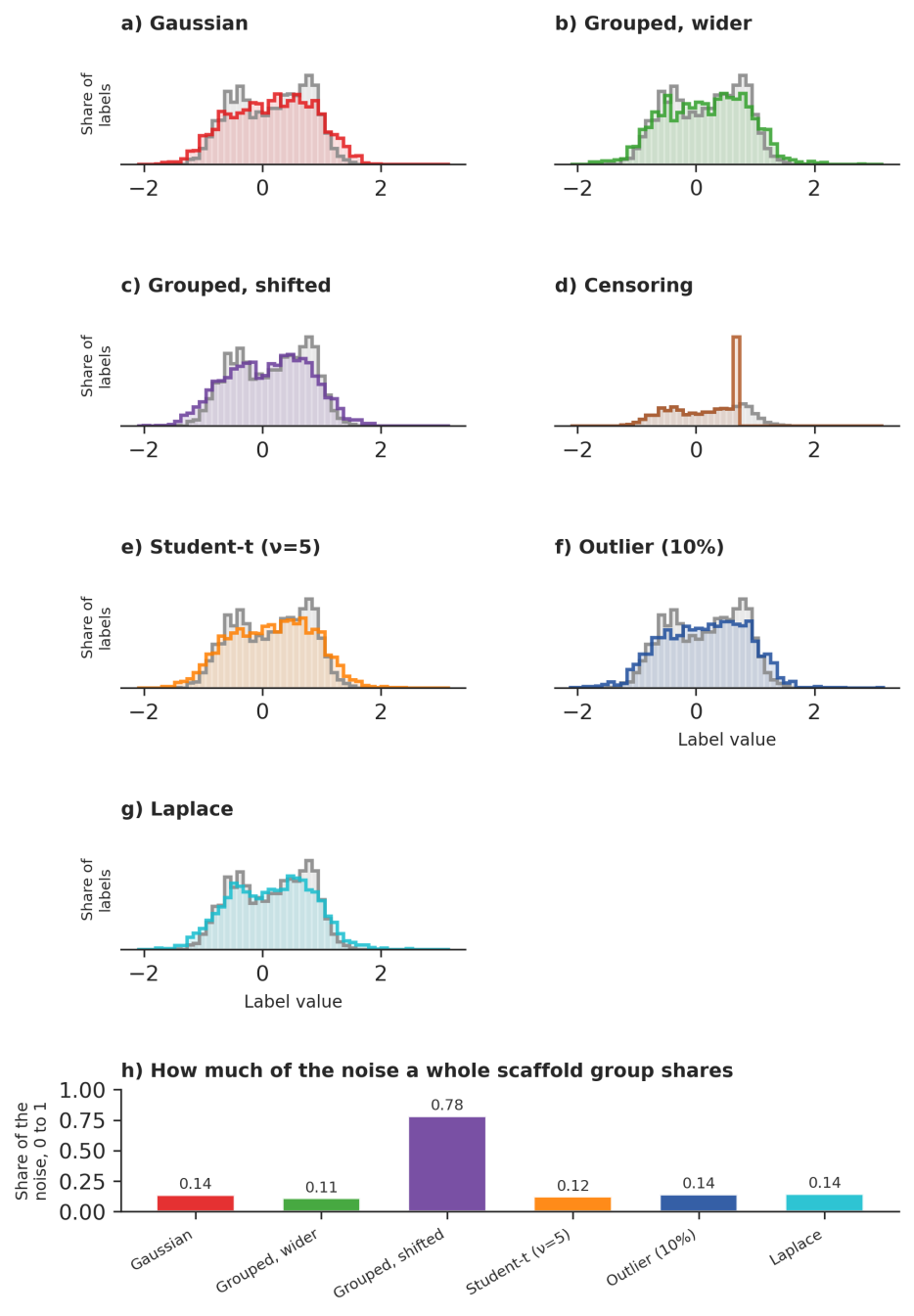


Figure 2.3: Label distributions under the seven noise conditions, for 2,000 synthetic labels in 40 random groups that stand in for scaffold families. a)–g) Clean (gray) and noised (color) labels at a noise level of 0.5 times the standard deviation of the clean labels, with 25% of labels clipped under censoring. h) Standard deviation of the mean error within a group divided by that of all errors. Noise that ignores the groups gives about 0.14, and censoring is omitted.

noise is spread across molecules.

The Grouped-shifted condition widens no molecule's error and instead gives every scaffold family g an offset of its own,

$$\epsilon_i = \frac{\tau}{G}\left(\sqrt{\rho}\, b_{g(i)} + \sqrt{1-\rho}\, w_i\right), \tag{2.5}$$

with one draw b per family and one draw w per molecule, where the two variances sum to τ^2 and the family offset carries a share ρ of that total.

We selected the parameter values from published measurement errors wherever possible, and Table 2.2 notes where no published estimate exists.

Condition	**What It Changes**	**Value**	**Source for the Value**
Gaussian	Shape: every label's error is Gaussian	–	The standard assumption in noise-robustness studies
Student-t	Shape: heavier tails than Gaussian	$\nu = 5$	Distributions fitted to repeated bioactivity measurements [79]
Laplace	Shape: sharper peak and heavier tails than Gaussian	–	As for Student-t [79]
Outlier	Error λ times wider on a random share of molecules (Huber's contamination model [97])	10% of molecules; $\lambda = 3$	1–10% of routine data contain gross errors, upper bound taken [98]; inter-laboratory error is about three times intra-laboratory error [99]
Grouped-wider	Error λ times wider on whole scaffold families	Families covering 20% of molecules; $\lambda = 3$	No published estimate of the share; λ as for outlier [99]
Grouped-shifted	Every scaffold family gets its own offset; no error is widened	$\rho = 0.62$	Laboratory identity explains 62% of the variance in Caco-2 across 23 laboratories [80]
Censoring	Labels past a limit are set to the limit; the level is the fraction clipped	Set by the noise level	Censored labels in industrial assays [81]

Table 2.2: The seven noise conditions and their parameter values. ν is the number of degrees of freedom of the Student-t distribution, λ the widening factor (Equation 2.4) and ρ the share of the noise variance carried by the scaffold-family offset (Equation 2.5). For censoring, the noise level is the fraction of labels clipped.

2.5.2 Learning with Noisy Labels

LNL covers every method for training a model when some of its labels are wrong, and we divide it into two parts. Noise mitigation changes, drops or reweights the training labels once, before any model is trained, so that any model can then be fitted to the result. Its methods include removal, down-weighting, smoothing, relabeling and correction, and Figure 2.4 shows how such a method is evaluated. Noise-robust training instead leaves the labels unchanged and alters how the model trains, through robust losses, noise-modeling layers, regularization, or reweighting inside the training loop, as in co-teaching and Meta-Weight-Net. A sample-selection step that is taken out of its training loop and used only to delete molecules before a refit counts as noise mitigation. Noise detection and noise estimation, which find noisy labels or estimate how many there are, feed into noise mitigation but are not LNL methods themselves. Song et al. [100] sort 62 deep-learning methods into five groups, namely robust architecture, robust regularization, robust loss function, loss adjustment and sample selection.

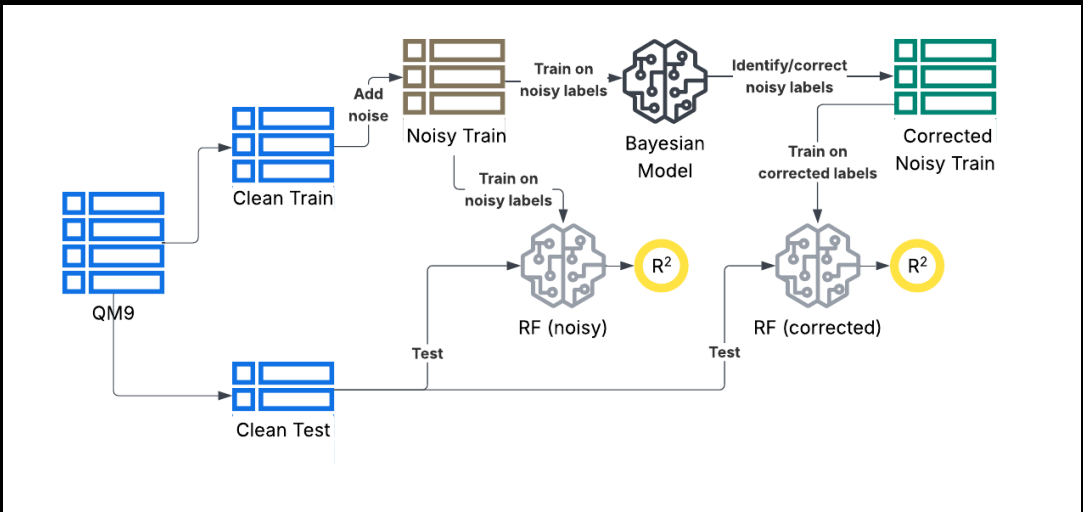


Figure 2.4: Evaluation of a noise mitigation method, with QM9 as the example. Noise is added to the training labels only. A random forest trained on the noisy labels is compared with one trained on labels corrected by a Bayesian model, and both are scored by R^2 on the clean test set.

No single method is expected to win, since averaged over all problems no learning algorithm outperforms another [101, 102]. The analog in chemistry is a study of twelve algorithms on twelve bioactivity datasets with simulated experimental errors [17]. Neighbor-based cleaning dates back to editing a training set by its nearest neighbors [103], and it relies on similar molecules having similar labels, an assumption that activity cliffs break. QSAR models also frequently fail to predict activity

cliffs [73], and Chapter 6 tests this assumption.

2.6 NoiseInject: A Benchmarking Framework

The noise conditions described above are implemented twice. The three experimental datasets and every uncertainty result use NoiseInject, an open-source Python package, whereas QM9 uses a separate pipeline, written in Python and Rust, that carries its own Rust implementation of the same conditions. An automated check compares the two implementations before any experiment is run on the computing cluster. Because the random number generators of Rust and NumPy produce different streams, the check compares statistics rather than individual draws. On the same labels, groups and seeds, it compares the delivered noise, the factor G, the median error, the number of badly corrupted labels and the number of molecules affected.

2.6.1 NoiseInject

NoiseInject (version 1.0.0) evaluates model robustness and uncertainty calibration under artificial label noise, for both regression and classification. It operates on arrays of labels and on the predictions of any externally trained model, which makes it model-agnostic, and its core depends only on NumPy, pandas, SciPy and scikit-learn. Results are returned as `pandas` data frames with one row per noise level and one aggregate summary row. The software is available under an MIT license and archived on Zenodo [104].

Regression noise. The regression injector specifies a noise shape (Gaussian, Student-t or Laplace) and a targeting rule (Uniform, Grouped-wider, Grouped-shifted, Outlier or Censoring) separately. Each named condition in the package is one combination of the two, together with its parameters, and this name is recorded with every result. The package defines more settings than this study uses, namely Student-t distributions with $\nu \in \{3, 5, 10\}$ and outlier contamination of 1%, 5% and 10% of molecules. It also defines an outlier variant in which the unselected molecules receive no noise at all, rather than the base-scale noise of Equation 2.4.

The requested noise level is the root-mean-square noise τ to be delivered, in the units of the label. The injector computes the per-molecule multipliers s_i and solves

for the scale τ/G in closed form (Equation 2.4), so no calibration search is needed. A level of zero returns the labels exactly unchanged. The affected molecules are selected with a second random number generator, which is seeded separately from the noise draws and re-seeded on every call. As a result, the affected molecules are the same at every noise level, including zero, while the noise draws themselves change. Under the Grouped-wider condition, whole scaffold families are added in random order until the fraction of affected molecules is as close as possible to the target of 20%, because a fixed fraction of families gives a very different fraction of molecules when family sizes are uneven.

The Censoring condition is parameterized by the fraction f of labels clipped, and its limit is the $(1 - f)$ quantile of the clean training labels. On the training split, the censored set is therefore exactly the $k = \mathrm{round}(fn)$ largest labels, whereas on any other split it consists of whichever molecules lie past the training limit.

Alongside the noisy labels, each injection returns a record of how they were produced. This record contains the target amount of noise, the factor G, the solved scale, the delivered root-mean-square noise (in label units and as a fraction of the clean label standard deviation), the mean of ϵ, the fraction of molecules affected, the number of groups and the largest share held by one group, and the random seeds. The injector also checks the delivered amount of noise. Because the delivered amount is an estimate from a finite sample, it is compared against a band of three standard errors,

$$\left|\frac{\hat{\tau}}{\tau} - 1\right| \le \max\left(3\sqrt{\frac{\kappa - 1}{4\, n_{\mathrm{eff}}}},\ 0.005\right), \qquad n_{\mathrm{eff}} = \frac{\left(\sum_i s_i^2\right)^2}{\sum_i s_i^4}, \tag{2.6}$$

where $\hat{\tau}$ is the delivered root-mean-square noise and κ is the sample kurtosis of the unscaled draws z, bounded between 3 and 60. The effective count n_{eff} is the number of independent contributions over which the delivered amount of noise averages, and it falls below n when the noise is concentrated on a few molecules. Under the Grouped-shifted condition it is $n_{\mathrm{eff}} = \left(\rho^2/g_{\mathrm{eff}} + (1-\rho)^2/n\right)^{-1}$, with $g_{\mathrm{eff}} = \left(\sum_g n_g\right)^2 / \sum_g n_g^2$ for family sizes n_g. At $\nu \le 4$ the Student-t distribution has no fourth moment, so the standard error is instead $1.3\, n_{\mathrm{eff}}^{2/\nu - 1}$, with the constant calibrated on the measured spread. A draw outside the band raises a warning rather than an error. At three standard errors, a correct injector falls outside the band on about 1% of draws, and the delivered amount is recorded with the result in either case.

A second function of the package returns the noise scale that each molecule's region would receive, without drawing any noise. We use it to score held-out molecules against the pattern of noise to which the training labels were exposed. Under the Uniform and Grouped-shifted targeting rules this scale is the same for every molecule, so the package flags the result as degenerate, because a rank correlation against a constant is undefined and must not be reported as zero.

A separate injector sizes the noise against a unit supplied by the user rather than against the label standard deviation. In the experiments implemented in the KIRBy repository, this unit is the standard deviation of a model's out-of-fold residuals on the clean training labels, so that the noise is sized against the error the model already makes. Its conditions either add Gaussian noise with variance equal to the level times the square of the unit, or move 2%, 5% or 10% of labels by the level times the unit, with a random sign or always upward. They can also add Gaussian noise whose standard deviation follows either a per-molecule covariate or a hidden log-normal draw, and both of these are scaled to the same mean variance as the first condition.

Classification noise. The package provides six label-flipping strategies, namely uniform flips, flip rates set by class frequency, asymmetric flip rates for binary tasks, a random flip rate for each sample, a separate flip rate for each class, and flips directed by a confusion matrix. Because the realized flip rate differs from the nominal flip probability under most of these strategies, a binary search finds the flip probability that produces a target fraction of changed labels.

Robustness metrics. For regression, the package computes R^2, the root mean squared error (RMSE) and the mean absolute error at every noise level. For classification, it computes accuracy, together with macro- and weighted-averaged precision, recall and F1 score and a per-class breakdown. Robustness is read from the retention curve $\mathrm{ret}(x) = m(x)/m(0)$, where m is a bounded metric for which higher values are better (R^2 for regression; accuracy, F1, precision or recall for classification) and x is the noise level. The primary summary is the normalized area under this curve,

$$\mathrm{AUC}_{\mathrm{norm}} = \frac{1}{x_{\max} - x_{\min}} \int_{x_{\min}}^{x_{\max}} \mathrm{ret}(x)\,\mathrm{d}x, \tag{2.7}$$

evaluated by the trapezoid rule. It is the mean fraction of clean performance retained across the noise levels, lies approximately within $[0, 1]$, and does not depend on how well the model performed on clean labels. A secondary summary fits $\mathrm{ret}(x) \approx \exp\bigl(-(x/\tau_W)^\beta\bigr)$ to the curve. The shape parameter β separates curves that hold and then collapse ($\beta > 1$) from curves that fall early and then plateau ($\beta < 1$), while τ_W sets the scale of the decay, and the fit requires at least four noise levels. These two summaries replace the linear slope of performance against noise used in earlier versions, which misdescribed a nonlinear curve and was coupled to clean performance. The package also reports the clean baseline and the retention at the highest noise level. A flag marks any curve that falls below zero retention, and an optional threshold withholds the curve summaries for models whose clean baseline is too low for a meaningful degradation.

Uncertainty metrics. For a model that returns a predicted standard deviation u_i for each sample, the package computes several metrics at each noise level. These are the Spearman correlation between u_i and the absolute error $|y_i - \hat{y}_i|$, the empirical coverage $\frac{1}{n}\sum_i \mathbb{1}[|y_i - \hat{y}_i| \le k u_i]$ at $k = 1$ and $k = 2$ against Gaussian targets of 68.3% and 95.4%, the mean interval width $\frac{1}{n}\sum_i 2u_i$, and the expected calibration error (ECE),

$$\mathrm{ECE} = \sum_{b=1}^{10} \frac{|B_b|}{n} \left| \bar{u}_b - \overline{|y - \hat{y}|}_b \right|, \tag{2.8}$$

computed over ten equal-count bins B_b of u. All of these metrics are computed on the clean test labels. The Spearman correlation between u_i and the injected noise $|\epsilon_i|$ can only be computed where the noise is known, which is on the corrupted molecules, so it requires a separate pair of inputs. It is reported within each noise level, and a correlation pooled across all levels is returned separately. The pooled value mostly reflects the fact that uncertainty and noise both rise with the level, so it does not measure whether a model can identify which molecules are corrupted.

Three optional wrappers produce u_i from a point predictor. The first applies split-conformal prediction to any scikit-learn-compatible model, optionally scaled by the label spread of each molecule's nearest training neighbors. The second fits an exact Gaussian process with a Tanimoto or RBF kernel through `Gauche` [66], and the third applies Monte Carlo dropout to any `PyTorch` network [105]. Example notebooks and scripts cover regression and classification from start to finish.

2.7 Evaluation Methodology

RMSE and the coefficient of determination (R^2) were primarily used to compare QSAR predictive performance.

When evaluating uncertainty, we assessed the relationship between predicted uncertainty and prediction quality. Spearman's rank correlation coefficient (ρ) was used to quantify the relationship between predicted uncertainty (u_i) and absolute error ($|y_i - \hat{y}_i|$), as well as between predicted uncertainty and injected noise magnitude. Higher correlations indicate that the model's uncertainty estimates reliably identify predictions that are likely to be incorrect.

For probabilistic models, the empirical coverage at 1σ and 2σ and the ECE (Equation 2.8) are defined in Section 2.6.1, together with the other NoiseInject uncertainty metrics.

2.8 Conclusions

This chapter has introduced both halves of a predictor, namely the representation and the model. Because real assay noise is not independent, identically distributed or symmetric, our artificial noise model comprises seven conditions. We have also set out the methods for LNL, of which noise mitigation is the subset that acts on the labels, and noted that no single method wins on every dataset. NoiseInject injects these noise conditions and reports the robustness and uncertainty metrics, and the section on evaluation methodology fixes the splits and metrics. Chapter 3 asks whether the representation or the model architecture governs how strongly a predictor degrades under label noise, through the four questions set out in Chapter 1. The representations, similarity definitions and noise conditions introduced here are reused in Chapters 4 to 6.

Chapter 3

Noise Robustness Across Molecular Representations and Model Architectures

3.1 Introduction

While there have been many investigations into the effect of noise on molecular property prediction, direction comparisons between how different molecular representations responds to label noise remains limited. The choice of molecular representation certainly affects predictive performance. Deng et al. [106] evaluated 62,820 models and found that ECFPs frequently outperform learned graph embeddings across models. Whether these conclusions translate to noise robustness is our question. Between different types of label noise, Heid et al. [41] found no difference in model performance between error distributions with the same mean and standard deviation. We expand on this by considering alternative patterns of noise, namely noise concentrated in structural groups or on a sparse subset of samples.

This chapter addresses the first four questions set out in Chapter 1, namely how representation and model architecture divide the variance in predictive performance and in noise robustness, what probabilistic models add, whether the type of label noise matters, and whether uncertainty estimates track label noise.

3.2 Methods

We used four datasets, namely QM9 with the HOMO–LUMO gap as the prediction target and three assay datasets for LogD, Caco-2 and hERG K_i. On each dataset we combined six representations with 20 model configurations, which comprise thirteen base models and seven variant models. Every pairing of representation and model was exposed to seven noise conditions (Section 2.5.1) at seven noise levels, and the hyperparameters of every model are given in Additional file 1.

3.2.1 Datasets

Although we are investigating the effects of label noise stemming from experimental settings, for the purposes of testing we inject controlled amounts of artificial noise into clean labels. Data were extracted from the QM9 molecular property dataset distributed by PyTorch Geometric [107], which contains 130,831 small organic molecules with pre-computed quantum-mechanical properties obtained from density functional theory calculations at the B3LYP/6-31G(2df,p) level of theory. We selected the HOMO–LUMO gap as the prediction target in QM9, as it captures a molecule's electronic excitability, charge transfer capability, and chemical stability [108]. We assumed that QM9 labels were free of noise, notwithstanding approximations in the level of theory. Each experiment was replicated ten times with a distinct random seed and a randomly selected subset of $N = 10{,}000$ molecules, split 80/10/10 into 8,000 training, 1,000 validation and 1,000 test molecules. All models were fitted on those 8,000 training molecules, except the Gaussian processes, which were fitted on a random subset of 5,000 of them; the validation and test molecules were unchanged.

We repeated these experiments on three molecular property datasets with experimentally determined endpoints. From the OpenADMET initiative [109], we used LogD and Caco-2 efflux permeability, with $N = 5{,}039$ and $N = 2{,}161$ respectively. We also selected hERG K_i data from ChEMBL [110], following a protocol inspired by Landrum and Riniker [16]: filter for binding assays, deduplicate by median pChEMBL value, and remove compounds with an inter-assay standard deviation greater than 1.0 log unit, resulting in $N = 1{,}415$ compounds. Each of the three was partitioned into five folds by grouped k-fold cross-validation, and each fold's training block gave up a further grouped fifth as a validation set for early stopping. A model therefore fits between 3,194 and 3,364 of the 5,039 LogD molecules, de-

pending on the fold. Unlike QM9's ten replicates, which are ten independent draws from one pool, each assay dataset is a single fixed set in which every molecule is tested exactly once and nothing is redrawn.

All splits and cross-validation folds were grouped by the chirality-blind Bemis–Murcko scaffold, implemented such that each distinct acyclic molecule forms its own group, and groups are filled in random order. Prior to modeling, all molecular structures were sanitized and canonicalized using `RDKit` to remove invalid valence states and standardize atom and bond typing [32]. Prediction targets were standardized to zero mean and unit variance. Experiments that track uncertainty were run once rather than in replicates.

3.2.2 Representations

In this study, we evaluated six molecular representations: ECFP4, Avalon, Sort & Slice, PDV, ChemBERTa and Molecular Hypergraph Grammar GNNs (MHG-GNNs). The same six were used on QM9 and on the three assay datasets. ECFP4 is an ECFP with radius $r = 2$ with $d = 2048$ bits [4, 32]. Sort & Slice takes the same substructures as ECFP4, but it ranks them by prevalence and selects the top L ($L = 1024$) [6]. We fitted the Sort & Slice vocabulary on the training molecules alone, refitting it for each QM9 replicate and for each cross-validation fold. Avalon is a 2,048-bit fingerprint, generated with the Avalon toolkit through RDKit [7, 32]. The PDV representation is a vector of 200 physicochemical descriptors computed with RDKit's `MolecularDescriptorCalculator` [31, 32]; the 200 are listed in Additional file 1, and the list is fixed in our code rather than taken from RDKit's default descriptor set, which grows between releases. ChemBERTa embeddings are the mean over the non-padding token embeddings of the `DeepChem/ChemBERTa-77M-MTR` checkpoint, producing 384 features [28]. MHG-GNNs are a GIN-based autoencoder pre-trained on 1.34 million PubChem molecules using a β-variational autoencoder loss, producing 1024-dimensional embeddings through iterative message passing and found in the published `mhggnn_pretrained_model_0724_2023` checkpoint [8]. We z-score normalized PDV, ChemBERTa and MHG-GNN per feature, using the training molecules' mean and standard deviation; ECFP4, Avalon and Sort & Slice reach the model as built.

3.2.3 Models

To understand model behavior in the presence of label noise, we fitted models from several families, namely tree ensembles (random forests, quantile regression forests, XGBoost, LightGBM and NGBoost), a support vector machine, Gaussian processes and neural networks. Several of these models are variants of one another that differ only in the probabilistic components they add, and the hyperparameters of all models described below are listed in Additional file 1.

While we used 100 trees for RF, we observed a noisy quantile estimate with 100 trees for QRF, prompting us to use 300 instead. To observe the effect of the quantile mechanism, we then fit a second RF at 300 trees so the two models were idential in all other settings.

NGBoost relies on scoring off validation data, which differs by dataset. On QM9 the validation split is already held out of every fit. However, on the three assay datasets the validation set is a scaffold-grouped fifth of the rows NGBoost is handed, which already exclude the validation molecules.

We used an RBF with $C = 1.0$ and $\gamma =$ `scale` as the kernel, as it accomodates continuous vectors.

We fitted three variants of GPs, each with a constant mean and a scaled kernel. The first uses an RBF kernel and runs on all six representations. The second uses the Tanimoto kernel [67, 68] and was run on ECFP4 alone. The third is a heteroscedastic variant in which the RBF process is given a second network that predicts the observation noise of each molecule. As such, we used a standard GP and only fit on at most 5,000 training molecules, drawn by a seeded sample. Due to the size of the datasets, this cap only affected QM9. We implemented these GP models in `GPyTorch` [111], with the Tanimoto kernel taken from the `Gauche` framework [66].

While preliminary tests indicated that the Tanimoto kernel provides GP with substantial benefits over other kernels, for SVM, the RBF kernel was sufficient for all types of data.

We tested two deterministic, fully connected feed-forward architectures, both implemented in PyTorch [112]. NN-α has two hidden layers of 128 and 64 units with dropout after each hidden layer, and NN-β has two hidden layers of 128 units with dropout applied before the output layer. Both were trained with the Adam optimizer and early stopping on the validation loss. Bayesian transformations converted these networks into Bayesian neural networks (BNNs) with ReLU activations and dropout

regularization ($p = 0.2$), and Additional file 1 gives the full widths, dropout fractions, learning rates and batch size of every network.

To get these networks producing uncertainty estimates, we applied four probabilistic transformations to each architecture, producing eight probabilistic networks. The first replaces every linear layer with a Bayesian layer carrying a Gaussian prior $\mathcal{N}(0, 0.1^2)$ on its weights, producing BNN-α and BNN-β. They are trained on an evidence lower bound whose Kullback–Leibler (KL) divergence term is divided by the number of training molecules. While this is a straightforward approach, it is computationally expensive. We explored alternative techniques like our second variant, where we add an output head to the standard NN and fit the negative log likelihood of a Gaussian with that variance [113]. The third, Variational Bayesian Last Layer (VBLL), replaces every linear layer with a layer that produces mean-field variational posteriors $q(\mathbf{W}) = \mathcal{N}(\boldsymbol{\mu}_W, \mathrm{diag}(\boldsymbol{\sigma}_W^2))$ over the last-layer weights and trains by maximizing the evidence lower bound. This can be thought of as minimizing the reconstruction loss plus KL divergence $D_{\mathrm{KL}}(q(\mathbf{W})\|p(\mathbf{W}))$ from a standard normal prior, scaled by $1/N$ [114]. Because only its output layer keeps a learned observation-noise parameter, VBLL networks report only epistemic uncertainty. The fourth makes observation noise a function of the input. All eight probabilistic networks estimate their predictive distributions from 100 stochastic forward passes. Each network and its Bayesian counterpart were tuned to the same set of parameters so that NN-α against BNN-α and NN-β against BNN-β differ by the Bayesian layers alone.

3.2.4 Noise Conditions and Levels

The seven noise conditions used in this chapter are defined in Section 2.5.1.

We ran experiments across seven noise levels, 0, 0.2, 0.3, 0.5, 0.75, 1.0 and 1.5. We added the noise to the raw label and standardized the target afterward, using the clean training mean and spread. For censoring, the level is the fraction of labels clipped, at 0, 0.1, 0.2, 0.25, 0.3, 0.4 and 0.5 [81].

We ran Gaussian, grouped-wider and grouped-shifted noise on every model and representation pairing. The remaining four conditions ran on selected pairs.

3.2.5 QM9 Data Pipeline

QM9 [107], loaded through PyTorch Geometric, was processed by a separate pipeline in which Python code loads and splits the data, computes every representation, and trains and scores the models. Between featurization and training, a program written in Rust injects the noise, standardizes the labels and writes the files from which the models are trained.

Each replicate used a new scaffold split, drawn as described in the Datasets subsection above. Each acyclic molecule forms its own group, identified by its canonical SMILES, and no group is ever divided between two splits. The same grouping defines the scaffold families targeted by the grouped noise conditions.

The six representations described above are computed in Python. Binary fingerprints are stored in bit-packed form, Sort & Slice counts as 16-bit integers, and the descriptors and learned embeddings as 32-bit floating-point numbers. The run halts if any molecule has an ECFP4 or Avalon fingerprint that is entirely zero, so that a molecule that failed featurization cannot enter training. The pipeline can also build one-hot encoded SMILES, although these were not used in this thesis.

Conventional approaches load entire datasets into memory from comma-separated values (CSV) or Parquet files, which creates memory bottlenecks at scale and repeats the cost of deserialization in every experimental replicate. Our pipeline instead serializes the preprocessed data into custom memory-mapped binary files that are read on demand, so that memory use stays low and constant regardless of dataset size (Figure 3.1). Python writes one file per split, the Rust program reads each file and writes a new one in which the processed labels replace the raw ones, and Python then reads the new files back for training.

Each memory-mapped file stores a sequence of fixed-format entries. An entry holds the isomeric and canonical SMILES strings in UTF-8 with a length prefix, the raw target value as a 4-byte float, the requested representations as fixed-width blocks, and the processed target value, which includes the noise and normalization, as a 4-byte float. The blocks occupy 2,048 bytes for Sort & Slice (1,024 16-bit integers), 800 bytes for PDV (200 32-bit floats), 1,536 bytes for ChemBERTa (384 32-bit floats), 4,096 bytes for MHG-GNN (1,024 32-bit floats), and 256 bytes each for Avalon and ECFP4. Only the requested representations are written, so the byte offset of every field depends on the list of representations. This format permits zero-copy memory mapping, in which Python and Rust access the same disk-backed

memory pages without deserialization and memory use stays constant regardless of dataset size.

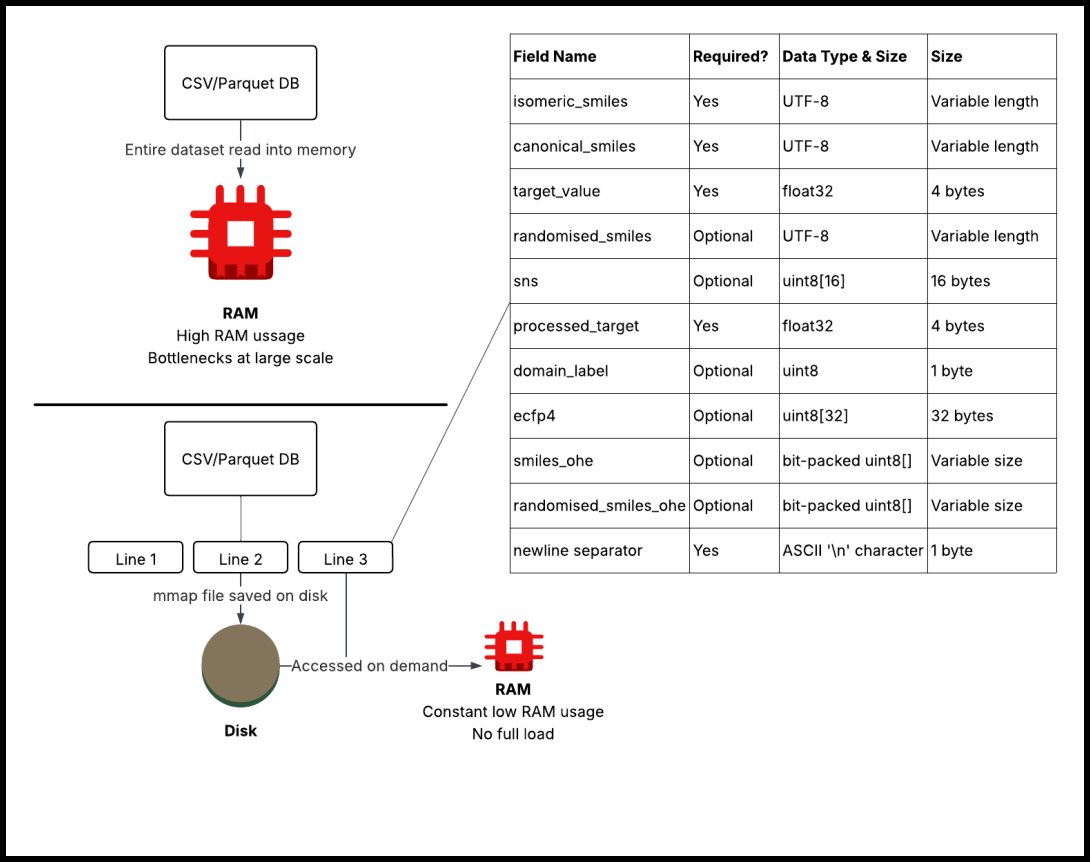
The Rust program draws the noise for each split separately, following the order in which that split's records are stored. The noise parameters are set on the training split, since the amount of noise, the censoring limit and the affected scaffold families are all computed from the clean training labels. Validation labels receive an independent draw, from a seed derived from the training seed, with the same amount of noise in label units, the same limit and the same affected families, and the program verifies that validation received as much noise as training. Test labels are never corrupted, although every split records the per-molecule noise scale its region would have received, so that held-out molecules can be scored against the pattern applied in training.

For every molecule in every split, the Rust program records its canonical SMILES, clean label, injected noise, noise scale, the noise pattern applied to its region, noisy label and the standardized value written to file. It also writes a summary of each run that lists the parameters of the noise condition and the amount of noise delivered. The run halts if the record at any row index holds a different molecule when it is written than when its noise was drawn. A separate self-test mode applies the same checks on the amount and targeting of noise to a label file, without running the rest of the pipeline.

Model training. The pipeline trains the models described in the Models subsection above, with hyperparameters taken from package defaults, tuned with Optuna [115], or read from a JSON configuration file. For models that return uncertainty, the pipeline can also score the training molecules out of fold, so that the uncertainty of each training molecule comes from a model that never saw its corrupted label.

3.2.6 Uncertainty

Up to this point we have been concerned with how noise affects the predictive accuracy of QSAR models. However, label noise should also affect the uncertainty these models report. Some models return only a point prediction, while others also return an estimate of uncertainty, either as a predicted distribution or as the spread across an ensemble. Furthermore, some report a single value of uncertainty while others report one that can be decomposed into an aleatoric and an epistemic compo-



Field Name	Required?	Data Type & Size	Size
isomeric_smiles	Yes	UTF-8	Variable length
canonical_smiles	Yes	UTF-8	Variable length
target_value	Yes	float32	4 bytes
randomised_smiles	Optional	UTF-8	Variable length
sns	Optional	uint8[16]	16 bytes
processed_target	Yes	float32	4 bytes
domain_label	Optional	uint8	1 byte
ecfp4	Optional	uint8[32]	32 bytes
smiles_ohe	Optional	bit-packed uint8[]	Variable size
randomised_smiles_ohe	Optional	bit-packed uint8[]	Variable size
newline separator	Yes	ASCII '\n' character	1 byte

Figure 3.1: In-memory (top left) and memory-mapped (bottom left) data loading. Memory-mapped loading keeps each molecule as one line of a binary file on disk and reads lines into memory on demand. The table on the right lists the fields of one line.

nent [113]. The aleatoric component describes underlying noise the model associates with the data, while the epistemic component represents the uncertainty within the model's fit. We sought to determine if, as expected, the aleatoric component rises with injected label noise while the epistemic component does not.

For the BNN and VBLL transformations, we took the two components over $T = 100$ stochastic forward passes [113]:

$$\hat{u}^2_{\text{ale}}(x) = \frac{1}{T}\sum_{t=1}^{T}\hat{v}_t(x), \qquad \hat{u}^2_{\text{epi}}(x) = \frac{1}{T}\sum_{t=1}^{T}\left(\hat{\mu}_t(x) - \bar{\mu}(x)\right)^2, \tag{3.1}$$

where $\hat{\mu}_t(x)$ and $\hat{v}_t(x)$ are the mean and the variance returned for molecule x on pass t, and $\bar{\mu}(x)$ is the mean over passes. Here $\hat{v}_t(x)$ is the predicted observation noise variance. It varies per molecule in the variance-head and heteroscedastic variants, and is one number for the whole fit in the VBLL transformation. A full-BNN without a variance head returns no variance, so its uncertainty is epistemic alone.

The other models follow the same split. For GPs, the epistemic component is

the posterior variance of the latent function and the aleatoric component is the learned observation noise [64]. All variants of the GP produce one number for the whole fit except heteroscedastic. For QRF models, each tree holds multiple training molecules. So for a given molecule, the aleatoric component is the variance the labels the molecule's given leaf. The total aleatoric uncertainty are those variances averaged over the trees. The epistemic component is the variance of the leaf means across the trees. NGBoost predicts one variance for each molecule from a single fit, which is aleatoric alone [58]. In every model, the total uncertainty is the sum of the the aleatoric and epistemic components.

No held-out label carries injected noise, so every comparison against the injected noise was made on training molecules. Each training set was divided into five folds grouped on Murcko scaffolds, and each molecule was scored by a refit that excluded its own fold. For NGBoost on QM9, three of the five folds were scored to limit its cost. No uncertainty was calibrated after fitting.

3.2.7 Evaluation

We compute statistics on the held-out molecules, for QM9 the test split and for three assay datasets the held-out fold of the scaffold cross-validation. Different combinations of dataset, model, representation, noise condition, and noise level are examined in each scenario. We used the coefficient of determination (R^2) throughout. For molecule i, let y_i be the clean label, $\hat{y}_i$ the prediction, ϵ_i the noise injected into its label and u_i the predicted uncertainty.

To evaluate the effect of label noise, we tracked predictive performance retention under increasing artificial noise. For each QSAR pairing we recorded $R^2(\tau)$ at each noise level τ, and normalized it to obtain the retention ratio $R^2(\tau)/R^2(0)$. To quantify robustness, we used the normalized area under this retention curve, $\mathrm{AUC}_{\mathrm{norm}}$(Equation 2.7), evaluated using the trapezoidal rule over the seven noise levels. While we tested other metrics of robustness, we found $\mathrm{AUC}_{\mathrm{norm}}$to be less directly correlated to clean-label R^2 predictions. For censoring, we sweep over a set of fractions to clip the labels prior to normalization.

An $\mathrm{AUC}_{\mathrm{norm}}$value near 1 indicates that the QSAR model in question has retained predictive performance with artificial noise, meaning it is more robust to noise. Since we normalize the curve by its own clean-label predictive performance, a model with a low clean R^2 has less to lose, so $\mathrm{AUC}_{\mathrm{norm}}$is reported beside clean R^2. We excluded

replicates with a clean $R^2 < 0.3$ from the robustness analysis (Additional file 2), as performance retention ratios become unstable when the clean-label denominator approaches zero. On QM9 one $\mathrm{AUC}_{\mathrm{norm}}$is computed per replicate, and we report the median over the ten replicates. On the three assay datasets one value is computed per scaffold fold, and the median over the five folds is reported.

To compare the relative effect of the thirteen base models across datasets and noise conditions, we compared their rankings using Spearman's rank correlation coefficient (ρ) between clean R^2 and $\mathrm{AUC}_{\mathrm{norm}}$. We compared probabilistic models with its deterministic counterpart using a two-sided Wilcoxon signed-rank test paired on the replicate, at $\alpha = 0.05$, separately for each representation and noise condition. This test was run only on QM9, as with five scaffold folds the smallest attainable p is 0.0625.

We conducted a separate two-way analysis of variance (ANOVA) for each noise condition to identify the relative contributions of molecular representation and model architecture to both predictive performance and noise robustness. For a given metric y, either R^2 at a noise level of 1.0 or $\mathrm{AUC}_{\mathrm{norm}}$:

$$y_{ijr} = \mu + \alpha_i + \beta_j + (\alpha\beta)_{ij} + e_{ijr}, \tag{3.2}$$

where α_i represents the effect of model architecture i, β_j represents the effect of molecular representation j, $(\alpha\beta)_{ij}$ is the interaction term, and e_{ijr} is the residual for replicate r. The proportion of variance explained by each term was calculated as the effect size

$$\eta^2_{\mathrm{term}} = \frac{SS_{\mathrm{term}}}{SS_{\mathrm{total}}}, \tag{3.3}$$

using Type I (sequential) sums of squares, entered in the order model architecture, molecular representation, interaction. The ten QM9 replicates provide the residual. The uncertainty on each share is the range of η^2 when the decomposition is repeated with each replicate left out in turn.

ANOVA relies on the assumption that observations both within and across groups are independence. Across the whole set of models and representations in this study, some pairs produced similar predictions that are not independent. This causes the residual degrees of freedom to increase, while the residual sum of squares barely moves. To handle this issue, we computed pairwise Spearman rank correlations between all models and between all representations across noise strategies.

Models whose profiles correlated at $\rho > 0.99$ with another model were excluded, including QRFs, which were very similar to RF, as well as different BNN variants.

Across representations, Sort & Slice correlated at $\rho > 0.90$ with the ECFP4 fingerprint and was excluded to avoid inflating representation degrees of freedom, as both encode overlapping circular substructure information. We also computed intraclass correlation coefficients ICC(1,1) for all model pairs. ICC(1,1) measures the proportion of total variance attributable to between-subject differences, with values near 1.0 indicating that two models rank configurations almost identically. The full redundancy and ICC tables are reported in Additional files 3–5.

To assess the interaction between uncertainty estimation and label noise, we posed three questions. First, whether uncertainty rise with label noise, as measured by the mean u_i over held-out molecules at the highest noise level divided by its value with no added noise, in each replicate or fold. The second pertains to models who report separate aleatoric and epistemic components. We divided the increase of mean epistemic uncertainty between the highest level of noise and clean labels by the mean of the aleatoric component to determine how each responds to label noise. A ratio near 0 means the added noise was attributed to the labels while a ratio of 1 or above means the epistemic component grew as much or more. We also compared the mean aleatoric component with the standard deviation of the injected noise. The third question we had was whether per-sample uncertainty values track injected label noise, as measured by the Spearman correlation between u_i and $|\epsilon_i|$ over out-of-fold training molecules, per fold and per noise level. We also report the Spearman correlation between u_i and the absolute error of the prediction. Finally, we asked whether dividing a molecule's out-of-fold absolute error by u_i ranks the corrupted labels better than the error alone, and report the difference between the two Spearman correlations with $|\epsilon_i|$.

3.3 Variance Decomposition by Representation and Architecture

Considering the two main components of a QSAR model, the molecular representation and the model architecture, we sought to understand how each contributes to predictive performance and noise robustness. ANOVA decomposition across six noise strategies on models to predict HOMO–LUMO gap from QM9 demonstrates

that for predictive performance (R^2 at $\sigma = 0.3$), the model–representation interaction term is the largest source of variance across all six strategies, followed by model architecture and representation (Table 3.1; Figure 3.2a). However, for noise robustness as measured by the noise degradation slope (NDS), this ordering shifts. The choice of model architecture is instead the largest source of variance, while the interaction term drops to second and representation third. This observed inversion makes sense because the molecular representation dictates the chemical information the model receives, while the architecture determines how efficiently that information is used. The interaction term reflects the idea that these two are not independent. In the context of label noise robustness, representation has less of an impact since the features are not corrupted by noise. Meanwhile, the model's regularization, ensembling, and uncertainty handling have a stronger impact on how it handles noise.

The impact varied by type of noise, represented by noise strategy (Figure 3.2b). The strategies which degraded in performance the most, threshold and value-proportional, both corrupted label regions, forcing models to rely on their inductive biases. This translates to the model architecture explaining a significant portion of the variance. On the other hand, outlier noise only affects statistical outliers. For outlier noise, model and interaction effects were both smaller.

	Performance (η^2, %)				**Robustness (η^2, %)**			
Strategy	**Model**	**Rep**	**Inter.**	**Resid.**	**Model**	**Rep**	**Inter.**	**Resid.**
Gaussian	24.6	22.8	**48.1**	4.5	**48.7**	8.2	11.9	31.2
Quantile	25.1	22.4	**46.9**	5.6	**47.9**	6.8	14.9	30.4
Threshold	25.0	21.2	**48.0**	5.8	**48.0**	10.1	28.0	13.9
Heteroscedastic	27.9	23.5	**46.2**	2.5	**37.0**	4.5	17.5	41.0
Value-prop.	27.8	22.0	**46.5**	3.8	**52.8**	6.9	22.2	18.2
Outlier	25.3	22.0	**49.2**	3.5	12.7	3.2	4.7	**79.3**

Table 3.1: ANOVA variance decomposition by noise strategy on QM9. Entries are the share of variance η^2 (%) explained by the model (Model), the representation (Rep), their interaction (Inter.) and the residual (Resid.), for R^2 at $\sigma = 0.3$ (performance) and for the NDS (robustness). Bold marks the largest share in each block of each row.

The interaction between model architecture and representation is seen in Figure 3.3. While some model architectures, like SVM and full BNNs maintain consistent noise robustness scores across all representations, others like NN-β display greater variation. Model robustness on one representation strongly predicts robustness on another (Spearman $\rho = 0.73$, $p = 0.007$ for ECFP4 vs PDV; 11 models).

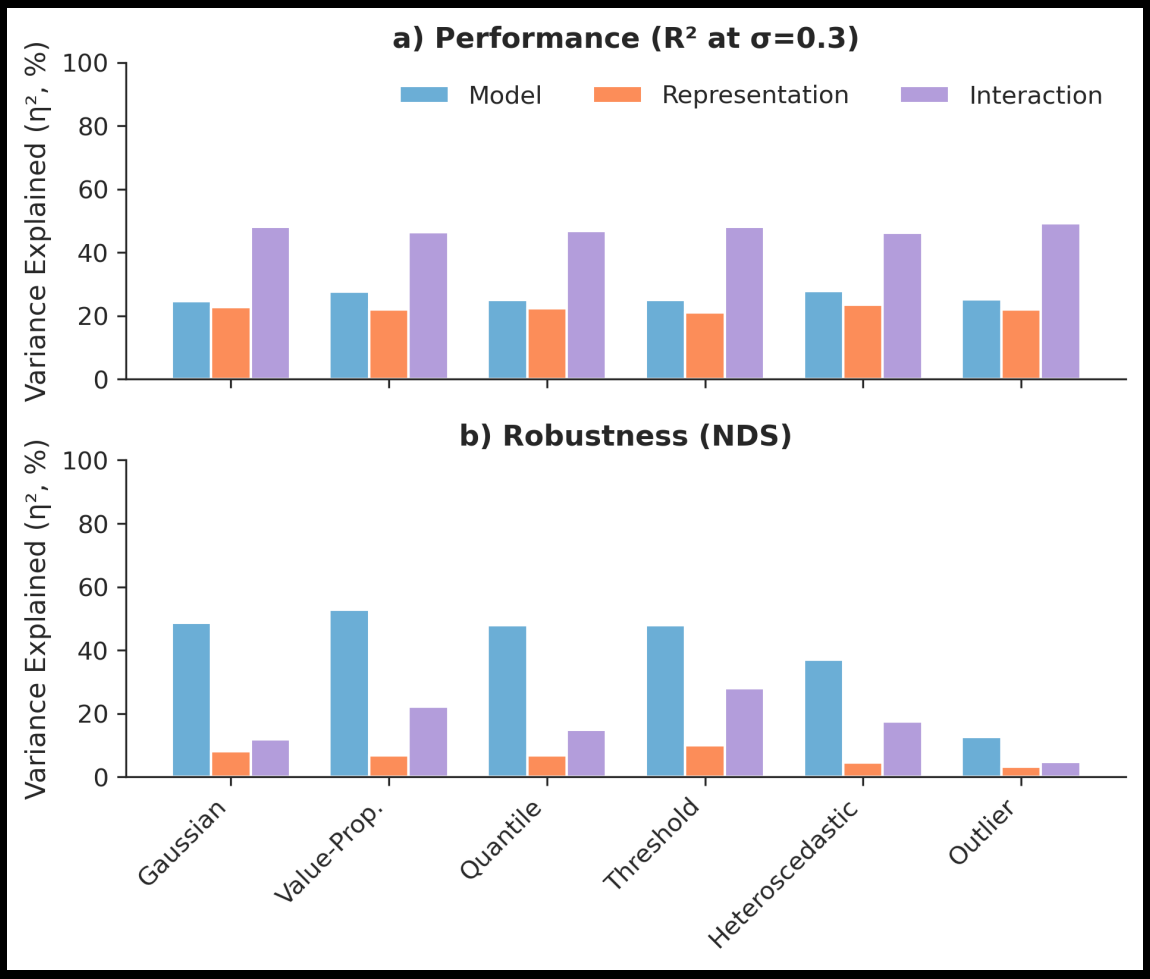


Figure 3.2: ANOVA variance decomposition of a) performance, as R^2 at $\sigma = 0.3$, and b) robustness, as the NDS, by noise strategy on QM9. Bars give the share of variance η^2 (%) explained by model, representation and their interaction, and the residual shares are in Table 3.1.

However, the scatter around the diagonal reflects the interaction term identified by the ANOVA.

We filtered out model–representation configurations with baseline predictive performance $R^2 < 0.6$, excluding 66 configurations where poor clean-data performance would produce misleadingly shallow degradation slopes (Additional file 2). We evaluated all model architectures under increasing label noise on the PDV representation across six strategies, considering both predictive performance as measured by R^2 as well as the NDS. We observe that NDS varies substantially across architectures and noise strategies, as seen in Figure 3.4.

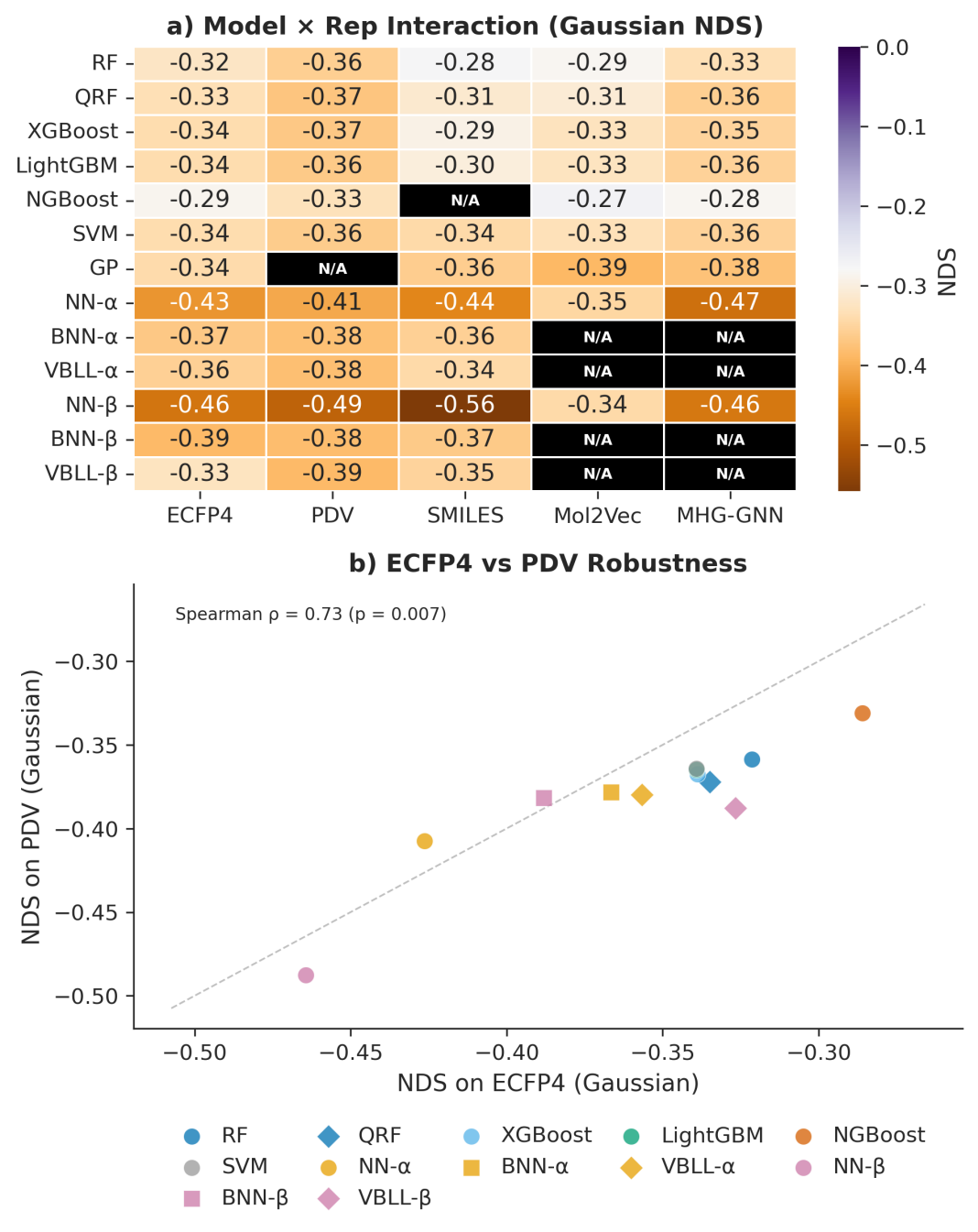


Figure 3.3: Model–representation interaction in noise robustness on QM9 under Gaussian noise. a) NDS of each model on each representation, where darker orange means faster degradation and N/A marks excluded configurations. b) NDS on PDV against NDS on ECFP4 for each model, with the line of equality (dashed) and the Spearman correlation ρ.

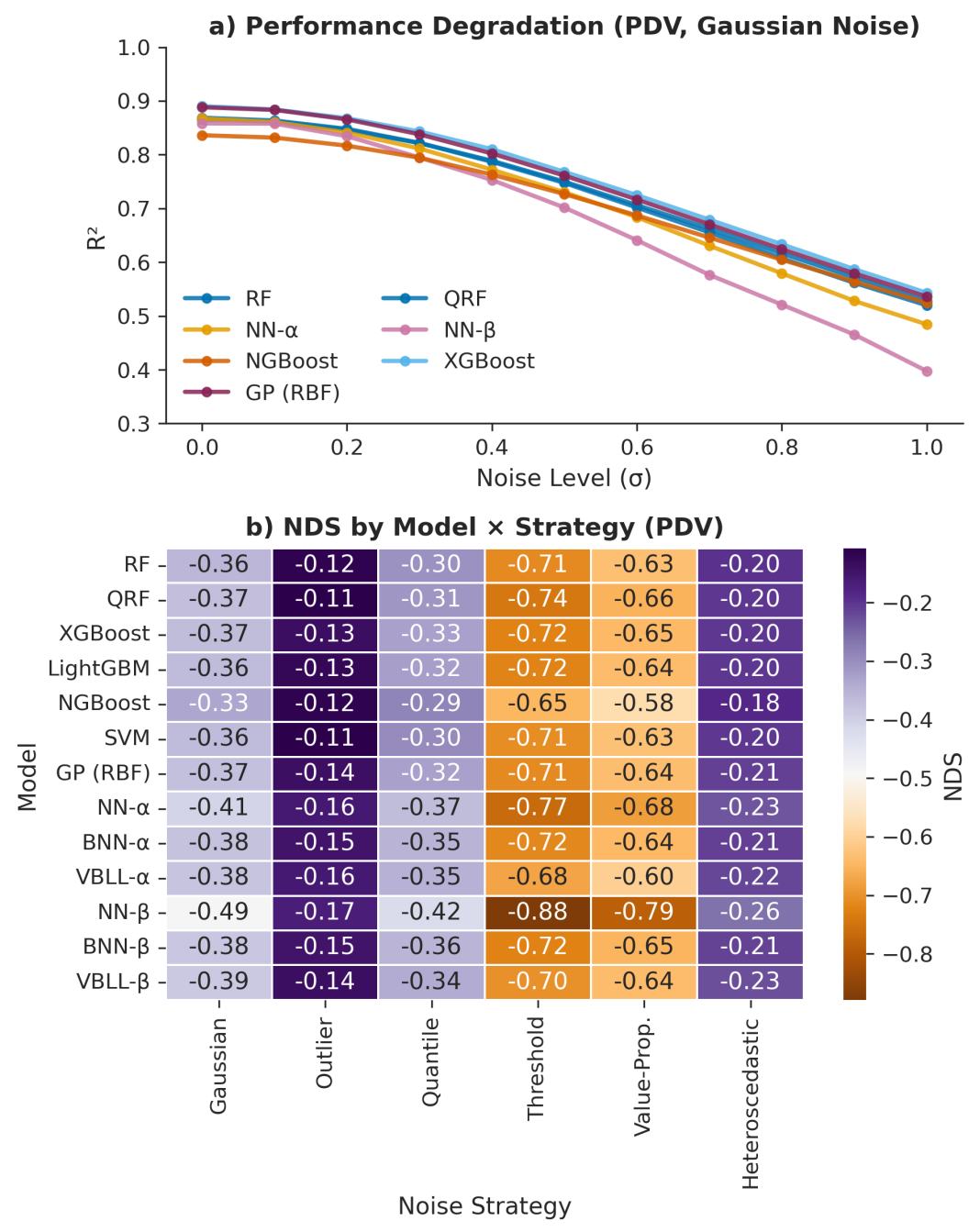


Figure 3.4: Noise robustness on QM9 with PDV. a) R^2 against the noise level σ under Gaussian noise for seven models. b) NDS of every model under six noise strategies, where orange means faster and purple slower degradation. The GP with an RBF kernel replaces the GP with a Tanimoto kernel, which cannot be applied to PDV.

3.4 Probabilistic and Deterministic Model Variants

To further explore the impact of model architecture on noise robustness, we examined variants of the same core architecture. Specifically, Bayesian transformations were used to explore the impact of the Bayesian framework on noise. By comparing models with the same core architecture, we observe that with full Bayesian transformations, representation explains less than 11% of NDS variance ($p > 0.25$) while VBLL on the same NNs is representation-dependent. On MHG-GNN and Mol2vec representations, VBLL models were unstable, such that all 24 VBLL $\times$ {MHG-GNN, Mol2vec} configurations were excluded due to baseline $R^2 < 0.6$. Further work could involve more thoroughly-tuning these models and exploring different core architectures to adapt to these embedding-style inputs.

In general, the full Bayesian transformations reduced the degradation in performance with Gaussian noise the most, as seen in Figure 3.5. Both full BNN and VBLL transformations significantly improved robustness for both NN-α and NN-β (Table 3.2). Full transformations are more computationally expensive, making VBLL a potentially stronger approach to handling noise with NNs when working with larger data sets or modeling architectures. The choice of transformation varies by representation, as seen in Additional file 8, with SMILES receiving the largest benefits from Bayesian transformations. QRF was significantly less robust than RF (Table 3.2), suggesting that quantile regression adds complexity and flexibility which may result in overfitting.

Base	Variant	Δ NDS	p-value	Sig.
NN-α	BNN-α	+0.056	2.9×10^{-10}	*
NN-α	VBLL-α	+0.061	1.2×10^{-6}	*
NN-β	BNN-β	+0.096	2.9×10^{-11}	*
NN-β	VBLL-β	**+0.124**	1.2×10^{-7}	*
RF	QRF	**−0.022**	1.6×10^{-10}	*

Table 3.2: Wilcoxon signed-rank tests of each deterministic model against its probabilistic variant on QM9. ΔNDS is the mean change in NDS from the base model to the variant, and positive values mean that the variant is more robust. An asterisk marks $p < 0.05$, and bold marks the largest positive and the only negative change.

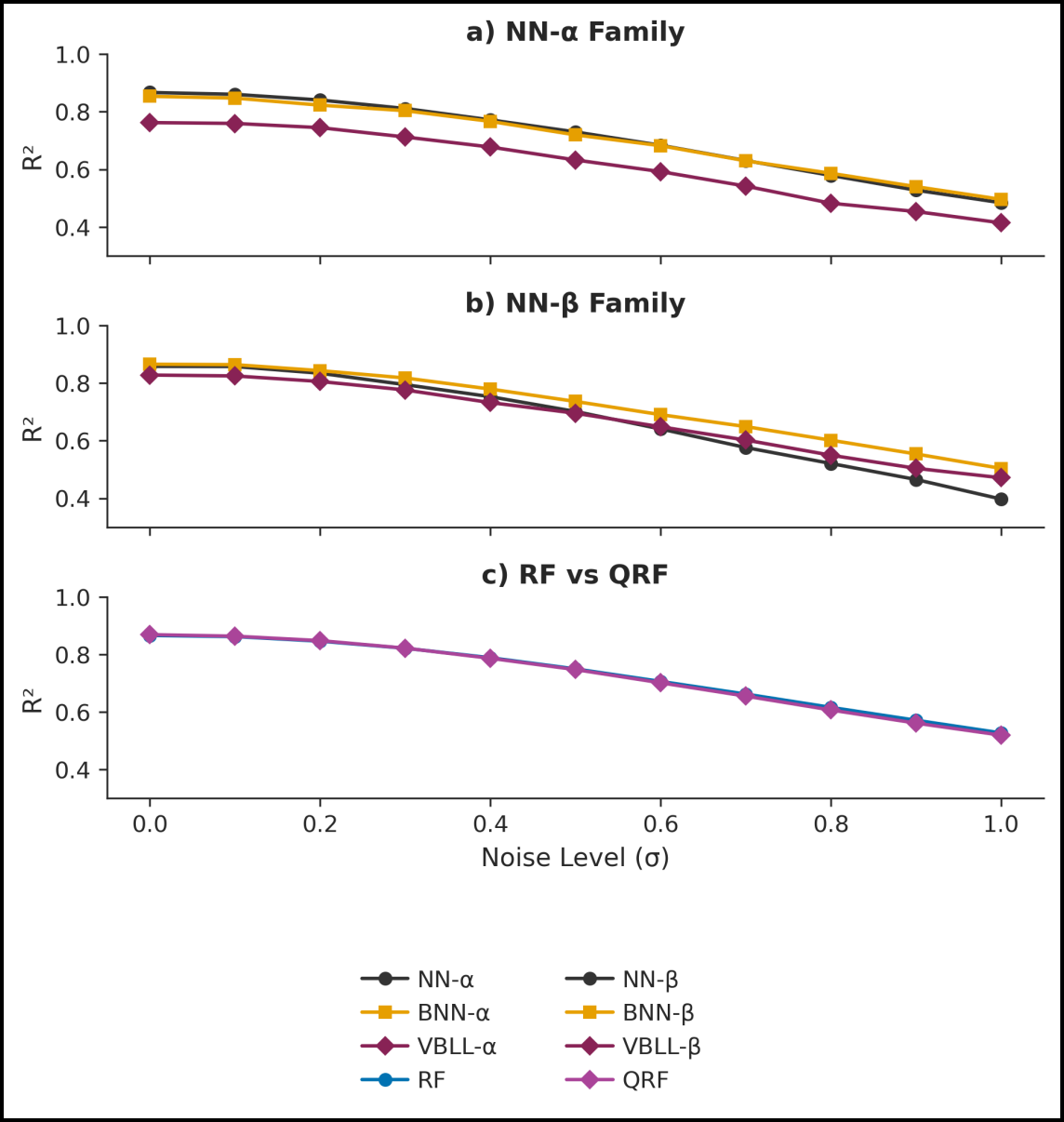


Figure 3.5: Performance of neural network families and random forests under Gaussian noise on QM9 with PDV. Each panel shows R^2 against the noise level σ for a) NN-α, BNN-α and VBLL-α, b) the corresponding NN-β family and c) RF and QRF.

3.5 Model Robustness Rankings across Noise Strategies

NGBoost and SVM showed the smallest degradation in predictive performance across all strategies, while NN-β showed the steepest (Table 3.3). When sorting models by NDS, their ranks across noise strategies were consistent: Kendall's $W = 0.92$ ($p = 2.7 \times 10^{-8}$; 11 models, 6 strategies), indicating that a model's relative robustness is largely strategy-independent. The same ranking pattern holds on the ECFP4 fingerprint (Additional file 6). Robustness is also largely decoupled from baseline

predictive performance: across models on PDV under Gaussian noise, NDS clusters near -0.38 regardless of baseline R^2 (Additional file 7), so a high-performing model is not automatically a more robust one.

Model	Gauss.	Outlier	Quant.	Thresh.	Hetero.	Val.-P.	Mean
NGBoost	**−0.33**	−0.12	**−0.29**	**−0.65**	**−0.18**	**−0.58**	**−0.36**
SVM	−0.36	**−0.11**	−0.30	−0.71	−0.20	−0.63	−0.38
RF	−0.36	−0.12	−0.30	−0.71	−0.20	−0.63	−0.39
LightGBM	−0.36	−0.13	−0.32	−0.72	−0.20	−0.64	−0.40
VBLL-α	−0.38	−0.16	−0.35	−0.68	−0.22	−0.60	−0.40
XGBoost	−0.37	−0.13	−0.33	−0.72	−0.20	−0.65	−0.40
VBLL-β	−0.39	−0.14	−0.34	−0.70	−0.23	−0.64	−0.41
BNN-α	−0.38	−0.15	−0.35	−0.72	−0.21	−0.64	−0.41
BNN-β	−0.38	−0.15	−0.36	−0.72	−0.21	−0.65	−0.41
NN-α	−0.41	−0.16	−0.37	−0.77	−0.23	−0.68	−0.44
NN-β	−0.49	−0.17	−0.42	−0.88	−0.26	−0.79	−0.50

Table 3.3: NDS of each model on PDV for QM9 under six noise strategies and their mean, with models ordered by the mean. Values closer to zero mean greater robustness, and bold marks the best value in each column and the two best models. Gauss. = Gaussian, Quant. = quantile, Thresh. = threshold, Hetero. = heteroscedastic, Val.-P. = value-proportional.

The strategies differed not only in predictive performance severity, but also in their ability to discriminate between architectures. The spread of NDS across models differed by noise strategy, with outlier noise having a spread of 0.06 and threshold noise having 0.23, presumably exposing architectural differences that are masked by other types of noise. As seen in Table 3.3, NGBoost was the most resistant to noise across strategies. With respect to the VBLL models, VBLL-α showed a similar pattern, outperforming all tree-based models except NGBoost under threshold and value-proportional noise despite ranking below them under Gaussian noise. This suggests that its learned noise variance is most beneficial when label corruption is systematic rather than uniform. This observation with VBLL is not alone; predictive performance on clean data is not well-correlated with noise robustness. In fact, some of the models with the lowest NDS across strategies such as NGBoost and SVM did not perform particularly well on clean data. These models may be more prone to picking up on noise rather than underlying signals in the data.

The ANOVA showed that model architecture and the model–representation interaction term dominate the variance in NDS. SVM and full BNNs stood out as noise-robust regardless of representation, and both rely on strong inductive biases, margin maximization in the case of SVM and weight priors in the case of BNNs. RF and NN-β showed the opposite pattern, since they can be robust to noise but

only when paired with particular representations, and these representations differ between the two models.

Although representation is not a dominant determinant of NDS variance, it has an effect through the model–representation interaction term. On SMILES, model choice explains over 91% of the variance in robustness, compared with 72% on PDV. Across all six noise strategies, PDV configurations produced the shallowest mean NDS of all representations, with the best values coming from models with strong inductive biases such as SVM and full BNNs. This suggests that PDVs are preferable when dealing with noisy data, and it appears that the compact feature space of PDV complements margin-based and Bayesian regularization. Embeddings performed poorly, particularly when paired with NNs. These patterns are consistent with the importance of the interaction term and indicate that the choice of representation modulates noise robustness within a given architecture.

Taking a closer look at model choice for noise robustness, we ranked all model–representation configurations on QM9 by their Gaussian NDS. Tree-based ensembles dominate the top 10: NGBoost appears five times, XGBoost and RF twice each, and LightGBM once. No neural-network configuration appears the top 10. Rankings are consistent across strategies and the top QM9 configurations maintain their advantage on absorption, distribution, metabolism and excretion (ADME) datasets, though SVM configurations rank higher on these ADME datasets than NGBoost.

3.6 Validation on Assay Datasets

In addition to working with "clean" quantum molecular properties, we tested our findings on experimentally-obtained data, replicating our process of adding artificial noise. Unlike QM9, where $\sigma = 0$ corresponds to a noise-free baseline, the experimental endpoints carry their own measurement noise in both training and test labels; the injected σ therefore stacks on top of an unknown noise floor. Per-dataset NDS heatmaps across all model architectures and noise strategies are shown in Figure 3.6. The ANOVA on these additional datasets confirms that model architecture dominates robustness variance on all three datasets, as seen in Additional file 10, and that the trends with respect to noise robustness generalize. NGBoost ranks first on both QM9 and external datasets, SVM also transfer quite well. Some models' predictive capabilities are limited on these external datasets, likely due to a combination of smaller training sets, narrower chemical coverage, and the unknown experimental

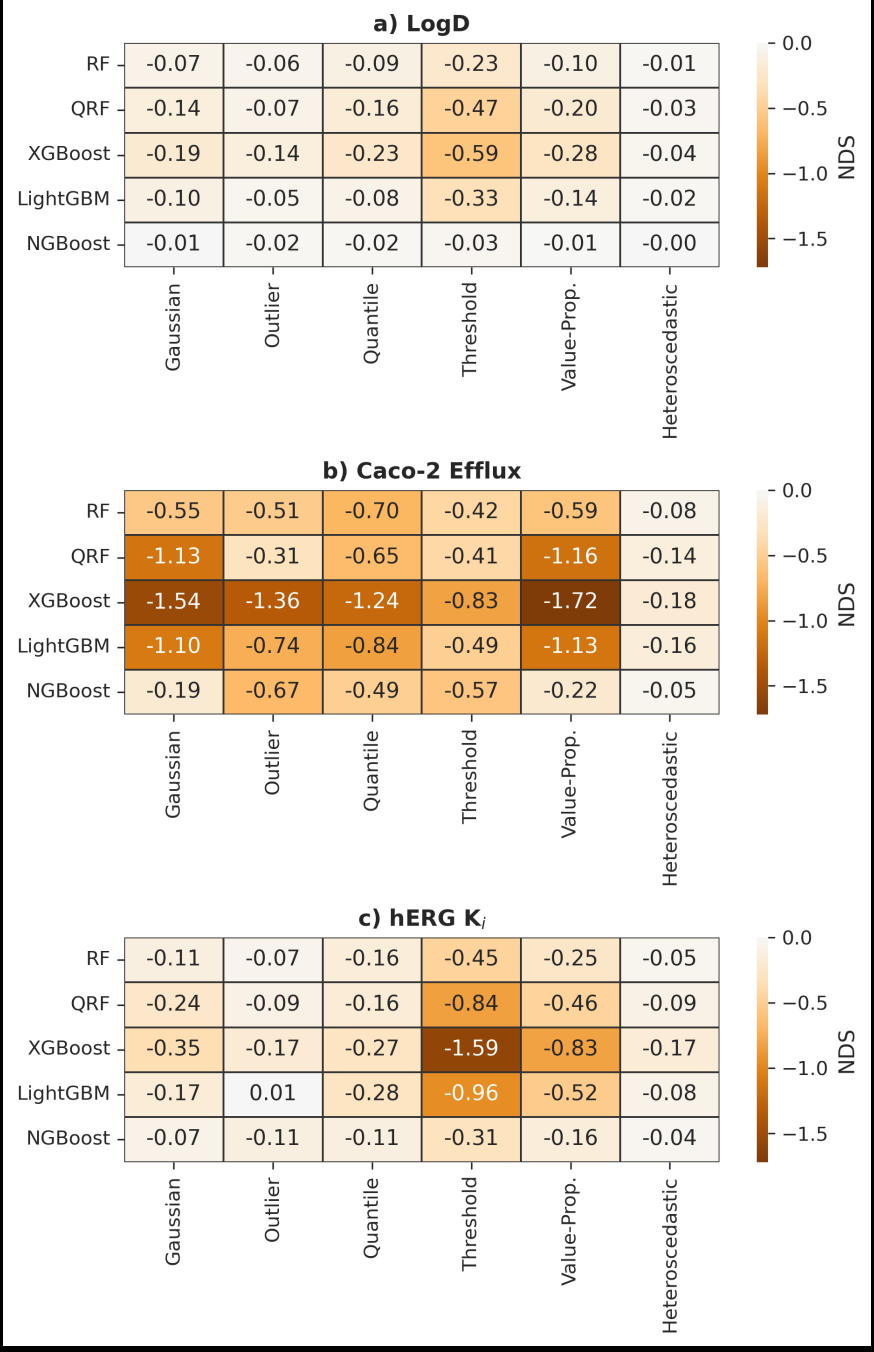


Figure 3.6: NDS of five tree-based models under six noise strategies on a) LogD, b) Caco-2 and c) hERG K_i. Darker orange means faster degradation, and the three panels share one color scale.

noise in the labels. XGBoost and BNN variants suffer the most here, as seen in Figure 3.7a.

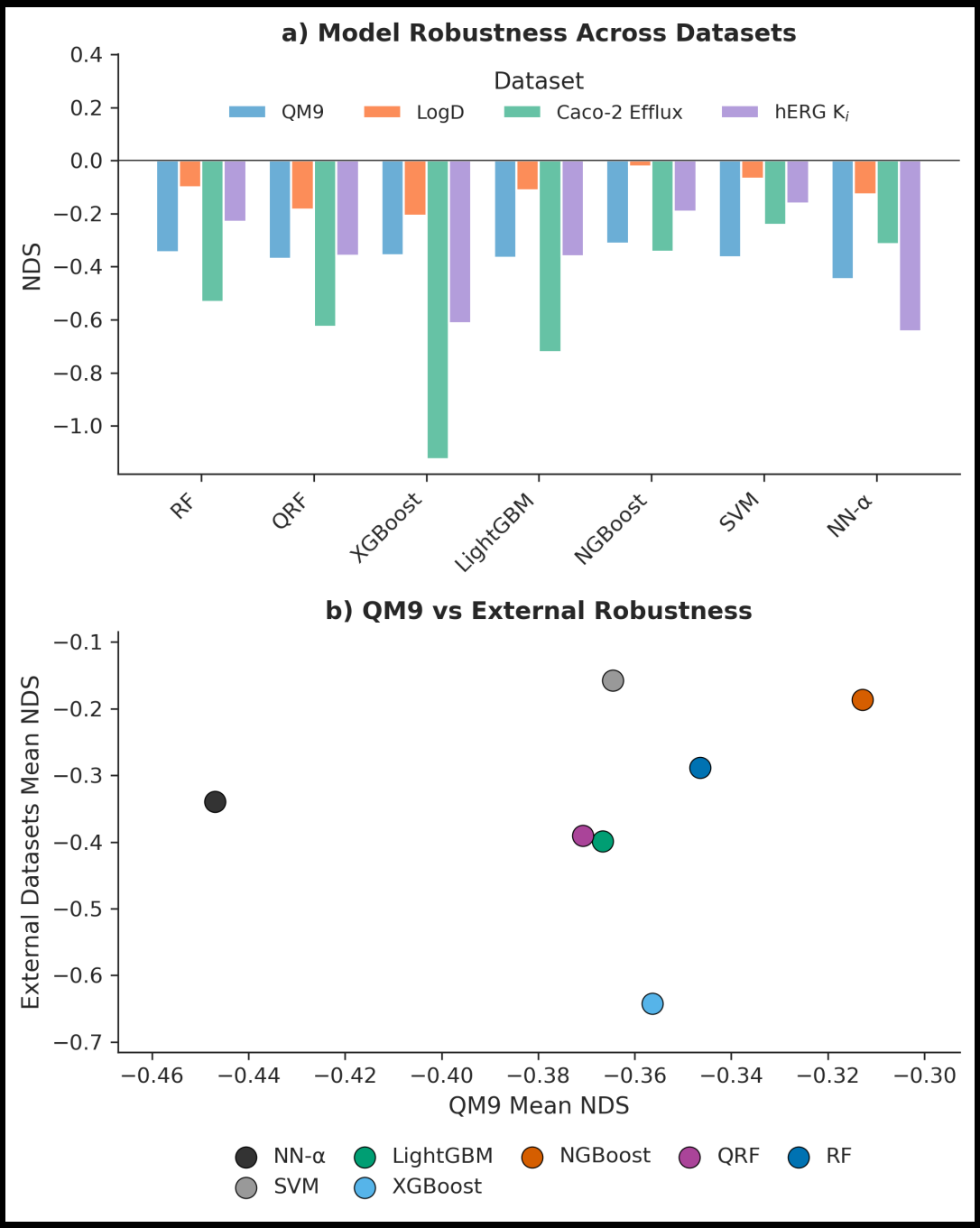


Figure 3.7: NDS on QM9 and the three assay datasets. a) NDS of seven models on each dataset. b) Mean NDS over the three assay datasets against the NDS on QM9, for each model.

QRF was consistently less robust than RF on every external data set (Additional file 11). As seen with QM9, choice of molecular representation has a minimal effect on noise robustness across all three external datasets, reinforcing the finding that model architecture, not representation, is the main driver of noise robustness.

3.7 Uncertainty under Label Noise

The sections above showcased the importance of model selection when working with noisy labels. We take this one step further and ask, for the models which produce uncertainty estimates, how those estimates are affected by noise and if they can be used to track noise.

3.7.1 Per-Sample Uncertainty and Label Noise

Kolmar and Grulke [9] found that the mean predicted uncertainty derived from the GPs increases with the amount of label noise in the training data. We go beyond the population level and instead ask if a model's per-sample uncertainty tracks label noise. One key sign that a model is able to handle noise is its ability to track it, represented by the per-sample Spearman correlation between predicted uncertainty and noise magnitude (Table 3.4). While GPs and NGBoost gave the strongest correlations, BNNs were moderate. Learned embeddings (MHG-GNN and Mol2vec) struggled, so much so that their correlations dropped to near zero or below on every model tested.

Model	Rep.	Unc-Noise ρ	Unc-Error ρ	ECE	Cov. 1σ	Cov. 2σ
GP	**SNS**	**0.56**	0.23	0.14	71.2%	94.6%
GP	ECFP4	0.53	0.22	0.14	71.4%	95.4%
NGBoost	**SNS**	**0.47**	0.16	0.13	69.3%	95.4%
NGBoost	**PDV**	**0.47**	**0.29**	**0.11**	69.0%	95.2%
NGBoost	PDV (binary)	0.44	0.25	0.13	69.8%	95.4%
NGBoost	ECFP4	0.44	0.20	0.14	69.8%	95.3%
BNN-α	SNS	0.42	0.21	0.17	73.4%	95.1%
BNN-β	SNS	0.40	0.21	0.18	73.5%	95.1%
BNN-β	ECFP4	0.38	0.20	0.18	72.1%	95.0%
BNN-β	PDV	0.38	0.24	0.19	72.1%	94.0%
Contrast: lowest correlations (graph/embedding representations)						
BNN-β	Mol2vec	−0.05	0.02	1.33	50.6%	76.4%
VBLL-α	MHG-GNN	−0.05	0.09	13.41	26.6%	38.6%
VBLL-β	MHG-GNN	−0.03	0.10	15.34	31.1%	47.1%
VBLL-β	Mol2vec	−0.03	0.15	23.86	44.8%	58.0%

Table 3.4: Model–representation combinations on QM9 under Gaussian noise with the highest (upper block) and lowest (lower block) Spearman correlation between predicted uncertainty and the injected noise magnitude $|\epsilon|$ (Unc-Noise ρ). Unc-Error ρ is the correlation with the absolute error, and Cov. 1σ and Cov. 2σ are empirical coverages with nominal values of 68% and 95%. Bold marks the highest Unc-Noise ρ for the GP and for NGBoost, and the best Unc-Error ρ and ECE.

3.7.2 Treatment of Large Residuals

Samples containing label noise tend to produce large residuals in training, and every model has its own way of handling them. NGBoost contains predicted scales that increase to absorb residual [58]. The GP contains has a single global observation noise term. It does not absorb individual residuals, so its per-sample uncertainty derives primarily from the posterior variance rather than label noise [63, 64]. BNNs treat these large residuals differently; they broaden their weight posteriors, increasing predicted uncertainty without explicitly modeling observation noise [105, 113]. VBLL adds a learned noise variance to the BNN loss [114], but this value is a global scalar, independent of the input. As seen in Table 3.4 and Additional file 9, both BNNs and VBLLs achieve moderate correlations which improve when paired with fingerprint or descriptor-based representations. Among models that learn separate distributional parameters during training, GP and NGBoost produced the strongest correlations between uncertainty and label noise, though QRF, which separates parameters using quantile distributions, produced both poor label predictions and uncertainty-noise correlations. At the population level, mean predicted uncertainty increases with artificial noise. Although we expect to see aleatoric uncertainty increase with injected noise, for VBLL both the aleatoric and epistemic components increased (Figure 3.8b).

3.7.3 Aleatoric and Epistemic Attribution

Mean predicted uncertainty rises with label noise, but this alone does not show to which component a model attributes the rise. We expected the aleatoric component to rise with injected label noise and the epistemic component to stay roughly flat. Three kinds of model split their uncertainty per molecule, namely the full BNNs with a variance head, the heteroscedastic GP and QRF [54, 64, 113]. We excluded the other GPs because their observation noise is a single number for the whole fit and therefore cannot follow individual molecules [64]. For each model, we divided the rise in mean epistemic uncertainty on held-out molecules, from no added noise to a noise level of 1.5, by the rise in mean aleatoric uncertainty over the same range. A ratio near 0 means that the added noise went to the aleatoric component, while a ratio of 1 or more means that the epistemic component grew as much or more (**[FIGURE NEEDED: table of the ratio of epistemic rise to aleatoric rise, one row per model (the two full BNNs with a variance head, het-**

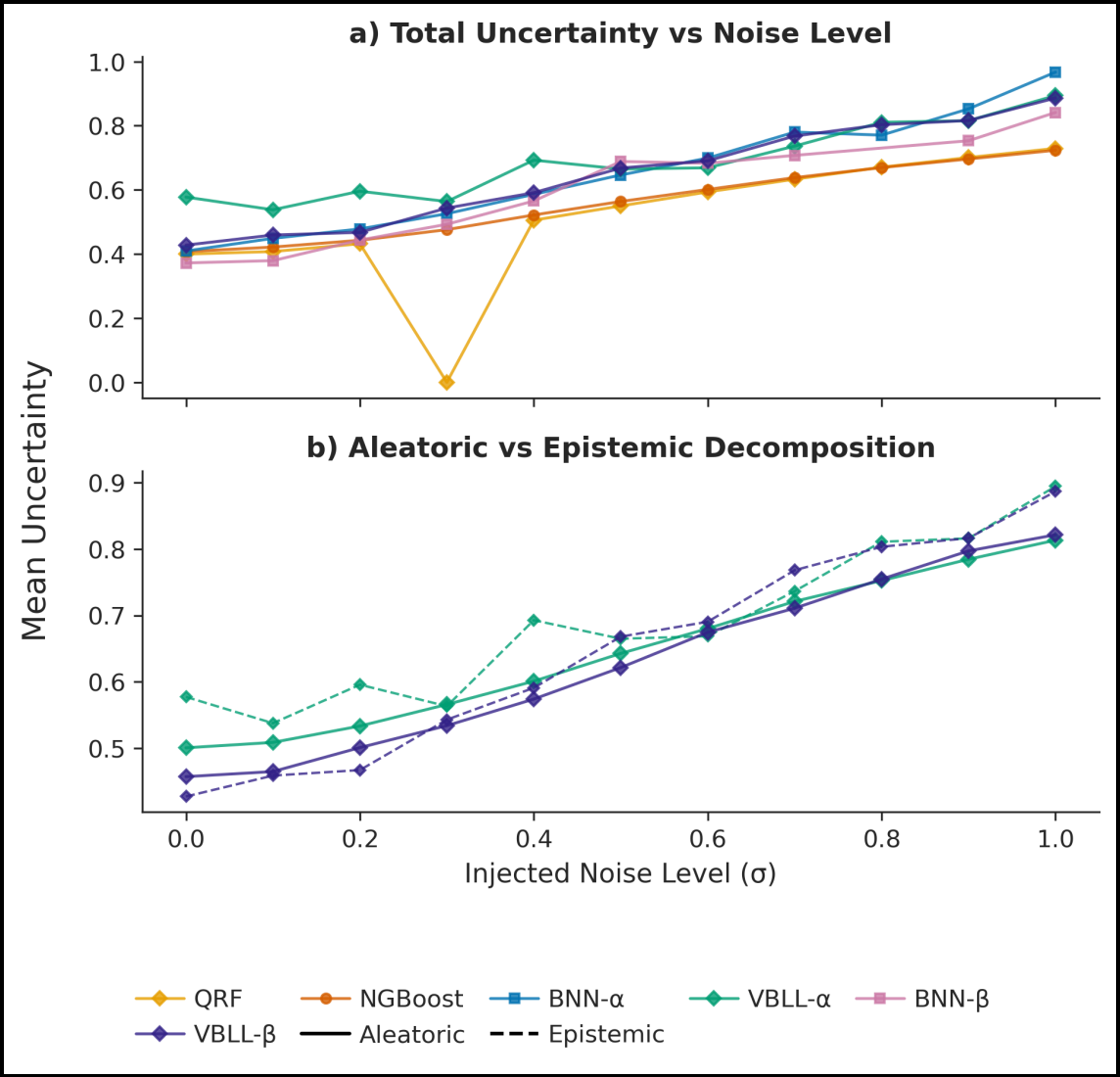


Figure 3.8: Mean predicted uncertainty against the noise level σ on QM9. a) Total uncertainty of QRF, NGBoost and the BNN and VBLL variants of NN-α and NN-β. b) Aleatoric (solid) and epistemic (dashed) uncertainty of VBLL-α and VBLL-β.

eroscedastic GP, QRF) and noise condition (Gaussian, grouped-shifted), one column per representation, each entry the range over datasets on held-out molecules]).

The full BNNs with a variance head split the added noise as expected, with a ratio of at most 0.43 for every model, dataset, representation and noise condition. QRF behaved differently, since both of its components rise with noise and on some representations the epistemic component rises faster (**[FIGURE NEEDED: aleatoric and epistemic parts against noise level, one panel per model with a per-molecule split, QM9 HOMO–LUMO gap, ECFP4, Gaussian noise]**). This holds although QRF is among the more noise-robust models, ranking 4th of the 19 models and variants on $\mathrm{AUC}_{\mathrm{norm}}$on four of the six representations on

QM9 under Gaussian noise. Its clean R^2 on the same data ranks between 6th and 13th of the 19, depending on the representation.

Representation explained little of the variance in noise robustness, but for QRF it determines how the added noise is split. Over all datasets and every noise condition except censoring, the QRF ratio on held-out molecules ranges from 0.63 to 1.09 on the three fingerprints (ECFP4, Avalon and Sort & Slice), from 1.03 to 1.42 on PDV, and from 1.28 to 2.05 on ChemBERTa and MHG-GNN. Under Gaussian noise alone, the held-out ranges are 0.65–0.75 on ECFP4, 0.69–0.76 on Sort & Slice and 0.81–0.89 on Avalon, compared with 1.17–1.32 on PDV, 1.54–1.72 on MHG-GNN and 1.60–1.88 on ChemBERTa. On the learned embeddings, QRF therefore attributes more of the added noise to its epistemic component than to its aleatoric component.

The type of noise also shifts the split, most clearly for the heteroscedastic GP. Under Gaussian noise, its ratio on held-out molecules lies between -0.05 and 0.36 on every representation. Under grouped-shifted noise, the range over datasets is 0.73–1.00 on ECFP4, 0.44–0.61 on Sort & Slice and 0.32–0.46 on Avalon, compared with 0.13–0.22, 0.14–0.23 and 0.21–0.25 on PDV, ChemBERTa and MHG-GNN, respectively. Laplace, Student-t and outlier noise give splits similar to those under Gaussian noise. Grouped-shifted noise raises the epistemic share of both the heteroscedastic GP and QRF for every dataset and representation. One possible explanation is that a shared offset within each scaffold family appears to the model as structure in the data rather than as label noise.

We next asked how closely the aleatoric component follows the amount of added noise. Because held-out labels carry no injected noise, this comparison uses out-of-fold training molecules. We fitted the slope of mean aleatoric uncertainty against the standard deviation of the added noise, both in the units of the label, under Gaussian noise, and we report the median over folds (**[FIGURE NEEDED: aleatoric part against the standard deviation of the added noise, per model, ECFP4 and PDV, out-of-fold training molecules, dashed line where reported noise equals added noise]**). On ECFP4 and PDV, the slope ranges from 0.36, for QRF on PDV for Caco-2, to 1.09, for a full BNN with a variance head on PDV for hERG K_i. Including ChemBERTa widens the range to between 0.28, for QRF on Caco-2, and 1.62, for a full BNN with a variance head on hERG K_i. The aleatoric component can therefore be used to compare noise between datasets or between noise levels, but it does not estimate the measurement error of a dataset.

Censoring behaves differently from the other noise conditions, because clipping

labels at a limit narrows their spread. NGBoost and QRF shrink their total uncertainty with that spread on every dataset, and the GPs do the same on hERG K_i and Caco-2. None of these models treats a clipped label as a noisy one, so they become more confident while the labels lose information.

3.7.4 Representation and Uncertainty Tracking

The choice of representation has a much stronger impact on per-sample uncertainty tracking than on noise robustness. Fingerprints and physicochemical descriptors provided a clear signal with which models such as GP and NGBoost could separate noise-induced residuals from structural variation. However, learned embeddings such as MHG-GNN and Mol2vec did not produce viable uncertainty-noise correlations (Table 3.4). Although these learned embeddings can be effective representations for many tasks, this research suggests that they provide challenges for per-sample noise tracking. However, there is room for further investigation, as only two embeddings were tested.

3.8 Conclusions

These results lead to two main conclusions. The first is that noise robustness, defined as the rate at which predictive performance degrades with increasing label noise, is primarily determined by the model's training mechanism. The margin maximization of SVM, the weight priors of BNNs, and the ensembling in tree methods all appear to keep those models from fitting to noise. The choice of representation has a stronger influence on predictive performance than noise robustness. The interaction term between representation and model has a stronger impact; some models like SVM and full BNNs remain robust across all representations, while RF and NNs are robust only with certain ones. While the pattern of noise has a strong impact on predictive performance, it does not influence noise robustness rankings between models or representations much.

We investigated how a wide variety of molecular representations and model architectures perform under increasing label noise in QSAR modeling. We used ANOVA variance decomposition across three noise conditions on the QM9 HOMO–LUMO gap task to show that with regards to predictive performance, model architecture and molecular representation account for comparable shares of the variance. How-

ever, for robustness to label noise, measured as $\mathrm{AUC}_{\mathrm{norm}}$, the normalized area under the R^2 retention curve, model architecture is the dominant factor, while representation explains less than 10% of the variance.

Threshold and value-proportional noise produce the widest spread of NDS across models; outlier noise barely separates them. Model rankings are highly concordant across strategies (Kendall's $W = 0.92$), suggesting that a model's robustness to noise is independent of type of noise. Predictive performance on clean data is not correlated with noise robustness; NGBoost and SVM, the most noise-robust models, did not perform particularly well on clean data. Embeddings (MHG-GNN, Mol2vec) struggled with regards to noise robustness, especially with neural network architectures. On the flip side, QRF was significantly less noise-robust than RF across datasets, suggesting that quantile regression may be overfitting to noisy labels rather than absorbing them.

The second conclusion concerns whether the uncertainty estimates of a model track per-sample label noise. Some models struggle to do so, particularly BNNs, whose uncertainty comes from a posterior over weights, and QRF, which produces a full quantile distribution but does not track the noise successfully. Models that fit a separate per-sample scale or observation-noise parameter during training track label noise more effectively. NGBoost identifies noisy labels through a per-sample predicted scale that absorbs large residuals. The GP does not, because it has a single global observation-noise term. Its per-sample uncertainty therefore comes from the posterior variance, which depends on the position of a molecule relative to the training molecules and not on its label. Among representations, uncertainty tracking collapses with the learned embeddings MHG-GNN and Mol2vec, while fingerprints and descriptors produce a usable signal. Neither strong predictive performance nor noise robustness implies noise tracking, which again depends on the architecture of the model and on how it handles the loss.

Although the choice of representation carries less weight than that of models, we observed an interesting phenomenon. PDVs produce the strongest predictive performance on clean data, but are the least noise-resistant. Embeddings, particularly Mol2vec, are the most robust to noise yet perform the worst noise tracking. At the end of the day, the choice of both model and representation comes down to your problem, including the amount of noise in your data, your objectives, and your compute limits.

QM9 served as a relatively clean baseline; it is computationally-derived with

negligible measurement noise. The validation datasets, LogD, Caco-2 [109], and hERG K_i [110], tested are smaller and cover a narrow range of the chemical and biological space, and have an unknown inherent amount of noise from experimentally-determined values, on top of the added artificial noise. These validation datasets supported the QM9 findings, with XGBoost the notable exception, degrading on the external datasets. Future work should extend this type of noise robustness benchmarking to classification tasks and larger experimental datasets with better-known uncertainty estimates.

When working with noisy experimental data in QSAR settings, it is important to keep in mind that noise robustness and noise-aware uncertainty estimation are primarily driven by the model's mechanism of training, and to a certain extent the choice of molecular representation, usually with respect to the model. While Bayesian transformations on neural networks improve noise robustness, they do not produce uncertainty estimates that correlate particularly strongly with noise. Models like NGBoost and GPs, which include a learned noise or scale parameter in the training objective, are the most robust to noise across both various types of noise and datasets and often produce uncertainty estimates which track per-sample label noise. Pairing these models with fingerprint-based representations provides additional benefits in restricting, and potentially detecting and mitigating label noise.

Chapter 4

The Flaws of the Similarity Assumption

This chapter asks whether the standard ways of comparing molecules suit the dataset and task at hand. Because general-purpose representations are built for all of chemistry, we first count how many of their features vary on a single dataset. On hERG K_i, most ECFP4 features take a single value in almost every molecule. Some representations also give different molecules the same vector, mostly stereoisomers, so that no model can tell those molecules apart. Because a distance compares every feature, features irrelevant to the task dilute it, and different representations disagree about which molecules are close. In an exploratory study of learning with noisy labels, the configurations with the lowest retention under noise all used a distance, and noise detection by neighbors usually made the model worse.

4.1 Introduction

One of the fundamental assumptions in cheminformatics is that two structurally similar compounds display similar chemical properties [74]. While there are important exceptions, namely activity cliffs [72, 75], this is a well-established concept. However, there is no single way to define similarity between compounds, and this is where the assumption begins to break down in practice. When defining similarity between two molecules, we typically rely on some pairwise mathematical function, or coefficient, that takes in two molecular representations as input. There's two varying factors here, the representation and the coefficient. Bajusz et al. [71] found that among standard coefficients, the choice between them matters little. This chapter

questions the underlying representation.

General-purpose molecular representations are designed to work across chemical domains. Consequently, when considering data from a specific domain, many of their features show little to no variance. Some representations assign separate molecules identical feature vectors. Since common fingerprints do not encode stereochemistry [116], they are often unable to differentiate stereoisomers, which can have substantially different measured activity [117]. As these coefficients compare every feature, irrelevant features dilute the result. Different representations also disagree about which molecules are close based on the information they carry.

The problem of whether neighboring molecules can predict chemical activity has been measured before. The modelability index of Golbraikh et al. [118] scores a binary dataset by the proportion of nearest-neighbor pairs that share an activity class, producing a single value for how strongly the similarity assumption holds on that particular combination of representation and data. It is not clear whether a low value is due to the data being intrinsically hard to model or to the representation not capturing adequate features for the task. Other work in this area relates to activity cliffs. Sheridan et al. [119] compared experimental error, activity distribution and activity-cliff metrics as predictors of how well a dataset can be modeled, and found the cliff metrics the strongest of the three. However, they found that these metrics cannot separate real cliffs from those produced by uncertain activity measurements. Medina-Franco [120] point out that some of these activity cliffs are artifacts of the chosen representation, similarity threshold, or noise in the data. Activity cliffs are distinct from noise, as they carry some of the most valuable information about the interaction between structure and activity. None of these measures ask specifically about whether the specific features within these representations matter for the property at hand.

There has been work that looks into how much the choice of representation changes similarity measure. Riniker and Landrum [121] benchmarked 12 fingerprints for similarity across 3 public datasets and found high correlations between them; only two simple baseline fingerprints differed significantly from the rest. The same was found for learned embeddings as neural embeddings gave negligible or no improvement over ECFP across 25 pretrained models and 25 datasets [122]. Huber and Pollmann [123] did find a difference between the structure of the fingerprints. Measured using a graph-based structural reference, fingerprints that store raw counts improve agreement with the reference over folded fingerprints. The process of folding

introduces bit collisions that distort similarity for molecules with many bit sets [123]. Activity cliffs follow the same pattern, across 15 combinations of representation and similarity measure on 3 datasets, the representation disagreed about which pairs form cliffs [124]. This supports the point Medina-Franco [120] make about cliffs as artifacts of their representation. Any method built on similarity like activity cliffs inherits this choice [103]. For the purposes of this thesis, these similarity measures are used across noise estimation and mitigation tasks.

The purpose of this chapter is to establish the problem that Chapter 5 sets out to solve. Section 4.2 asks for a given dataset, how much the features of standard representations vary and how many molecules map to identical feature vectors.

Section 4.3 looks at what happens when you used these features to quantify similarity, while Section 4.4 further follows the consequences of using these similarity measures in downstream methods. This is supported by a sweep of LNL methods reported in Section 4.4.1.

4.2 Flaws in Standard Base Representations

To quantify similarity between two compounds, we must first choose a representation to encode each compound. If the features within these representations are flawed, the similarity measure inherits them. Many standard representations are built to represent compounds across a wide array of chemical domains and be task-independent. However, chemical datasets often only cover a small region of the chemical space, leaving these general-purpose representations to describe chemistry which is absent from the dataset at hand.

4.2.1 Constant and Near-Constant Features

ECFP4 (Chapter 2) is built to describe any molecule before any dataset or task is known [4]. A single dataset often comes from a few chemical series, so most bits are off in almost every molecule. The features that matter for a task form a small subset of those that vary, and which subset matters depends on the task. Here we count only how many features vary at all; Chapter 5 asks which of them relate to the task.

A constant feature takes one value in every molecule of a dataset. A near-constant feature takes one value in at least 95% of the molecules, so the near-

constant count includes the constant features. We counted both, on all molecules, for the base representations of hERG K_i, Caco-2, LogD, QM9, MDR1–MDCK efflux, human liver microsome (HLM) clearance, rat liver microsome (RLM) clearance and the Lhasa hERG blocker training set, and for the MACCS keys wherever they were computed.

On these datasets the hashed substructure fingerprints have the most near-constant features, and the learned embeddings have none except MHG-GNN (Tables 4.1 and 4.2, Figure 4.1). ECFP4 on hERG K_i (1,412 molecules) has 1,815 near-constant features out of 2,048. Across the seven datasets where ECFP4 is a base representation, this count runs from 1,815 to 1,997 (QM9, 9,978 molecules). The same 292 of MHG-GNN's 1,024 features are zero in every molecule on all seven datasets that use it, so they come from the saved encoder output rather than the data; we have not checked why. The Weisfeiler–Lehman (WL) graph kernel keeps only labels present in at least 5% of a dataset's molecules, so it has no near-constant features.

Which count matters depends on the distance. Tanimoto similarity ignores bits off in both molecules, but a bit on in every molecule raises every similarity slightly. All 105 constant ECFP4 bits on hERG K_i are off. After standardization, a constant feature adds zero to a Euclidean or Manhattan distance, but a near-constant feature takes large values on the few molecules with the minority value. A bit on in 5% of molecules becomes 4.36 on those molecules, and a bit on in one of the 1,412 hERG K_i molecules becomes 37.56.

TODO: every cell of this table must hold a count. The dashes are representations outside that dataset's base representations; handoff in HANDOFF_fill_constant_features_tables.md.

TODO: every cell of this table must hold a count. The dashes are representations outside that dataset's base representations; handoff in HANDOFF_fill_constant_features_tables.md.

RDKit's MACCS keys include an unused bit 0, hence 167 features. All constant Mordred features are zero in every molecule, except one of the 307 on QM9, which is 1. Missing or infinite descriptor values are set to 0 (Chapter 5, "Representations by domain"), so some may be descriptors that failed to compute; we have not checked which.

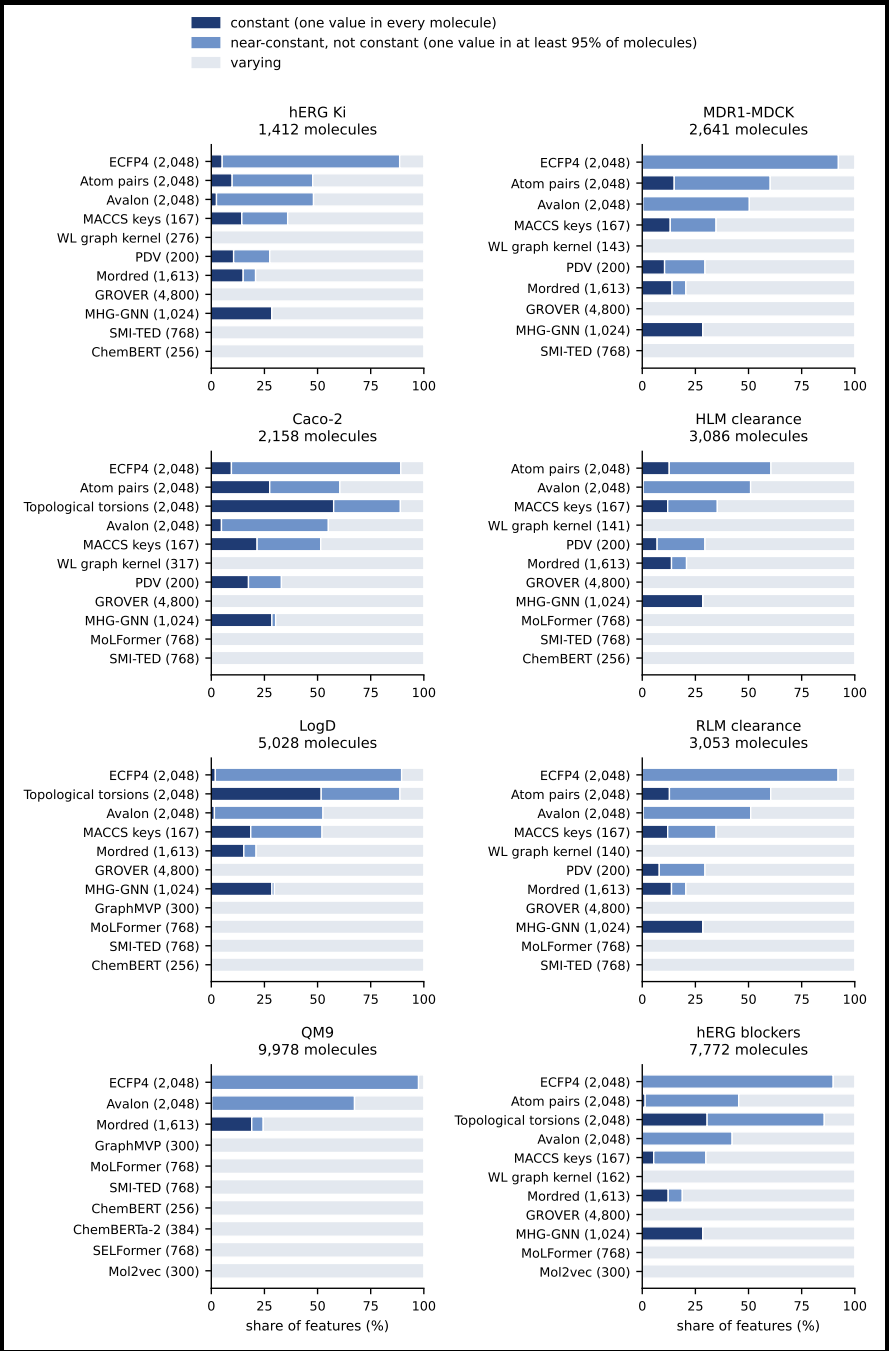


Figure 4.1: Shares of constant (dark blue), near-constant (mid blue) and varying (gray) features of each representation on eight datasets. The number of features of each representation is in parentheses.

Representation	Features	hERG K_i (1,412)	Caco-2 (2,158)	LogD (5,028)	QM9 (9,978)
ECFP4	2,048	105 / 1,815	194 / 1,826	39 / 1,834	0 / 1,997
Atom pairs	2,048	201 / 977	563 / 1,239	–	–
Topological torsions	2,048	–	1,179 / 1,822	1,059 / 1,818	–
Avalon	2,048	50 / 985	98 / 1,128	30 / 1,076	4 / 1,380
MACCS keys	167	24 / 60	36 / 86	31 / 87	n/a
WL graph kernel	140–317	0 / 0 / 276	0 / 0 / 317	–	–
PDV	200	21 / 55	35 / 66	–	–
Mordred	1,613	242 / 336	–	247 / 340	307 / 394
GROVER	4,800	0 / 0	0 / 0	0 / 0	–
MHG-GNN	1,024	292 / 292	292 / 310	292 / 304	–
GraphMVP	300	–	–	0 / 0	0 / 0
MoLFormer	768	–	0 / 0	0 / 0	0 / 0
SMI-TED	768	0 / 0	0 / 0	0 / 0	0 / 0
ChemBERT	256	0 / 0	–	0 / 0	0 / 0
ChemBERTa-2	384	–	–	–	0 / 0
SELFormer	768	–	–	–	0 / 0
Mol2vec	300	–	–	–	0 / 0

Table 4.1: Constant and near-constant features of each representation on hERG K_i, Caco-2, LogD and QM9. Each entry gives the number of constant features and the number of near-constant features, which take one value in at least 95% of molecules and include the constant ones. The number of molecules is under each dataset name. The width of the WL graph kernel varies, so its entries add its number of features. A dash marks a representation outside the dataset's base representations, and no MACCS keys were saved for QM9.

Representation	Features	MDR1–MDCK (2,641)	HLM clearance (3,086)	RLM clearance (3,053)	hERG blockers (7,772)
ECFP4	2,048	1 / 1,890	–	1 / 1,887	0 / 1,840
Atom pairs	2,048	308 / 1,233	260 / 1,240	262 / 1,238	28 / 929
Topological torsions	2,048	–	–	–	625 / 1,754
Avalon	2,048	5 / 1,032	8 / 1,044	8 / 1,045	0 / 866
MACCS keys	167	22 / 58	20 / 59	20 / 58	9 / 50
WL graph kernel	140–317	0 / 0 / 143	0 / 0 / 141	0 / 0 / 140	0 / 0 / 162
PDV	200	21 / 59	14 / 59	16 / 59	–
Mordred	1,613	226 / 331	222 / 336	222 / 331	197 / 304
GROVER	4,800	0 / 0	0 / 0	0 / 0	0 / 0
MHG-GNN	1,024	292 / 292	292 / 293	292 / 293	292 / 292
MoLFormer	768	–	0 / 0	0 / 0	0 / 0
SMI-TED	768	0 / 0	0 / 0	0 / 0	–
ChemBERT	256	–	0 / 0	–	–
Mol2vec	300	–	–	–	0 / 0

Table 4.2: Constant and near-constant features of each representation on MDR1–MDCK efflux, HLM clearance, RLM clearance and the Lhasa hERG blocker training set of 7,772 molecules. Entries read as in Table 4.1.

4.2.2 Molecules with Identical Feature Vectors

A standard fingerprint- or descriptor-based representation can produce the same feature vector for two separate molecules. In turn, models using this representation will produce a single prediction for the molecules with identical features. It should be noted that identical feature vectors within a dataset do not necessarily mean that they derive from the same underlying molecule. The dataset could just hold two different measurements of the same molecule twice, as is possible with data pooled across different sources.

Many fingerprints are computed without stereochemistry [116]. In this research, we build ECFP4 fingerprints with stereochemistry off while the 2D descriptor sets (PDV, Mordred) also ignore it such that for every dataset, each molecule they label a tie is a stereoisomer of another. Any representation read from the 2D graph merges stereoisomers, since the two share an identical graph. Separately, methods using the Weisfeiler–Lehman relabeling scheme are bounded by the 1-Weisfeiler–Lehman test explained in Chapter 2, so some non-isomorphic graphs also receive the same vector; the graph-kernel features tie 105 of 1,412 hERG K_i molecules, 56 of which differ only in stereochemistry and 49 in more than that. Whether a learned embedding differentiates stereoisomers is up to its individual implementation. SMI-TED, a pretrained SMILES encoder, ties 56 hERG K_i, 633 LogD and 177 Caco-2 molecules, all stereoisomers. A tokenizer can drop stereochemistry before the model even sees the input string. The ChemBERTa-77M-MTR tokenizer used in Chapter 3 gives 72 of 1,415 hERG K_i molecules the same token sequence as another molecule. If the difference between stereoisomers was negligible, this wouldn't matter. However, stereoisomers can differ in measured activity [117], meaning these representations are all lacking potentially important chemical information.

For each dataset and representation, we grouped molecules with identical stored feature vectors. A tied molecule is a repeat if another molecule in its group has the same InChIKey, a featurization failure if its vector is all zeros, and a distinct molecule otherwise. A distinct molecule is stereo-only when every molecule sharing its vector has the same InChIKey connectivity block, so they differ only in stereochemistry.

Every molecule in a tied group receives the same prediction, and the prediction with the lowest squared error is the group's mean label. The lowest error any model

can reach is therefore

$$\mathrm{RMSE}_{\min} = \sqrt{\frac{1}{n}\sum_{G}\sum_{i\in G}(y_i - \bar{y}_G)^2}, \tag{4.1}$$

where n is the number of molecules, G runs over the tied groups and $\bar{y}_G$ is the mean label of group G. The best R^2 is $1 - \mathrm{RMSE}^2_{\min}/\mathrm{Var}(y)$. The label spread of a tied group is its highest label minus its lowest. Repeat pK_i measurements of one compound disagree by about 0.54 log units [15], so a larger pK_i spread is not explained by measurement noise alone. We counted ties on hERG K_i (deduplicated by InChIKey), LogD and Caco-2, for each dataset's base representations and augmentation features.

No tie came from a repeated molecule or a row of zeros, so every tie is a merge made by the representation (Table 4.3). Atom pairs, PDV, Mordred and SMI-TED tie only stereoisomers. ECFP4, Avalon and topological torsions tie mostly stereoisomers, plus a few molecules that are not stereo-only. On hERG K_i, ECFP4, atom pairs, PDV, Mordred and SMI-TED all tie the same 56 molecules in the same 26 groups, so concatenating any of these five leaves all 56 tied. The WL graph kernel ties 49 hERG K_i and 26 Caco-2 molecules that are not stereo-only. The MACCS keys tie the most molecules on every dataset, mostly not stereo-only (only 30 of 377 are stereo-only on hERG K_i). The 3D representations and the learned embeddings ChemBERT, GROVER, MHG-GNN, MoLFormer and GraphMVP tie none. Concatenating all of a dataset's representations leaves no molecule tied on any of the three datasets.

On hERG K_i the lowest reachable error is at most 0.134 pK_i (MACCS keys), small against a label standard deviation of 0.915 pK_i. One tied group can still spread widely: the largest ECFP4 spread on hERG K_i is 1.14 pK_i, more than twice the repeat-measurement scale (Figure 4.2). The best any ECFP4 model can predict for this pair is their mean, 5.59 pK_i, which leaves each molecule 0.57 pK_i from its measured value.

The structure–activity landscape index of a pair of molecules i and j is

$$\mathrm{SALI}_{ij} = \frac{|y_i - y_j|}{1 - \mathrm{sim}(i,j)} \tag{4.2}$$

[125]. A tied pair has similarity 1, so its index is infinite whenever the labels differ. Such a tie is the most extreme activity cliff possible, and an artifact of the rep-

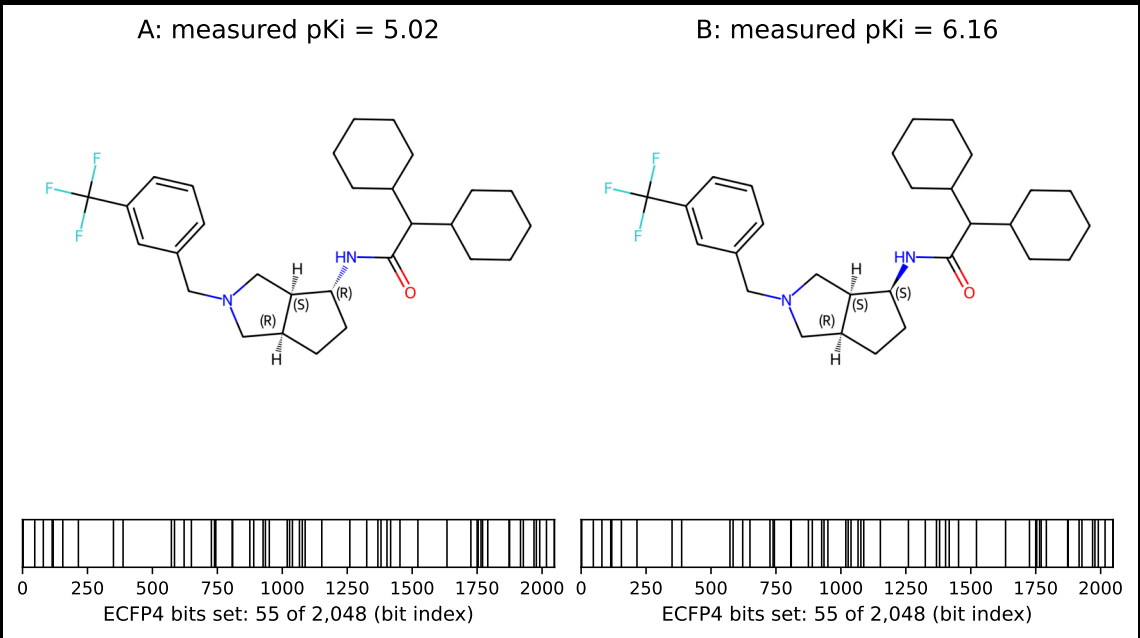


Figure 4.2: Two hERG K_i stereoisomers with identical ECFP4 vectors, which differ at the carbon bearing the amide nitrogen (R in A, S in B). The top row shows the structures with their measured pK_i values, and the bottom row shows the 55 of 2,048 ECFP4 bits set in both.

resentation. A nearest-neighbor method sees a tied neighbor at distance 0 with a different label. It may then flag a correct label as noise (Section 4.4.2).

On hERG K_i, the mean label spread of a tied group is 0.28 pK_i under ECFP4 and 0.39 pK_i under the MACCS keys. Of the 26 ECFP4 groups, 24 hold different labels and 6 differ by more than 0.54 pK_i. Of the 153 MACCS groups, 138 hold different labels and 38 differ by more than 0.54 pK_i.

It is worth noting that some of these representations are more straightforward than others in terms of the underlying chemistry they reference. For example, the PDV carries very clear physicochemical properties such that if you examined one of it's features, you'd know exactly the type of chemical information it contains. Other representations like frozen embeddings, representing the last layer of a network, do not tell us what type of chemical information, if any, they relate to.

4.3 Flaws in Similarity and Other Pairwise Computations

We've already discussed some of the flaws of individual representations, but what happens when you compare two instanes of one of these representations? These

Dataset	Representation	Molecules	Tied	Stereo-only	Best RMSE	Best R^2	Largest spread
hERG K_i (pK_i)	ECFP4	1,412	56 (4.0%)	56	0.041	0.998	1.14
	Atom pairs	1,412	56 (4.0%)	56	0.041	0.998	1.14
	PDV	1,412	56 (4.0%)	56	0.041	0.998	1.14
	Mordred	1,412	56 (4.0%)	56	0.041	0.998	1.14
	SMI-TED	1,412	56 (4.0%)	56	0.041	0.998	1.14
	Avalon	1,412	66 (4.7%)	56	0.049	0.997	1.20
	WL graph kernel	1,412	105 (7.4%)	56	0.071	0.994	1.49
	MACCS keys (aug.)	1,412	377 (26.7%)	30	0.134	0.978	1.80
LogD (log units)	ECFP4	5,028	641 (12.7%)	633	0.044	0.999	1.00
	Mordred	5,028	633 (12.6%)	633	0.043	0.999	1.00
	SMI-TED	5,028	633 (12.6%)	633	0.043	0.999	1.00
	Avalon	5,028	647 (12.9%)	629	0.047	0.998	1.40
	Topological torsions	5,028	675 (13.4%)	631	0.047	0.998	1.10
	MACCS keys (aug.)	5,028	2,343 (46.6%)	437	0.220	0.965	2.80
Caco-2 (log units)	ECFP4	2,158	179 (8.3%)	177	0.027	0.996	0.74
	Atom pairs	2,158	177 (8.2%)	177	0.027	0.996	0.74
	PDV	2,158	177 (8.2%)	177	0.027	0.996	0.74
	SMI-TED	2,158	177 (8.2%)	177	0.027	0.996	0.74
	Avalon	2,158	181 (8.4%)	177	0.027	0.996	0.74
	Topological torsions	2,158	186 (8.6%)	177	0.029	0.996	0.74
	WL graph kernel	2,158	203 (9.4%)	177	0.028	0.996	0.74
	MACCS keys (aug.)	2,158	804 (37.3%)	139	0.111	0.935	1.72

Table 4.3: Molecules that share their feature vector with at least one other molecule, for each representation that ties at least two molecules. Tied gives the number and share of such molecules, and Stereo-only counts those tied only with their own stereoisomers. Best RMSE and best R^2 are reached by predicting the mean label of each tied group, the lowest error any model can reach. Largest spread is the largest label range within one tied group, in the dataset's label units, and (aug.) marks an augmentation representation.

pairwise comparisons, as described in Chapter 2, have the potential to violate the similarty assumption. When considering similarity and other pairwise comparisons, we discuss both the representation and the coefficient.

The standard way of comparing two small molecules is the Tanimoto coefficient, which operates on any bit vector, applied to ECFPs [126]. Independently, the Tanimoto coefficient is described as the "coefficient of choice" for molecular similarity [71], while Morgan fingerprint, represented as ECFP4 in this research, is likewise one of the strongest fingerprints for the task, ranking among the strongest fingerprint for virtual screening and target prediction [127]. ECFP4 and ECFP6 specifically, alongside topological torsion, rank among the best of 28 fingerprints tested for ordering diverse structures by similarity [128]. However, it's not a one-size-fits-all solution. Across 96,456 human metabolites, 72.9% had an ECFP4 nearest neighbor at Tanimoto distance 0 [127], again running into the issue of the representation lacking stereochemical information and more generally being unable to distinguish

molecules.

The Tanimoto coefficient T is defined in Equation 2.3, and the corresponding distance is $1 - T$ [71].

All a similarity coefficient can do is compare every feature, yet it cannot tell which features matter to the prediction task at hand. And the similarity assumption holds only relative to such a measure, which is fixed by the choice of representation. To estimate what this costs, we categorize a representation's features into those relevant to the task, R, and those that are irrelevant, I. We take the squared Euclidean distance as a sum over both:

$$d^2(\mathbf{x}, \mathbf{y}) = \sum_{j \in R} (x_j - y_j)^2 + \sum_{j \in I} (x_j - y_j)^2. \tag{4.3}$$

Every irrelevant feature that varies adds one further term, so the irrelevant part grows linearly with $|I|$. After standardization each feature contributes roughly the same amount, so the share of the distance carrying signal is about $|R|/(|R| + |I|)$. When we use a general-purpose representation like ECFP4 on a dataset spanning a small portion of the chemiccal space, $|I|$ can be a lot larger than $|R|$. Features that remain constant across the dataset contribute nothing. Rather, the damage comes from features that vary but bear no relation to property in question. The worst case scenario involves a single rare bit set in only a handful of molecules. After standarization, uncommon bits take on larger values, outweighing common features that may be more relevant. This is why the near-constant count in Section 4.2.1 matters more than the constant count.

Larger representations, or higher dimensionality, compounds the problem. As the number of dimensions grow, the nearest and farthest molecules end up at nearly the same distance, meaning that these pairwise distances are less able to discriminate, as explained in Chapter 2. We can measure this effect by measuring the relative contrast, defined as the difference between the largest and smallest measured distance divided by the smallest, and hubness, a measure of how unevenely the molecules appear in other molecule's nearest neighbors [129]. These errors propogate along larger data pipelines, starting from the representation into similarity, from similarity into thes set of neighbors a molecule is compared against, and from there into a prediction of some kind.

We count activity cliffs with the rule of Chapter 2. A pair is similar at an ECFP4 Tanimoto similarity of at least 0.55, and the activity gap is set per dataset so that

15% of molecules are cliffs (14.5% on LogD, as the share moves in steps). The gap therefore varies between datasets (Table 4.4). On hERG K_i it is 1.76 pK_i, with 211 of the 1,412 molecules cliffs. On QM9, only 1,093 pairs among 9,978 molecules reach 0.55, and the gap is 0.05 eV, or 0.06 times the median absolute deviation of the labels. On the three datasets from the Biogen set (RLM clearance, MDR1–MDCK efflux and HLM clearance) it is 0.20 to 0.34 log units, or 0.50 to 0.60 times the median absolute deviation.

The count also depends on the representation. Keeping each dataset's gap, we replaced ECFP4 with each other base representation, taking as similar as many of that representation's closest pairs as ECFP4 calls similar. On hERG K_i the number of cliff molecules then ranges from 138 (atom pairs) to 267 (WL graph kernel). Only 57% to 74% of the pairs that ECFP4 calls similar are among another representation's closest pairs, and on QM9 1% to 24% (Table S.11).

We ask two questions of a distance. First, across all pairs, does distance track the difference in property? We summarize the neighborhood-behavior plot of Patterson et al. [130], descriptor distance against activity difference for every pair, as the Spearman correlation between pair distance and absolute label difference. Second, can each molecule's label be predicted as the mean label of its five nearest neighbors?

We computed both for every base representation of the seven regression datasets: the six of Table 5.14 and HLM clearance. Binary representations were compared by Tanimoto distance, and all others by Manhattan distance (next subsection) on standardized, non-constant features. For the all-pairs correlation, datasets above 2,000 molecules were reduced to a random sample of 2,000. For the neighbor prediction, each molecule's five neighbors were drawn from other scaffold groups.

Across the 70 combinations of dataset and representation, the all-pairs correlation ranged from -0.04 to 0.23 and was below 0.10 in 44 of the 70. The Spearman correlation of the five-neighbor prediction ranged from 0.29 to 0.89, and it was the larger of the two in all 70 (Table 4.4). On hERG K_i, ECFP4 gives an all-pairs correlation of 0.06, while its five-neighbor prediction reaches 0.72, with an RMSE of 0.63 pK_i against a label standard deviation of 0.92 pK_i.

Figure 4.3 shows why the two differ on hERG K_i. Of the 996,166 pairs, 93% have an ECFP4 Tanimoto similarity below 0.20 and 1.1% reach 0.55. The median label difference is 0.81 pK_i over all pairs but 0.20 to 0.30 pK_i above a similarity of about 0.4, though 8.3% of the 774 pairs at 0.8 or above differ by more than 1 pK_i. Under Mordred, the closest 5% of pairs have a median difference of 0.44 pK_i,

and every other 5% a median of 0.78 to 1.11 pK_i. The signal sits in this thin tail of close pairs. The all-pairs correlation is set mostly by the other 95%, whereas a five-neighbor prediction looks only at each molecule's closest pairs.

A third view asks how often two representations agree on each molecule's five nearest neighbors. We compared ECFP4, SMI-TED and Mordred (PDV on Caco-2, where Mordred is not a base representation). HLM clearance, which lacks ECFP4 among its base representations, used its stored ECFP4 features. On hERG K_i any two of the three share on average 0.47 to 0.52 of each molecule's five neighbors, and on the three Biogen datasets and QM9 only 0.13 to 0.21 (Table 4.5).

A fourth view asks whether fingerprint distance agrees with a structural distance. For 500 random pairs per dataset we computed the maximum common substructure (MCS) with RDKit [32], allowing each pair 2 seconds. The MCS distance is $1 - n/(a+b-n)$, where n is the number of atoms in the common substructure and a and b are the numbers of heavy atoms of the two molecules. Its Spearman correlation with ECFP4 Tanimoto distance ranged from 0.20 (hERG K_i and HLM clearance) to 0.51 (Caco-2 and LogD).

Coupry and Pogány [131] learn a molecular similarity from graph embeddings, but independently of any endpoint. Zheng and Tropsha [132] did choose the similarity for the task. They predict each molecule's activity as the average over its k most similar molecules by Euclidean distance over descriptors, and choose the descriptors by simulated annealing to maximize the leave-one-out q^2. Itskowitz and Tropsha [133] also optimize the number of neighbors and their weighting. Both select features for a nearest-neighbor model of one endpoint, whereas this thesis selects features across several representations. Every method in Section 4.4 takes a similarity as input and inherits the dilution described here.

Distance Metrics for Continuous Molecular Descriptors

We now consider four commonly used distance metrics and how their formulation impacts down-the-line applications to distance-based noise detection. While binary fingerprints are compared with the Tanimoto coefficient, as described in Chapter 2, continuous features need a different measure.

- Euclidean (L_2): standard geometric distance, $d(x, y) = \sqrt{\sum_i (x_i - y_i)^2}$
- Manhattan (L_1): sum of absolute differences, $d(x, y) = \sum_i |x_i - y_i|$, more robust to outlier features

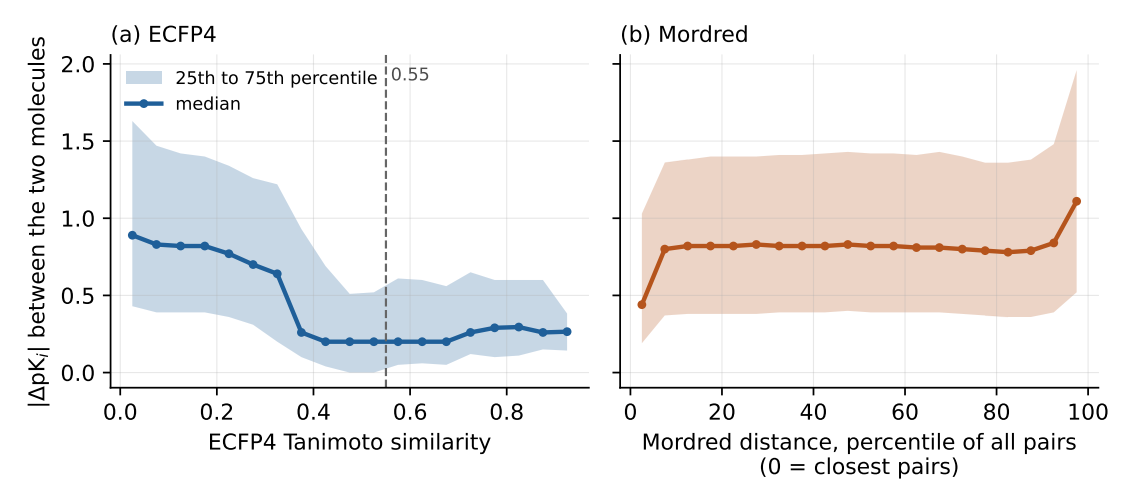


Figure 4.3: Absolute pK_i difference against distance for all 996,166 pairs of the 1,412 hERG K_i molecules, with the median in each bin (markers) and the interquartile range (shaded). a) ECFP4 Tanimoto similarity in bins of 0.05, omitting bins with fewer than 20 pairs, with the activity-cliff threshold of 0.55 dashed. b) Manhattan distance on standardized Mordred features, as a percentile of all pairs.

		All pairs			Five neighbors			Cliffs	
Dataset	Molecules	ECFP4	Desc.	SMI-TED	ECFP4	Desc.	SMI-TED	Molecules	Gap
hERG K_i	1,412	0.06	0.07	0.09	0.72	0.72	0.69	211	1.76 pK_i
LogD	5,028	0.12	0.06	0.10	0.76	0.80	0.76	731	2.90 log
Caco-2 efflux	2,158	0.08	0.18	0.14	0.59	0.62	0.57	324	1.21 log
RLM clearance	3,053	0.02	0.07	0.04	0.36	0.49	0.46	458	0.34 log
MDR1–MDCK efflux	2,641	0.02	0.16	0.10	0.51	0.58	0.59	397	0.20 log
HLM clearance	3,086	−0.00†	0.07	0.03	0.37†	0.49	0.45	462	0.27 log
QM9	9,978	0.19	0.18	0.13	0.88	0.84	0.85	1,495	0.05 eV

Table 4.4: All-pairs correlation, five-nearest-neighbor prediction and activity cliffs for each dataset. All pairs gives the Spearman correlation between pair distance and absolute label difference, on a sample of 2,000 molecules for larger datasets. Five neighbors gives the Spearman correlation between each label and the mean label of its five nearest neighbors from other scaffold groups. Desc. is Mordred, or PDV on Caco-2 efflux. Cliffs gives the number of cliff molecules and the label gap, where a cliff pair has an ECFP4 Tanimoto similarity of at least 0.55 and a label difference above the gap, which is set so that 15% of molecules are cliffs. †ECFP4 is not a base representation of this dataset. Table S.11 gives every representation.

- Cosine: angle-based, $d(x, y) = 1 - \frac{x \cdot y}{\|x\|\|y\|}$, invariant to magnitude
- Mahalanobis: covariance-adjusted distance, $d(x, y) = \sqrt{(x - y)^{\top} S^{-1} (x - y)}$, where S is the feature covariance estimated on the training molecules [134]

All features are standardized to zero mean and unit variance before distances are computed, so that features on different scales contribute equally.

Chapters 5 and 6 use four distances: Manhattan by default, Euclidean, cosine, and Minkowski with $p = 0.5$, $d(x, y) = \left(\sum_i |x_i - y_i|^{0.5}\right)^2$, a dissimilarity rather than

Dataset	Desc.	Five-neighbor overlap			ECFP4 against MCS		
		ECFP4–Desc.	ECFP4–SMI-TED	Desc.–SMI-TED	Spearman	Pairs	Timed out
hERG K_i	Mordred	0.52	0.50	0.47	0.20	495	5
LogD	Mordred	0.38	0.41	0.41	0.51	494	6
Caco-2 efflux	PDV	0.38	0.41	0.38	0.51	492	8
RLM clearance	Mordred	0.15	0.21	0.18	0.28	500	0
MDR1–MDCK efflux	Mordred	0.15	0.20	0.17	0.26	500	0
HLM clearance	Mordred	0.15	0.21	0.18	0.20	500	0
QM9	Mordred	0.13	0.18	0.21	0.46	500	0

Table 4.5: Agreement between representations for each dataset. Five-neighbor overlap is the mean fraction of a molecule's five nearest neighbors, drawn from other scaffold groups, that two representations share. ECFP4 against MCS gives the Spearman correlation between ECFP4 Tanimoto distance and MCS distance on 500 random pairs, with the numbers of pairs used and left out at the 2-second limit.

a metric because it breaks the triangle inequality. Feature weights from the hybrid are applied after standardization (Chapter 6).

4.4 Consequences for Similarity-Based Methods

We have identified flaws in the representations themselves and in the similarities computed over them. For a given molecule, its set of neighbors, built from one representation and one distance, is the input to several other processes such as predicting a molecule's activity from its nearest neighbors, detecting noisy labels by comparing a label against its neighbors' labels, defining an applicability domain by distance to the training set [135], grouping molecules into clusters for data splits [136], identifying activity cliffs, and, in federated learning, weighting each site's vote on a molecule by that molecule's distance to the site's own data [137]. If the distance is flawed, then every application of distance will suffer the consequences.

Section 4.4.1 reports the exploratory LNL study, in which every configuration that used a distance ranked among the worst. Section 4.4.2 turns to noise detection by neighbors, the most direct use of similarity in this thesis.

4.4.1 Exploratory Study of Learning with Noisy Labels

This section reuses an exploratory study of LNL carried out earlier in this project. The model had the stronger effect on the performance lost under noise (Chapter 3), so we asked whether training methods built for noisy labels improve robustness, and

whether the representation still matters under them.

LNL methods and the five groups of Song et al. [100] are described in Section 2.5.2, and such methods have been studied across machine learning, including QSAR. In the study of Cortes-Ciriano et al. [17], gradient boosting tolerated noise worst, SVM and GP models with polynomial kernels were sensitive at high noise in some cases, and the other algorithms degraded smoothly and linearly with the noise level. Most were built for image classification with flipped labels [100], so they do not transfer automatically to continuous labels.

The study tested 12 LNL methods on the QM9 HOMO–LUMO gap (10,000 molecules, scaffold splits), adding Gaussian noise with σ from 0 to 1.0 in steps of 0.1 to the training labels only. The representations were ECFP4, Sort & Slice [138], PDV, canonical SMILES and randomized SMILES. Depending on the method, a configuration could add the model's own uncertainty, a distance between molecules (Tanimoto, Euclidean, Manhattan, cosine or Mahalanobis), an alternative GP kernel or an alternative loss function, giving 221 configurations. Four methods, small-loss selection, confident learning, distance-based selection and multistage cleaning, remove suspected molecules and retrain. These are noise mitigation methods; the other eight are noise-robust training methods.

We measured noise robustness by retention, R^2 under noise divided by R^2 with clean labels, as a percentage. Methods that used a distance did worse than those that did not, and in an analysis of variance over the adapted components, whether a distance was used explained the most variance in R^2. All 20 configurations with the lowest retention used a distance, and none with a distance reached both a clean R^2 above 0.6 and a retention above 80%. The heteroscedastic GP compares molecules through its kernel and still held up, so the failure is specific to judging one label by the labels of its neighbors.

The methods that held up estimate noise from the model's own loss or uncertainty. DivideMix [139] retained the most, 86.7% (standard deviation 0.4%), followed by small-loss selection [140] at 84.2% (2.1%) and the uncertainty curriculum [141] at 82.7% (3.5%). The heteroscedastic GP [142] retained 79.0%, averaged over 15 configurations, with R^2 of 0.71 on clean labels and 0.56 at $\sigma = 0.6$, where its retention was measured. The original study does not state this noise level for the other methods. Regularization alone retained less, 73.0% for Mixup and 74.0% for sharpness-aware minimization. Under these methods, the choice of representation explained even less variance than in Chapter 3.

The study used one dataset, one property and one noise type (Gaussian noise on every training label), and Chapter 6 repeats the question on more datasets and noise types. It also added the distance step to methods designed without one, so poor adaptation is an alternative explanation. The recommendation of noise-robust training over noise mitigation (Chapter 7, "Limitations") comes from this study.

The distances may have failed for two reasons, which the evidence here does not separate. First, a single representation may lack the information a distance needs, with and possibly even without noise. PDVs encode global molecular properties, ECFP4 contains substructure information and MHG-GNNs encode the molecular graph [8], and any one of these sources alone may not be sufficient for robust similarity estimation. Second, some representations are sparse, with many features irrelevant to the molecules of a given dataset (Section 4.3). We sought to address the second with the budget-constrained hybrid representation, which concatenates the top-ranked features by importance from several base representations.

4.4.2 Noise Detection by Neighbors

Neighbor-based noise detection treats a label far from its neighbors' labels as probably wrong, an idea that goes back to editing nearest-neighbor training sets [103]. The wider literature on filtering noisy labels concerns classification [143, 144]. Martín et al. [145] adapt fourteen classification noise filters to regression, seven of them nearest-neighbor filters that use the neighbors' mean label. They find that the filters improve regression methods on 20 general regression benchmarks with injected label noise, none of them QSAR datasets. The neighbor-based noise scores, sample removal, sample weighting, label smoothing and the recovery rate are defined in Chapter 6, "Noise mitigation methods".

The method needs a distance that picks the right neighbors, and neighbors with clean labels. Earlier sections of this chapter argue that the first condition fails, and the second fails whenever noise is spread over the whole training set. A large gap from the neighbors' labels can mean a noisy label, a noisy neighbor or the wrong neighbors, and the score cannot tell which. Activity cliffs also look like noise to such a detector (Chapter 2, "Molecular similarity and activity cliffs").

The exploratory study also used neighbors to clean labels, on 5,000 randomly chosen QM9 molecules (HOMO–LUMO gap, scaffold split) with Gaussian noise on the training labels at $\sigma = 0.3$ and $\sigma = 0.6$. Each cleaning method was judged by

retraining a random forest on ECFP4 with clean, noisy and cleaned labels, giving the recovery rate in R^2 (Chapter 6, "Recovery rate"). Neighbor-based scores compared each label with those of the molecule's k nearest training molecules under Euclidean, Manhattan or cosine distance. Table 4.6 holds only the configurations built on a single representation, because those built on a hybrid used an allocation method that is no longer part of the thesis. This study is separate from the sample-removal study of Chapter 6.

With the neighbor-based scores used as sample weights at strength $\alpha = 0.1$, mean recovery was negative for every representation at both noise levels (Table 4.6). Of all 860 cleaning configurations, 8 had positive recovery at both noise levels; 6 of these used a single representation, and 4 of those 6 were neighbor-based. Label smoothing toward neighbor or model estimates gave positive recovery in 5.0% of configurations at $\sigma = 0.3$ and 0.8% at $\sigma = 0.6$. Mean recovery was negative at every weighting and smoothing strength tried, from 0.1 to 0.7.

	Mean recovery (%)		Positive (% of configurations)	
Representation	$\sigma = 0.3$	$\sigma = 0.6$	$\sigma = 0.3$	$\sigma = 0.6$
ECFP4	−3.0	−7.3	33.3	16.7
PDV	−2.4	−3.4	50.0	27.8
MHG-GNN	−0.1	−7.2	38.9	5.6

Table 4.6: Recovery rate in R^2 when neighbor-based noise scores are used as sample weights ($\alpha = 0.1$), for each representation used to find the neighbors. The task is the QM9 HOMO–LUMO gap on 5,000 molecules with Gaussian noise on the training labels, and the downstream model is a random forest on ECFP4. Positive is the share of configurations with recovery above zero.

Judging a label by its neighbors usually made the model worse and rarely helped at both noise levels, but it did not always fail. The best configuration at $\sigma = 0.6$ was neighbor-based (PDV, Euclidean distance, local variance ratio score) and recovered +17.2%. Because noise was added to every training label, the neighbors' labels are noisy too; the original study gave this as the reason label smoothing failed.

Uncertainty-based methods, which use no neighbors, showed potential at lower noise but degraded at higher noise. Sample weighting did better than label smoothing but usually still hurt, with positive recovery in 14.2% of configurations at $\sigma = 0.3$ and 4.7% at $\sigma = 0.6$, and mean recoveries of −22.9% and −20.9%.

The study also found neighbors in an early hybrid of 100 PDV and MHG-GNN features, built by greedy forward selection, which is no longer part of the method.

Its mean recovery with neighbor-based scores was -0.9% at $\sigma = 0.3$ and -3.0% at $\sigma = 0.6$, with 44.4% and 27.8% of configurations positive. Chapter 6 repeats the measurement in a hybrid and under noise types other than Gaussian.

4.5 Conclusions

On a single dataset, often very little of a standard representation varies. On hERG K_i, 1,815 of the 2,048 ECFP4 features take one value in at least 95% of the 1,412 molecules (Section 4.2.1). Several representations also give different molecules, mostly stereoisomers, the same feature vector, and no model can separate those molecules, whatever their labels. ECFP4 gives 56 of the 1,412 hERG K_i molecules a vector shared with another molecule, all of them stereoisomers ("Molecules with identical feature vectors").

A distance sums over every feature, so when only a few features matter for the property, the rest dominate the sum (Section 4.3). Across 70 combinations of dataset and representation, the Spearman correlation between pair distance and label difference was below 0.10 in 44. Predicting each label from its five nearest neighbors gave a higher Spearman correlation in all 70, so the useful information sits in a few close pairs rather than in the distance as a whole.

In the exploratory LNL study, configurations that used a distance did worst under label noise (Section 4.4.1), and judging a label by its neighbors' labels usually made the model worse (Section 4.4.2). It did not always fail, since one neighbor-based configuration gave the best recovery at the higher noise level. That noise was Gaussian on every training label, so each molecule's neighbors were noisy too; Chapter 6 turns to other noise types.

Of the two possible reasons for this failure (Section 4.4.1), which our evidence does not separate, the requirements below address the second, irrelevant features.

A representation that avoids these flaws would need five properties (Table 4.7). Its features would be chosen for the task, using only each fold's training labels. It would be compact, dropping features that do not vary or do not matter, with the number kept searched rather than fixed. It would draw on several representations, since each misses information the others capture; plain concatenation does this but is not compact (Chapter 5, "Combining representations and the alternatives"). It would work both as a model input and as a space for similarity. Finally, it would be traceable: each chosen feature leads back to its representation and can be read

chemically wherever the base feature can.

Feature selection cannot fix every flaw. Molecules identical in every base representation stay identical in the hybrid, and a selected hashed bit still merges every substructure folded into it. Selection also needs enough labeled molecules to estimate which features matter.

These flaws are where the thesis began (Chapter 1), and Chapter 5 builds the method they led to, the adaptive hybrid representation. The settings that method searches over are listed in Chapter 5, "The Search Space".

Flaw (where shown)	Requirement	Where handled
Features that do not vary ("Constant and near-constant features")	Features chosen for the task, within a budget	"Feature importance scoring"
Molecules merged into one vector ("Molecules with identical feature vectors")	Features drawn from several representations	"Combining representations and the alternatives"
Distance dominated by irrelevant features ("Flaws in similarity and other pairwise computations")	Compact, with the number of features searched	"The search space and why each axis is swept"
Poor performance of neighbor-based methods ("Consequences for similarity-based methods")	Usable as a space for similarity	"Similarity in a hybrid" (Chapter 6)
Features that are hard to read chemically ("Flaws in standard base representations")	Traceable, so that each chosen feature leads back to its representation and can be read chemically wherever the base feature can	"Blueprints and their chemical interpretation"

Table 4.7: Flaws of standard representations and similarity, the requirement each sets for a representation, and where the requirement is addressed, in Chapter 5 unless stated.

Chapter 5

Building Adaptive Hybrid Representations

This chapter asks how several representations can be joined without also joining their irrelevant features. Our answer is the adaptive hybrid representation, a set of at most a fixed number of features, the budget, selected from several base representations by a search for one dataset and one downstream model. The search covers the importance method, the allocation method, the number of representations, the internal evaluator, the budget and the downstream model. Because every selected feature traces back to its representation, each search yields a readable blueprint. We test the method on small molecules, antibodies, reactions, nucleotide sequences and tumor data. **[RESULT NEEDED: datasets where the hybrid beats the best single representation; whether preferred settings differ by dataset and model]**.

5.1 Introduction

Chapter 4 showed that standard representations carry many features that do not vary, or do not matter, on a given dataset, and that distances over all features inherit the problem. It called for features chosen for the task from several representations, with the irrelevant ones removed. Chapter 3 showed that the best representation changes with both the dataset and the model, a reason to search over this choice rather than fix it.

In this chapter we build the adaptive hybrid representation, hereafter the hybrid, and report first results on when and why it helps.

This chapter measures prediction only. Distances in a hybrid, noise detection and learning with noisy labels are covered in Chapter 6.

5.1.1 Combining Representations and the Alternatives

Conjoint fingerprints predicted binding affinity better than their component fingerprints across a range of models [23]. Adding bioactivity-based fingerprints to structure-based ones like ECFP4 improved both predictive performance and scaffold-hopping [146]. Features selected from six non-hashed fingerprints and combined by feature importance score within individual models consistently improved predictive accuracy over the original fingerprints [147].

Most of this work combines standard fingerprints and linear representations like SMILES [23, 146]. We also include GNN embeddings, since graph-based representations capture structural information that fingerprints often lose. We extend how feature importance is computed and how features are selected and joined, and we test augmentation, adding small sets of structural features to one base representation.

Concatenation, the most common strategy, needs only one downstream model but degrades when the feature sets are not homogeneous [148]. It also obscures which representation is responsible for the score. Deng et al. found that appending RDKit2D descriptors to GROVER lowered RMSE on 9 of 24 datasets, yet called the practice "misleading" for measuring the contribution of individual representation types [106]. Late fusion trains one model per source and aggregates the predictions [149], which avoids the dimensionality problem but loses interactions between sources. Intermediate fusion keeps them (MOGONET [150], cross-attention [151]) but must be retrained from scratch to add a source. Multiple kernel learning [152, 153] works only with kernels and yields no interpretable feature subsets. There is no decisive evidence that any of these strategies is broadly superior [149].

In practice, most groups default to one representation, and few combine representations beyond concatenation. Concatenating ECFP4 (2,048 features), ESM-2 (1,280) and Mordred descriptors (∼1,600) gives about 5,000 features, while a dataset may contain only a few hundred labeled samples. At such ratios, more features mean worse predictions [154, 155]. A concatenation of two representations scored between its two parts, rather than above the better one, on **[RESULT NEEDED: count of datasets]**. We propose that this is because concatenation adds the irrelevant features of both parts along with the useful ones, so a method that joins representations

has to combine and filter at the same time.

New models keep appearing, well beyond small molecules. A new downstream model may prefer a different feature set, and it therefore receives its own search. A new model's frozen embeddings enter as one more base representation without any change to the method.

In a comparison of 62,820 models on molecular property prediction benchmarks, RDKit2D descriptors matched or outperformed learned features from graph neural networks and pretrained language models on the majority of datasets [106]. Descriptor-based models held 73% of top-three positions on 11 benchmarks [156]. In 1,310 ChEMBL assays, feedforward networks on ECFP fingerprints outperformed graph convolutions and SMILES-based recurrent models [157]. The input often matters more than the model, yet no single representation wins, and there is no systematic way to exploit this. No single protein language model is consistently best on all tasks [158, 159], and no pathology foundation model dominates on clinical endpoints [160].

The representations used in this thesis are listed in Tables 5.9, 5.10, 5.11, 5.12 and 5.13. A recent review counted over 40 molecular foundation models published between 2020 and 2024 [161].

The hybrid search keeps individual original features from several base representations, so its selection spans representation boundaries. Its closest analogy is perhaps the combined algorithm selection and hyperparameter optimization (CASH) problem [162], to which it adds a third choice, namely which features from which representations enter the model.

Ensembling fits one model per representation and pools their predictions, combining representations after prediction rather than before. Consensus QSAR works this way [163], so it cannot exploit cases where one representation's noise is another's signal. No model in an ensemble sees features from two representations together, and every representation and every model must be computed for each new molecule.

Feature selection keeps a subset of the original features, unlike principal component analysis (PCA), which builds new ones. Of its three families [164], filter methods score features against the labels before any model is fitted, as minimum-redundancy maximum-relevance selection (mRMR) ranks features by their mutual information with the label minus their redundancy with features already chosen [165]. Wrapper methods score subsets of features by fitting the model [166]. Embedded methods select features while the model is fitted, as the lasso does [167].

Recursive feature elimination fits a model, drops the lowest-ranked features and repeats [168].

The hybrid ranks features by a fitted model's importance scores, as embedded methods do, and keeps a fixed number of the top-ranked ones, as a filter does. Classical methods treat all features as one flat list. The nearest classical relative, the group lasso, keeps or drops predefined groups of variables whole [169], whereas the hybrid splits the budget among representations ("Feature allocation methods"). Because only original features are kept, each traces back to one representation, which makes blueprints possible.

PCA replaces the original features with fewer linear combinations of them, ordered by the variance they explain [170]. Its components usually depend on all the original features, so they are hard to interpret, and they depend on feature units unless each feature is z-scored first [170]. This matters when fingerprint bits, descriptors and embeddings are joined. PCA never sees the labels, and components with small variance can still predict the property [171]. Partial least squares (PLS), the supervised counterpart in chemometrics, builds components that covary with the property [172]. Neither gives components that can be traced to one substructure, descriptor or representation, so no blueprint can be read from them.

Every alternative above except ensembling also reduces the number of features, but models may want different numbers of features and different features. Distance-based models such as k-nearest neighbors (kNN) and SVM use every feature in every distance (Chapter 4), whereas tree ensembles choose one feature per split and can partly ignore irrelevant ones. Dablander suggests that using the labels may add a risk of overfitting [138, p. 138], and Sort & Slice, which uses no labels, outperformed two label-based schemes for selecting substructure features [6]. Table 5.1 compares these alternatives with the hybrid.

5.1.2 The Adaptive Hybrid Representation

One approach rarely suits every problem, as Chapter 3 illustrates for NGBoost on clean labels.

Filtering out useless features matters as much as selecting useful ones. Pooling many representations adds irrelevant features that act as noise (Chapter 4), and in an earlier exploratory study, the noise handling methods that used a distance between molecules performed worst (Section 4.4.1).

Method	Uses the labels	Keeps original features	Several representations	Retrain to add one
Concatenation [148]	no	yes	yes	yes
Late fusion and ensembling [149, 163]	only if pooling weights are learned	yes	no	no, one more model
Intermediate fusion [150]	yes	no	yes	yes
Multiple kernel learning [152, 153]	yes	no, kernels only	yes	yes
PCA and PLS [170, 172]	PCA no, PLS yes	no	yes, on a concatenation	yes
Flat feature selection [164]	yes	yes	yes, on a concatenation	yes
Group lasso [169]	yes	yes, whole groups	yes	yes
Adaptive hybrid representation	yes	yes	yes	search rerun, method unchanged

Table 5.1: Methods for combining or reducing representations, compared by whether they use the labels, keep the original features, let one model see several representations, and must refit to add a representation.

The questions this chapter and Chapter 6 address are set out in Chapter 1.

Formally, an adaptive hybrid representation is a set of up to a fixed number of features, the budget, selected from several base representations of the same molecules. A search chooses it for one dataset and one downstream model, so a different dataset, split or model gives a different hybrid. The hybrid is an input rather than a model, so any standard supervised model can be trained on it. It is intended for base representations that describe different aspects of the molecule, such as PDV descriptors, ECFP4 bits, graph embeddings and sequence embeddings.

The budget is split between the representations before any feature is picked, and each representation's features are ranked only against each other. In a single ranking, a wide representation (thousands of features) can crowd out a narrow one (tens), and importance scores from different types of feature are not on a common scale. One pooled allocation method, the global pool, ranks the features offered by every representation together, so that per-representation and pooled selection are compared instead of assumed.

A hybrid can also be used to measure similarity in two ways. Distances can be computed in an ordinary hybrid, or the search can pick features by how well each

molecule's activity is predicted from its five nearest neighbors. Both are defined in "Similarity in a hybrid" (Chapter 6).

5.1.3 Blueprints and Their Chemical Interpretation

A finished search is saved as a blueprint, which records the settings needed to rebuild the hybrid. Because each selected feature traces back to its representation and importance rank, the search returns information about the problem as well as a better score. The best configuration is kept for each downstream model and budget, giving one blueprint per pair, so comparing blueprints at one budget shows whether tree ensembles and neural networks draw on different features.

Some selected features can be read by a chemist. A PDV descriptor has a name, such as molecular weight or logP, an ECFP4 bit can be shown as the atom environment that sets it, and a MACCS key stands for a known substructure. For an antibody the readable unit is a sequence position and its residue, and for gene expression a named gene set, such as a pathway or a chromosome arm. A feature of a learned embedding shows only which representation was useful, not the chemistry (Chapter 4).

Readable features serve two purposes. Substructures or descriptors that recur among the kept features can point to parts of the molecule that may be involved in binding or activity, a starting point for a chemist rather than a finding. Kept features that make chemical sense for the endpoint support the claim that the selection finds real signal instead of fitting noise. Explainability is a recognized need in drug discovery [173]. Directly readable inputs are an alternative to explaining a black-box model after the fact [174]. A selected feature is predictive but not necessarily causal, and a correlated unselected feature may matter as much. Features should also be read from the fit on the dataset in question, since a blueprint rebuilt on other molecules can select other features.

A hybrid also defines a distance chosen for one task, which methods that use nearby molecules as evidence could use in place of a standard fingerprint distance (Section 4.4). The predictor that scores proposed molecules in generative design [175] could also take a hybrid as input, which we leave to future work.

5.1.4 Beyond Small Molecules

The search needs only a supervised task and several feature matrices for the same samples. The default search objective scores prediction only and reads no chemical structure, so the method applies to any tabular task with several encoders or measurements of the same inputs. Two parts are specific to chemistry. Reading substructures from a blueprint requires named substructure features, and an optional objective that compares distances in the hybrid with the maximum common substructure of pairs of molecules applies only to molecules. The nearest-neighbor objective of Chapter 6 needs only labels and a distance, so it is also searched on the antibody and other non-molecular datasets.

Most one-hot sequence features carry no importance, because at most positions the residue never changes across a dataset. One-hot encodings can nevertheless still be useful. Among the four antibody datasets, one-hot is a base representation on nanobody thermostability only, and the position-wise blocks substitution matrix (BLOSUM) encoding on humanness and trastuzumab–HER2 binding. One-hot was among the selected representations on **[RESULT NEEDED: antibody datasets where one-hot was selected, with score]**. We recommend language-model sequence embeddings over raw one-hot encodings.

Antibodies, the main domain after small molecules in this work, have four datasets in this chapter, Koenig expression, nanobody thermostability, humanness and trastuzumab–HER2 binding (Table 5.8). Reactions (two Buchwald–Hartwig yield datasets) and nucleotide sequences (the H3K4me3 histone-mark and TATA promoter tasks, Table 5.12) are proofs of concept. On data from The Cancer Genome Atlas (TCGA), each representation is a different measurement of the patient (Table 5.13), whereas elsewhere each is a different encoder of one input. The search treats both cases the same way.

The hybrid can help only on datasets with enough samples (see "Limitations") and with base representations whose features carry importance. Where every representation already scores poorly, there are too few useful features to improve on.

5.2 Methods: The Hybrid Search

Figure 5.1 outlines the hybrid search.

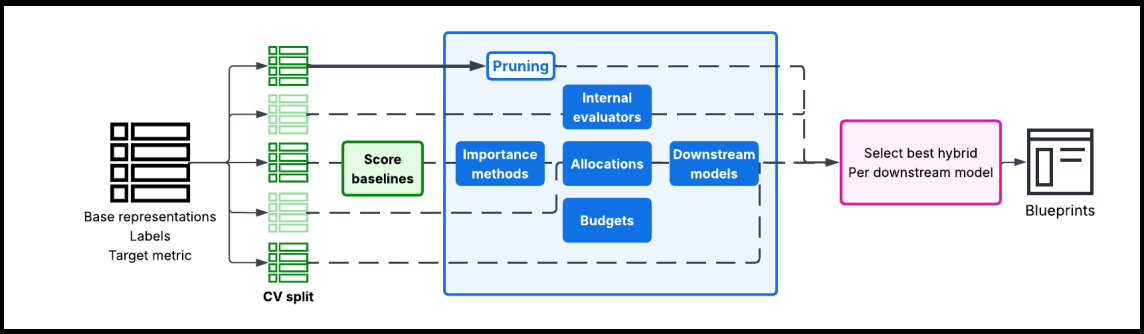


Figure 5.1: Schematic overview of the hybrid search. The inputs (left) are split into five cross-validation folds, and the single representations are scored as baselines. Each fold is searched over importance methods, allocation methods, budgets, internal evaluators and downstream models (center), with optional adaptive pruning on the first two folds. The best hybrid for each downstream model is stored as a blueprint (right).

5.2.1 From Base Representations to a Hybrid

The search is given K frozen representation matrices $\mathbf{X}^{(k)} \in \mathbb{R}^{n \times d_k}$, $k = 1, \ldots, K$, for the same n samples, whose widths d_k range from tens to thousands of features. It also takes a label vector $\mathbf{y}$, one group label per sample for cross-validation (for example its scaffold), the dataset's metric, the task type, one or more downstream models and a list of budgets. It returns a hybrid of at most B features before any uncapped augmentation, where the budget B is a maximum rather than a target, and a blueprint that records the settings needed to rebuild it.

Representations differ in scale, distribution and missingness. Pooling them before splitting would leak validation-set statistics into training, a well-documented source of optimistic bias in feature selection [176, 177], so all preprocessing runs within each cross-validation fold. When a single representation is scored, missing values are imputed with training-fold medians and entirely missing features are dropped. In the hybrid, missing values are set to 0, and every feature, binary bits included, is standardized with training-fold statistics only. Binary features stay 0/1 only when the search scores a single representation to pick the comparator, and representations wider than 5,000 features are pre-screened for that step only (Additional file 17).

Outer folds follow the splitting rules in "Datasets and splitting". A representation learned from the labels is retrained on each outer training fold only. Within that fold, the search runs its own group-aware cross-validation, 5 folds by default. Inside each of its training folds, a deterministic internal split, the first fold of a 5-fold grouped split, holds out about 20% of the molecules as whole groups. With fewer than five groups, both splits use as many folds as there are groups (Additional

file 18). Importance and standalone scores come from the training fold only, and the outer test molecules score only the finished hybrid.

For each representation k, the importance method gives every feature a score $s_j^{(k)} \geq 0$ on the internal training part, which ranks features within that representation only. The standalone score r_k is the internal evaluator's score, fitted on representation k alone, on the internal held-out part in the dataset's metric. We rescale r_k to $\tilde{r}_k$ so that 0 means "no better than a trivial predictor", which makes the budget split independent of the metric. For Spearman, Pearson, the Matthews correlation coefficient (MCC) and R^2, $\tilde{r}_k = \max(0, r_k)$. For AUROC and the concordance index, $\tilde{r}_k = \max(0, 2(r_k - 0.5))$. For RMSE, $\tilde{r}_k = \max(0, 1 - \mathrm{RMSE}_k/\mathrm{RMSE}_0)$, where RMSE_0 is the error of predicting the training mean, and mean absolute error is rescaled likewise against the training median. For accuracy and weighted F1, $\tilde{r}_k = \max(0, (r_k - r_0)/(1 - r_0))$, where r_0 is the score of always predicting the majority training class.

An optional representation count N keeps the N representations with the highest standalone scores on the fold. Each remaining representation receives a weight $w_k = g(\tilde{r}_k)$, with g set by the allocation method, and a share $b_k^* = B\, w_k / \sum_l w_l$ features, rounded by largest remainder and capped at its width. Thus $\sum_k b_k = \min(B, \sum_k d_k)$ and $b_k \leq d_k$, and a representation with $w_k > 0$ keeps at least one feature. The global pool does not use these shares (see below).

Let S_k be the top b_k features of representation k by $s_j^{(k)}$, among those with $s_j^{(k)} > 0$; if none scores above 0, its full b_k is taken. The cut exists because tree importance gives exactly 0 to every feature no tree split on, and filling the budget with those would pick them by their position in the matrix. The hybrid is $\mathbf{H} = [\mathbf{X}^{(1)}_{:,S_1}, \ldots, \mathbf{X}^{(K)}_{:,S_K}]$, with $\sum_k |S_k| \leq B$, so it can be narrower than its budget.

A configuration assigns one value to every searched setting. It is scored by fitting the downstream model on the hybrid in each fold of the search and averaging the held-out scores. The winner is picked separately for each pair of downstream model and budget, never across models. Because this score chose among many configurations, it is optimistic [178, 179], so the comparison we report is nested. The whole search runs inside each outer training fold, and the winning settings are rebuilt there and scored once on the untouched outer test fold, both for the winner at each budget and for each downstream model's best across budgets. The inner search score exceeds the outer test score by **[RESULT NEEDED: inner minus outer test score, per dataset, in its own metric]**.

Chapter 6 uses two variants of this search. Under label noise, training quantities use the noisy labels and held-out scoring uses clean labels unless stated otherwise. Under the nearest-neighbor objective, each molecule's activity is predicted from its five nearest neighbors in the feature space (Chapter 6, "Similarity in a hybrid").

Blueprints. A blueprint records the search settings needed to rebuild the winning hybrid, so that its features can be examined and the hybrid rebuilt on new datasets. On other molecules, a rebuild re-scores the representations and re-picks the features, so it need not recreate the same hybrid. Some fields serve analysis only, such as which representations fed the hybrid on each fold, and each representation's stored score, measured on every molecule the search saw. A rebuild never splits the budget with this stored score, and the per-fold rebuild in the experiments reads it only to choose representations when the representation count is set and no per-fold scores are given.

Listing 5.1 lists every field of a blueprint and its type.

```
Blueprint
  # What the search chose. A rebuild reads all of these.
  reps                  : list[str]           # every representation searched, not only those used
  rep_count             : int or None         # how many of them may contribute; None = all
  budget                : int                 # most features the hybrid may have
  allocation_method     : str
  importance_method     : str
  internal_evaluator    : str
  objective             : str                 # 'predictive' or 'similarity'

  # How the search scored configurations.
  metric                : str
  lower_is_better       : bool
  task_type             : str                 # 'regression' or 'classification'
  n_folds               : int
  seed                  : int

  # Augmentation settings. A rebuild reads all of these.
  augmentation_strategy : str
  aug_per_source_budget : int, or {source: int}, or None
  augmentation_margin   : float
  augmentation_fraction : float or None
  candidate_ranking     : str
  augmentation_threshold: float or None
  augmentation_budget   : int
  aug_max_candidates    : int

  # Stored for inspection.
  rep_scores            : {representation: float}
                          # each representation's best score over every molecule the search saw;
                          # the rebuild uses it to choose which representations to keep only when
                          # rep_count is set and no per-fold scores are given
  allocation_per_fold   : {fold: {representation: int}}      # features each representation got
  kept_augmentations    : {fold: {source: list[int]}}        # extra features kept, by index
```

Listing 5.1: Fields of a blueprint and their types. Comment lines group the fields, and each field is given with its type and, where needed, a comment on its meaning.

The blueprint stores how many features each representation received in each fold

and which augmentation features were kept, but not the selected base-representation features. It can be reused in two ways. Re-running its settings on new labeled data recomputes importance and standalone scores, so the features adapt but may differ. Alternatively, the exact features of a labeled fit can be taken from new molecules, which needs no labels and so is the only option for unlabeled molecules, but assumes the same features matter. Importance and allocation can also be computed within k-means clusters of the molecules' ECFP fingerprints, to ask whether the budget split changes across chemical space.

Because the blueprint does not store feature names, a separate explanation step, run on a finished hybrid, records where each selected feature came from and, for some representations, what it means. An ECFP4 bit is named by the atom environment that sets it, written as a SMILES fragment from a molecule with the bit set. MACCS keys are named by their SMILES arbitrary target specification (SMARTS) pattern and a short name, and descriptor features by descriptor name. Protein composition, descriptor and k-mer features, antibody composition, one-hot, BLOSUM, structural and inverse-folding features, gene-expression pathway sets and PCA components are named too. MAP4 and DRFP features get only a bit index, and Avalon, Mordred, atom-pair, graph-kernel and learned-embedding features only the representation name and dimension index.

5.2.2 The Search Space

Selecting features raised several questions: how to measure a feature's importance, how many features to take from each representation, and how to score a candidate feature set. As with the choice of representation, different prediction tasks preferred different answers, so the search treats each question as a setting and chooses its value per dataset.

Every combination of setting values is built and scored, because the settings interact; for example, the best budget depends on the downstream model, as shown below for nanobody thermostability. The grid measures, rather than assumes, the chapter's claim that no single setting suits every dataset, a claim motivated by, but not proved by, the result that no learning method is best on every problem [101].

Six settings are searched, namely the downstream model, the budget, the representation count, the importance method, the allocation method and the internal evaluator. Table 5.6 gives their values at the full and fast tiers, and Additional files,

Section S.3, the defaults and the number of configurations per downstream model. The importance method is LightGBM gain, TreeSHAP or DeepLIFT. It is searched because importance depends on the model as well as on the data [180]. The three methods come from different model families and rank features differently, and a different ranking builds a different hybrid. The search objective and augmentation are not searched within a single search; each objective and each augmentation condition is a separate search.

The downstream model is trained on the finished hybrid. Five are searched, namely random forest, XGBoost, LightGBM, SVM and a deep neural network (DNN), and a Gaussian process is added on datasets under 1,000 molecules ("Models and metrics"). The downstream model determines which features are useful and how many it can use, so the best hybrid for one model need not be the best for another.

The budget is the maximum number of features in the hybrid, and eight budgets are searched (10, 25, 50, 100, 250, 500, 1,000 and 2,000 features). Too few features leave informative ones out, and too many add uninformative ones that give small datasets room to overfit. Budgets above the number of training molecules are kept. In a preliminary 5-fold run on nanobody thermostability, with an earlier budget list that included 300 features, SVM performed best at 300 features (Spearman 0.6384), DNN, LightGBM and random forest at 50, and XGBoost at 100. **[RESULT NEEDED: best budget per downstream model on nanobody thermostability, full runs]**

Each dataset has a fixed set of base representations, chosen before the search (Table 5.14). The representation count (2, 3, 5 or all) sets how many may contribute features, keeping those with the highest standalone scores on each fold. It is the only step that drops whole representations before allocation, and correlation between representations plays no part in it. It is searched because datasets differ in how many representations are worth using, and pooled feature selection over all representations has no such setting. Every allocation method is paired with every count. In preliminary runs, some datasets performed best with 2 or 3 representations and others with all 10. **[RESULT NEEDED: datasets best with 2 or 3 representations and with all, full runs]**

Feature Importance Scoring

The importance method gives every feature of one representation a single non-negative score, from a model freshly fitted on that representation alone; the downstream model is not used. Only the order within a representation is used, so scores are not normalized. Split-based tree importance is biased toward features with many possible split points [181], which matters here because binary fingerprint bits sit beside continuous embeddings.

No single importance method worked best across datasets and downstream models, and the preferred method changed with the dataset. We kept three methods whose rankings did not correlate strongly with each other and whose selected features performed well as model input (Additional file 15).

We score features using LightGBM gain importance, SHapley Additive exPlanations (SHAP) values for tree ensembles (TreeSHAP) and DeepLIFT (Deep Learning Important FeaTures) [182]. LightGBM gain measures how much a feature reduced the training loss where trees split on it [57]. TreeSHAP gives each feature's exact Shapley value for each molecule's prediction from a tree model [183]. DeepLIFT measures how much a feature moves the output of a small network, trained with Adam [184], away from its output on an all-zero reference input, which after standardization places every feature at its training mean [182, 185, 186]. On Caco-2, the median share of features scoring above 0 was 30% for LightGBM gain, 70% for TreeSHAP and 90% for DeepLIFT (preliminary, over 10 representations and 3 folds). Table 5.2 gives the settings of each method, including the pre-screen of representations wider than 2,000 features.

The full search sweeps all three importance methods, whereas the fast tier uses LightGBM gain only (Table 5.6).

Feature Allocation Methods

The allocation method decides how many of the B features each representation contributes; the importance method decides which. Of the five methods searched, four give each representation a fixed share computed from its rescaled standalone score (linear, squared, softmax and rank-decay allocation). The fifth, the global pool, lets the candidate features offered by every representation compete in one importance ranking.

The four fixed-share methods compute shares from the standalone score on the

Method (family)	Model fitted per representation	Score of one feature	Classification labels	Representations over 2,000 features
LightGBM gain (split gain)	LightGBM, 50 trees, maximum depth 6	Loss reduction of every split on the feature, summed over all trees; 0 if never split on	One model on the class labels	Only the 2,000 highest-variance features are fitted; the rest score 0
TreeSHAP (Shapley values)	Random forest, 50 trees, maximum depth 10	Mean absolute Shapley value over the training molecules	Binary: positive class. Multiclass: mean absolute value over classes	As for LightGBM gain
DeepLIFT (reference-based attribution)	Small neural network, ReLU hidden layers of 64 and 32 units, 100 full-batch epochs of Adam, learning rate 0.001	Mean absolute attribution against an all-zero reference, Rescale rule	Binary: positive class. Multiclass: mean absolute value over classes	Every feature is scored

Table 5.2: Settings of the three importance methods. Each method is fitted separately on every representation inside each fold, and settings not listed are the library defaults.

internal validation part. Only linear allocation makes the share proportional to that score. All four assume that representations with stronger predictive performance contribute more valuable features.

The assumption fails when a strong representation owes its score to a few features, so that a large share fills with weak ones. Fixed shares also ignore complementarity, so two representations carrying the same information both receive a share. Only the global pool ranks features from different representations against each other.

The fixed-share methods differ only in how w_k is computed from $\tilde{r}_k$ (Table 5.3), with settings fixed for every dataset. Under linear and squared allocation, a representation with rescaled score 0 receives 0 features. Under softmax and rank decay, no representation receives 0 features because of its score. Since every rescaled score lies between 0 and 1, softmax cannot give one representation most of the budget, and with 10 representations the best takes at most 23.2% of it. Rank decay uses only the order, so with 10 representations the best receives 27.3% and the tenth 1.8%, whatever the gaps between their scores. Table 5.4 gives a worked example.

The global pool sizes each representation's offer with squared weights $\tilde{r}_k^2$, as shares of a pool of $2B$ candidate features, each offer capped at the representation's

Method	Weight w_k	Fixed setting	How features are chosen	When a representation receives 0 features
Linear	$\tilde{r}_k$	none	Top b_k of its own ranking	$\tilde{r}_k = 0$
Squared	$\tilde{r}_k^2$	power 2	Top b_k of its own ranking	$\tilde{r}_k = 0$
Softmax	$\exp\left((\tilde{r}_k - \max_l \tilde{r}_l)/T\right)$	$T = 1$	Top b_k of its own ranking	never from its score
Rank decay	$e^{-0.3\rho_k}$, ρ_k = rank by score, 0 for the best	decay 0.3	Top b_k of its own ranking	never from its score
Global pool	$\tilde{r}_k^2$, sizing its offer to a pool of $2B$ candidates	power 2; pool $2B$	Top B of one joint ranking over all offered candidates	$\tilde{r}_k = 0$, or none of its candidates in the joint top B

Table 5.3: The five allocation methods. w_k is the weight computed from representation k's rescaled standalone score $\tilde{r}_k$, B is the budget, b_k is the share of representation k, ρ_k is its rank by score and T is the softmax temperature. Under every method, a representation left out by the representation count, or whose standalone fit fails, receives no features.

Method	$\tilde{r}_1 = 0.6$	$\tilde{r}_2 = 0.5$	$\tilde{r}_3 = 0.2$
Linear	46	39	15
Squared	55	39	6
Softmax	39	35	26
Rank decay	44	32	24

Table 5.4: Worked example of the four fixed-share allocation methods for a budget of 100 features and three representations. Each entry is the number of features a representation receives, with shares rounded by largest remainder.

width and holding only features scoring above 0. The same importance method is fitted once on the joined offers, and the hybrid keeps the top B features that score above 0. Each representation's count is therefore an outcome of the joint ranking, with no minimum of one feature, and a feature's rank depends on what the other representations offered. The cost is one extra importance fit per configuration.

The five methods were chosen in a preliminary comparison (Additional file 15).

Search Configuration and Internal Evaluation

Features that help a random forest may not help an SVM, and vice versa, because variable importance depends on the model [180]. The internal evaluator is a model fitted on one representation at a time, separate from the downstream model. Four are searched, namely random forest, DNN, LightGBM and the matched evaluator, a

model of the downstream model's own family, and none is chosen in advance. Each is fitted on the internal training part and scored on the internal validation part in the dataset's metric.

The internal evaluator's scores set how the budget is split and which representations are kept; it does not rank features within a representation. Judging a feature set by the score of a model fitted on it is the wrapper idea of Kohavi and John [166].

The full search tests every combination of the six settings under 5-fold cross-validation, giving 8,160 configurations per dataset where all five downstream models run, fewer where one is left out, and 10,080 on humanness, where a Gaussian process is added. A fast-tier search has 816. Additional file 14 gives the breakdown by downstream model and the number of model fits.

Importance scores are computed once per fold and importance method, and standalone scores once per fold and internal evaluator. Every configuration reuses them, so the cost grows more slowly than the number of configurations.

Preliminary design experiments set the values each setting may take (Additional file 15).

For random forest, LightGBM and DNN downstream models, the matched evaluator is already among the four, so these are searched with 3 evaluators. For XGBoost and SVM, and for the Gaussian process on humanness, it adds an evaluator of that family, giving 4. Only the DNN evaluator shares its downstream model's architecture and training settings, and even it draws its 20% early-stopping molecules at random, whereas the downstream DNN splits them by the outer split's groups. Additional file 12 gives every evaluator setting.

5.2.3 Augmentation

Some feature sets are too small or too specialized to take part in budget allocation, such as MACCS fingerprints (167 features) or log-likelihood ratios (3 features). Others, such as mutation features that record differences from a parent antibody, carry meaning only beside a full representation. Excluding them would lose potentially useful signal, but including them would distort the budget split, since they cannot contribute many features. We therefore add them as augmentation, calling the finished hybrid **H** the base and the feature sets that supply extra features the donors.

Donors are set per dataset (Augmentation column of Table 5.14). They range from small fingerprints such as MACCS to structural sets such as 3D pharma-

cophores and Coulomb matrix eigenvalues, and include features that describe only differences between variants, such as antibody mutation features. MACCS is a donor on four of the six small-molecule datasets and a base representation on MDR1–MDCK efflux. On QM9, MACCS is not offered and the topological torsion fingerprint (2,048 features) is a donor, the only wide one. Bahia et al. [187] found that combining 2D and 3D descriptors gave many more significant models across six protein–ligand series (461 structures), which they attribute to complementary properties. Their 3D descriptors came from bioactive conformations in the Protein Data Bank, whereas our 3D donors use one generated conformer per molecule.

Donor features are ranked by what the base gets wrong. Inside each outer training fold, an 80/20 internal split keeps each scaffold or sequence group on one side. A small random forest (50 trees, depth at most 10) is fitted on the base features, and 3-fold group-aware cross-validation gives out-of-fold residuals $r_i = y_i - \hat{y}_i^{\text{oof}}$ for the internal-training molecules, where for binary tasks $\hat{y}$ is the positive-class probability. In-sample residuals would shrink toward zero. All donor features are pooled, each is scored by its absolute Pearson correlation with the residuals, $c_j = |\rho_{\text{P}}(\mathbf{x}_j, \mathbf{r})|$, and the top $M = 50$ go forward. Ranking against $\mathbf{y}$ would favor features that repeat what the base captures; ranking against $\mathbf{r}$ favors features that explain its errors, as in gradient boosting [188] and least angle regression [189].

Blocks of the top $k \in \{0, 5, 10, 20, 40\}$ ranked features, capped at the number of candidates, are each added to the base and scored with the downstream model on the internal-validation molecules. The chosen k^* has the best score and must beat $k = 0$, which wins ties, so keeping no extras is the default. Five block choices instead of about 50 accept-or-reject decisions leave less room to overfit a small validation set [179]. The chosen donors and feature positions are stored and re-extracted for the outer fold. **[RESULT NEEDED: how often $k^* = 0$, and per-fold scores for no, uncapped and capped augmentation, per dataset and downstream model]**

In uncapped mode, extras are appended outside the budget, so the final width is $|\mathbf{H}| + k^*$, at most $B + 40$ features. In capped mode, they displace base features and the width stays at most B. The base features are ranked once by random forest importance (50 trees, depth at most 10) and the k^* weakest are dropped, after any unspent budget is filled, keeping at least one base feature. Uncapped augmentation and no augmentation differ in width at the same B, so their comparison mixes better features with more features. Capped mode, where an extra must beat a base feature,

removes this difference.

Augmentation is a separate condition rather than a searched setting, so each dataset with donors is searched three times, with no, uncapped and capped augmentation. Fourteen of the 16 datasets run all three, while humanness and trastuzumab–HER2 binding have no donors. The three conditions also run under the nearest-neighbor objective, on the datasets given in Chapter 6, "Similarity in a hybrid". In a second use, a user keeps one strong representation at full width as the base and adds a few extras, with B its width, so only uncapped mode makes sense. We test this use, with planted decoy features, in "Comparators, controls and ablations". This use matters for adoption, because a chemist can keep the representation they already trust and learn which few outside features help, and how many. Augmentation also asks directly whether another representation holds signal the base lacks (Chapter 4). On Koenig expression, the mutation features and the ESM-2 log-likelihood ratios reach the model only through augmentation. **[RESULT NEEDED: donors kept most often, per domain]**

The residual ranking has four limitations. $|\rho_{\mathrm{P}}|$ captures only linear relations, one feature at a time. It ignores redundancy, so two near-identical donor features can both rank high. Donors are pooled with no per-donor limit, so a 2,048-feature donor gets more chances than a 49-feature one. Finally, the margin over $k = 0$ is zero, so a near-tie on one internal split can admit extras.

Table 5.5 summarizes the three conditions.

Condition	Final width (features)	What an extra feature must beat	Datasets that run it
No augmentation	at most B	nothing is offered	all 16
Uncapped	$\lvert\mathbf{H}\rvert + k^*$, at most $B + 40$	keeping no extras	the 14 with donors
Capped	at most B	keeping no extras, and the weakest base feature it replaces	the 14 with donors

Table 5.5: The three augmentation conditions. B is the budget and $|\mathbf{H}|$ the width of the base hybrid, both in features, and k^* is the chosen number of extra features, which are drawn from a small or specialized feature set offered to the hybrid.

5.2.4 Fast Tier and Adaptive Pruning

The search can be made cheaper in two ways. The fast tier uses shorter value lists for some settings, fixed in advance and identical on every fast-tier dataset, and accepts a possibly worse answer for a much smaller grid. Adaptive pruning decides during the search from the dataset's own fold scores, and aims to find the same best configuration while no longer scoring configurations that have clearly lost. Without either, the full search does not finish on the largest datasets.

Six of the 16 datasets use the fast tier, namely QM9, LogD, Koenig expression, trastuzumab–HER2 binding, H3K4me3 and TATA promoter (marked in Table 5.14). A dataset uses it when the full search would take longer than about 40 hours, the time four consecutive 12-hour jobs cover, or when it has more than 10,000 molecules or sequences. Estimated full-search run times were about 103 hours for QM9, 64 for Koenig expression, 62 for LogD and 39 for TATA promoter, and H3K4me3 has 18,244 sequences. The values the fast tier drops are still searched on every other dataset, so it is a compromise on compute, not a claim that those values are useless.

The fast lists were chosen by one rule in a preliminary comparison (Additional file 15).

Search Tier Settings

Table 5.6 gives the values of each setting at both tiers.

Adaptive Pruning

In adaptive pruning, every configuration is first scored on the first two folds and ranked by its mean over them, in the dataset's own metric. Budgets are scored from smallest to largest. Once every budget up to 100 features is scored, a checkpoint runs within each group of configurations that share a downstream model, internal evaluator and representation count. Pairs of allocation and importance method with no configuration in the group's better half are not searched at larger budgets, although at least one pair per group is kept. Each setting is then voted on within groups that fix every other setting, budget included. The best few values in each group receive a point, and a value is removed if it receives a point in fewer than a cut-off share of groups (Table 5.7), but a vote never removes every value. The survivors are scored on the remaining folds, and the best is picked by its mean over all five folds, as without pruning.

Setting	Full tier	Fast tier
Downstream models	Random forest, XGBoost, LightGBM, SVM[a], DNN; Gaussian process[b]	Random forest, XGBoost, LightGBM, SVM[a], DNN
Internal evaluators	Matched evaluator, random forest, LightGBM, DNN	Same as full
Feature budgets (features)	10, 25, 50, 100, 250, 500, 1,000, 2,000	Same as full
Allocation methods	Linear, squared, softmax, rank decay, global pool	Linear, squared, rank decay
Importance methods	LightGBM gain, DeepLIFT, TreeSHAP	LightGBM gain
Representation counts	2, 3, 5, all representations	3, all representations

[a]SVM is not fitted above 10,000 molecules (5,000 when augmentation features are added outside the budget). [b]Added only on Humanness. Of the 16 datasets, it is the only one that is under 1,000 molecules, has its size recorded and is not excluded for run time.

Table 5.6: Values searched for each setting at the full and fast tiers. Configurations removed by adaptive pruning are scored on only the first two of the five folds. The matched evaluator belongs to the same model family as the downstream model.

Configurations far behind after two folds would only confirm their loss on the other three, so pruning stops scoring them. Its one criterion is never to remove a configuration that would have been the best; time saved is secondary. As submitted, every dataset's search uses pruning, at both tiers, and on fast-tier datasets the importance vote does not run because only one importance method is searched. Successive halving [190] and Hyperband [191] likewise drop hyperparameter configurations on partial evaluations, here two of the five folds.

Because votes are grouped by every other setting, a value that is best for a random forest at 25 features is not removed for losing to a DNN at 500 features. One fold is not enough. In the representation-count vote on breast cancer PAM50 subtype after fold 1 alone, 726 of 1,020 groups had every value at an identical score, and ties broken by list order then remove values by accident. After one fold, no cut-off of any vote was safe on the calibration datasets, whereas after two folds each vote had at least one safe cut-off.

The cut-offs were calibrated in a preliminary study on five datasets, LogD, hERG K_i, breast cancer PAM50 subtype, nanobody thermostability and humanness, by replaying each rule on fold scores from an unpruned five-fold run. A cut-off counts as safe if the best five-fold score is unchanged for each of the 25 combinations of

Rule	How groups are formed	Kept, or receives a point	Cut-off	Removed on the calibration datasets
Checkpoint at 100 features	configurations at budgets up to 100 features with the same downstream model, internal evaluator and representation count	(allocation, importance) pairs with a configuration in the better half of the group	better half of the configurations	configurations within groups only; never a whole allocation or importance method
Representation count	every other setting fixed, budget included	best 1	10% of groups	the count of 5 on LogD; the count of 2 on breast cancer PAM50 subtype
Allocation method	every other setting fixed, budget included	best 2	20% of groups	softmax on LogD
Importance method	every other setting fixed, budget included	best 2	20% of groups	DeepLIFT on nanobody thermostability and hERG K_i; TreeSHAP on LogD
Internal evaluator	every other setting fixed, budget included	best 3	10% of groups	the SVM evaluator on LogD

Table 5.7: The adaptive pruning rules. Every rule ranks configurations by their mean score over folds 1 and 2 in the dataset's own metric, and the last column gives what the rule removed on the five calibration datasets in a preliminary run.

downstream model and budget, on every dataset. Among safe cut-offs that made the same cuts, we chose the one whose neighboring cut-off was also safe. TreeSHAP accounts for 71% of LogD's fitting time and was never the best there, so most of the saving comes from it. Estimated run times for eight budgets, without and with pruning, were 79 and 44 hours on LogD, 42 and 35 on hERG K_i, 42 and 30 on PAM50 subtype, 32 and 24 on nanobody thermostability, and 15 and 13 on humanness. The calibration applied the votes one after another, whereas the search applies all four at once to the fold 1 and 2 scores. Five datasets form a small calibration set, which limits the rules.

With pruning on, folds 1 and 2 both select the survivors and score them, so the winner's five-fold mean is slightly optimistic [179]. Single representations are never pruned. A fair comparison with a single representation therefore leaves both voting folds out of both scores, or re-scores without them.

5.3 Methods: Experimental Setup

Every experiment splits each dataset into outer folds as set out in "Datasets and splitting", except one analysis of the antibody expression scan, which uses three outer folds of unequal size. The search sees only the training samples of each outer fold, and the selected features are rebuilt on them and applied unchanged to the held-out samples.

The hybrid is compared with the best single base representation at its full number of features, which is what a user would otherwise run. This comparator is chosen per downstream model and per fold from the search's scores on the outer training samples, then refit on those samples and scored on the same held-out samples as the hybrid, so it can differ between folds. Choosing it on the held-out samples would give it an advantage that the hybrid does not get. The hybrid is held to its budget, plus any augmentation features, so the comparator is often many times wider, and both widths are recorded for every fold, since a difference between the two scores is partly a difference in width as well as in method. The main quantity per fold is the hybrid's score minus that of the best single representation, signed so that a positive value means the hybrid scored better (for RMSE the sign is flipped).

Chapter 6 reuses this protocol with noise added to the training labels of each outer fold, while the held-out labels stay clean.

5.3.1 Datasets and Splitting

The hybrid experiments use 16 datasets from six domains (Table 5.8), whose curation is given in Additional file S.5 and whose representations are listed in Table 5.14. Folds keep each group whole, with groups chosen to match what an unseen sample means in each field.

Pre-processing. Additional file S.5 gives the shared and per-dataset pre-processing steps, with the counts after each step.

Splitting rules. Folds are formed as in Section 3.2: no group is split, and classification folds are stratified with a fixed seed. Every dataset uses five outer folds. No split is random, because random splitting inflates performance estimates for molecular data by placing structurally similar compounds on both sides of the train/test

Dataset	Label (unit)	Task, metric	n	Folds grouped by (groups)	Best single representation (score)
Small molecules: ADME and toxicity					
hERG K_i [110]	pK_i	regression, RMSE ↓	1,412	scaffold (769)	**[RESULT NEEDED: name, score]**
LogD [109]	LogD (log units)	regression, RMSE ↓	5,028	scaffold (2,413)	**[RESULT NEEDED: name, score]**
Caco-2 efflux [109]	$\log_{10}$ efflux ratio	regression, RMSE ↓	2,158	scaffold (1,086)	**[RESULT NEEDED: name, score]**
Rat liver microsome clearance [192, 193]	$\log_{10}$ CL_{int} (mL/min/kg)	regression, RMSE ↓	3,053	scaffold (2,625)	**[RESULT NEEDED: name, score]**
MDR1–MDCK efflux [192, 193]	$\log_{10}$ efflux ratio	regression, RMSE ↓	2,641	scaffold (2,318)	**[RESULT NEEDED: name, score]**
Quantum chemistry					
QM9 [107, 194]	HOMO–LUMO gap (eV)	regression, RMSE ↓	9,978	scaffold (3,452)	**[RESULT NEEDED: name, score]**
Chemical reactions					
Buchwald–Hartwig yield [195]	yield (%)	regression, RMSE ↓	3,955	additive (22)	**[RESULT NEEDED: name, score]**
Buchwald–Hartwig plates [195]	yield (%)	regression, RMSE ↓	4,312	additive (24)	**[RESULT NEEDED: name, score]**
Antibodies					
Koenig expression [196, 197]	expression enrichment ratio (no unit)	regression, Spearman ↑	4,275	mutated position modulo 5 (5)	**[RESULT NEEDED: name, score]**
Nanobody thermostability [197, 198]	melting temperature (°C)	regression, Spearman ↑	672	95% identity cluster (309)	**[RESULT NEEDED: name, score]**
Humanness [199]	human or not (198 : 366)	binary, AUROC ↑	564	95% identity cluster (461)	**[RESULT NEEDED: name, score]**
Trastuzumab–HER2 binding [200]	binder or not (1,639 : 3,361)	binary, AUROC ↑	5,000	mutations from wild type (10)	**[RESULT NEEDED: name, score]**
Nucleotide sequences					
H3K4me3 [201]	mark present or not (9,122 : 9,122)	binary, MCC ↑	18,244	chromosome (23)	**[RESULT NEEDED: name, score]**
TATA promoter [201]	promoter or not (2,637 : 2,637)	binary, MCC ↑	5,274	chromosome (23)	**[RESULT NEEDED: name, score]**
Cancer genomics (TCGA)					
Lung adenocarcinoma vs squamous [202–204]	tumor type (515 : 502)	binary, AUROC ↑	1,017	tissue source site (68)	**[RESULT NEEDED: name, score]**
Breast cancer PAM50 subtype [204–206]	subtype (421 : 192 : 141 : 67)	4 classes, weighted F1 ↑	821	tissue source site (19)	**[RESULT NEEDED: name, score]**

Table 5.8: The 16 datasets of the hybrid experiments after curation, grouped by domain, with the best single base representation of each and its mean score over the five outer folds. n is the number of samples, and the fold groups are given with the number of groups in parentheses. For classification tasks, the class counts are in parentheses in the label column. CL_{int} is the intrinsic clearance, and arrows show whether lower or higher is better.

boundary [207, 208]. On a given dataset, every representation, model and hybrid is scored on the same folds.

Groups by domain. The small-molecule and QM9 datasets are grouped by Bemis–Murcko scaffold (Section 3.2), here keeping stereochemistry. On QM9 the largest group is the empty scaffold of acyclic molecules, with 1,041 of the 9,978 molecules. The two Biogen datasets (rat liver microsome clearance and MDR1–MDCK efflux) have 1.2 and 1.1 molecules per scaffold, respectively, so holding out a scaffold there is close to holding out single molecules. Chapter 3 used a different scaffold key and different curation for its versions of LogD, Caco-2, hERG K_i and QM9, and Appendix S.5 lists the differences.

The reaction datasets are Buchwald–Hartwig C–N couplings from high-throughput plates. Following Ahneman et al. [195], they are grouped by additive, so each held-out fold holds four or five additives unseen in training. The two sets share 3,955 reactions (Additional file S.5).

In the Koenig deep mutational scan of the anti-vascular endothelial growth factor (VEGF) antibody G6, each variant carries one mutation. Variants are grouped by the mutated position modulo 5, as in ProteinGym [209], giving five folds of 836 to 874 variants, so a model cannot look up a per-position effect at test time [210]. The nanobody and humanness sets are grouped by single-linkage clusters at 95% sequence identity, after Kenlay et al. [158]. The trastuzumab variants differ from the wild type only in the ten residues of CDR-H3 and are grouped by how many of them are mutated.

The two nucleotide tasks, from the revised Nucleotide Transformer benchmark [201], are grouped by chromosome, so each held-out fold holds two to six of the 23 chromosome groups (Additional file S.5).

The TCGA datasets use RNA sequencing (RNA-seq) expression and other measurements of primary tumors, one per patient, from UCSC Xena [204]. Samples are grouped by tissue source site, the institution that supplied the sample, so a model cannot score well by recognizing the site.

Consequences of the grouping. Some groupings give very unequal folds. The trastuzumab held-out folds range from 13 to 3,035 variants, with binder shares from 15.5% to 92.3% against 32.8% overall, and the breast cancer folds from 3 to 403 tumors. None of the 68 lung sites supplied both tumor types, so one lung fold holds

20 adenocarcinomas and 113 squamous tumors. Per-fold scores on these datasets spread more widely, so results are read per dataset and never pooled.

5.3.2 Representations by Domain

We combine a variety of component, or base, representations to create these hybrids.

Any representation that gives one fixed-length vector per sample can serve as a base representation, as set out in the description of how the hybrid is built earlier in this chapter. The first version of the method used three molecular representations: PDV, ECFP4 and MHG-GNN. These three are now one small-molecule example among many. Every base representation is frozen: it is computed once from the raw input (a SMILES string, a sequence, a reaction or a patient profile) and is never trained on the labels.

For each dataset we pre-selected at most ten base representations that carry different information and each showed some positive result on its own for the task.

To choose them, every candidate representation was first scored on its own with a gradient-boosted tree model [57] under the dataset's grouped folds. Candidates were then compared by the Spearman correlation between their out-of-fold predictions, since two representations whose predictions correlate highly carry much the same signal. At most ten were then chosen by hand, looking at each candidate's own score next to these correlations and favoring representations that encode the input in different ways. Two nucleotide datasets have fewer. H3K4me3 had 12 candidates, and four of them predicted almost the same as a better-scoring candidate, with a Spearman correlation of 0.95 or above between their out-of-fold predictions; eight remain. TATA promoter had 10 candidates, one was dropped by the same rule, and nine remain. Augmentation features do not count toward the ten. They are added after the hybrid is built (see the Augmentation subsection above), and the same representation can be a base representation on one dataset and augmentation on another. The most and least correlated pairs of base representations on each dataset, by the Spearman correlation of their out-of-fold predictions, are **[RESULT NEEDED: most alike pair]** and **[RESULT NEEDED: least alike pair]**.

If an input cannot be parsed or embedded, it gets an all-zero row, so that every representation of a dataset keeps the same rows. In computed descriptors, a missing or infinite value is set to 0, and values are stored unscaled. A pretrained model gives one vector per input, the mean of its last-layer token vectors with padding excluded,

unless the tables say otherwise. Scaling and filling of missing values happen inside each fold, on the training rows only, as described in the construction of the hybrid.

Small molecules. The small-molecule representations (Table 5.9) are the fingerprint, descriptor, graph and language-model families described in Chapter 2. The only additions are four sets of 3D shape and pharmacophore features, used as augmentation only. All four are computed from a single conformer per molecule, embedded with a fixed seed and optimized with the Merck molecular force field. A single conformer may misdescribe flexible molecules.

Chemical reactions. Reaction datasets (Table 5.10) use three kinds of representation. The condensed graph of reaction (CGR) merges reactants and products into one graph in which the bonds that change are marked [211]; here it is hashed into a circular fingerprint after atom mapping [212]. Reaction language models read the reaction SMILES directly [213, 214]. Any molecular representation can also be applied to the reactants, to the product, or to their difference. A one-hot encoding of which aryl halide, ligand, base and additive was used carries no chemistry. It is kept because on plate data such identity features can predict yield about as well as chemical descriptors [215].

Antibodies. Antibody representations (Table 5.11) start from the heavy- and light-chain amino-acid sequences. Besides one-hot and BLOSUM62 encodings [216], most are frozen language models, either trained on all proteins or on antibody repertoires only. Most models embed each chain separately and join the two vectors, so their width doubles on datasets with both chains. Several representations read one model in different ways, for instance averaging over earlier layers or over the CDR residues only, with CDRs found by IMGT numbering [217] using ANARCI [218]. Datasets of point variants of one parent antibody also use features that describe each mutation.

Nucleotide sequences. A k-mer frequency vector counts how often each DNA word of length k occurs in a sequence, divided by the total number of words [219]; the canonical variant merges each word with its reverse complement [220]. DNA language models are networks pretrained on genomes to predict masked or next tokens [201, 221, 222]. Table 5.12 lists both kinds, along with the small sequence

summaries used as augmentation.

Cancer patients. For the two TCGA datasets (Table 5.13), each representation is a different measurement of the same patient's tumor, not a different encoding of one input. The measurements are RNA-seq gene expression (read through foundation models or summarized per chromosome arm), DNA methylation on the HM450 array, gene-level copy number, microRNA abundance, protein levels from reverse-phase protein arrays, and embeddings of the diagnostic whole-slide image. Data were obtained from TCGA through UCSC Xena [204, 223]. The most-variable selections, the z-scoring and the principal components for methylation, copy number, microRNA and protein were fitted once on all patients in a cohort, held-out patients included. No labels are used in these steps. A patient with no measurement for one data type gets an all-zero row for that representation.

We list all of these representations alongside their associated datasets and prediction tasks below:

Table 5.9: Representations used for small molecules. In the Role column, B marks a base representation, A an augmentation and B/A a representation that is a base representation on some datasets and an augmentation on others.

Representation	What it encodes	Features	Source	Role
ECFP4	hashed circular substructures, radius 2	2048	[4]	B
Atom pairs	hashed atom-type pairs with bond distance	2048	[224]	B
Topological torsions	hashed paths of four bonded atoms	2048	[225]	B/A
Avalon	hashed paths, rings and atom environments	2048	[226]	B
MACCS keys	166 predefined substructure keys	167	[227]	B/A
WL graph kernel	Weisfeiler–Lehman label counts, 5 rounds, labels present in at least 5% of molecules	140–317	[228]	B
PDV	RDKit 2D physicochemical descriptors	200	[32]	B
Mordred	2D molecular descriptors	1613	[229]	B
GROVER	frozen pretrained graph transformer	4800	[230]	B
MHG-GNN	frozen pretrained graph encoder	1024	[8]	B
GraphMVP	frozen 2D graph encoder pretrained with 3D views	300	[231]	B
MoLFormer	frozen SMILES language model; public checkpoint trained on 10% of its corpus	768	[40]	B
SMI-TED	frozen SMILES encoder–decoder	768	[232]	B

Representation	What it encodes	Features	Source	Role
ChemBERT	frozen SMILES language model; community checkpoint trained on ChEMBL following the cited method	256	[233]	B
ChemBERTa-2	frozen SMILES language model	384	[28]	B
SELFormer	frozen SELFIES language model	768	[234]	B
Mol2vec	summed substructure word vectors	300	[25]	B
3D pharmacophore	counts of 7 pharmacophore types, and mean and spread of distances for the 21 type pairs, one conformer	49	[32]	A
USR	shape moments from 4 reference points, one conformer	12	[235]	A
USRCAT	USR per pharmacophore atom type, one conformer	60	[236]	A
Coulomb matrix	sorted eigenvalues of the Coulomb matrix, one conformer	50	[237]	A

Table 5.10: Representations used for chemical reactions. Role is read as in Table 5.9, and the suffix in parentheses states whether a molecular representation is computed on the reactants, on the product or as their difference.

Representation	What it encodes	Features	Source	Role
CGR fingerprint	circular fingerprint, radius 2, hashed from the condensed graph of reaction	2048	[211]	B
rxnfp	frozen reaction language model	256	[213]	B
ReactionT5	frozen reaction language model	768	[214]	B
Component one-hot	which aryl halide, ligand, base and additive	45	[195]	B
GROVER (reactants)	GROVER averaged over reactants	4800	[230]	B
Mordred (reactants)	Mordred averaged over reactants	1613	[229]	B
Mol2vec (reactants)	Mol2vec averaged over reactants	300	[25]	B
ECFP4 (reactants)	ECFP4, bitwise OR over reactants	2048	[4]	B
Avalon (difference)	product minus reactant Avalon bits	2048	[226]	B
MHG-GNN (difference)	product minus reactant embedding	1024	[8]	B
MACCS (difference)	product minus reactant MACCS keys	167	[227]	B
Mol2vec (difference)	product minus reactant Mol2vec	300	[25]	B
MACCS (reactants)	MACCS keys, bitwise OR over reactants	167	[227]	A
MACCS (product)	MACCS keys of the product	167	[227]	A
3D pharmacophore (reactants)	3D pharmacophore averaged over reactants	49	[32]	A
3D pharmacophore (product)	3D pharmacophore of the product	49	[32]	A

Table 5.11: Representations used for antibodies. Feature counts marked /chain double on the datasets with heavy and light chains, Koenig expression and trastuzumab–HER2 binding. A dash in the Source column marks a standard encoding or one built for this thesis, and Role is read as in Table 5.9.

Representation	What it encodes	Features	Source	Role
One-hot	one-hot residues, both chains padded to fixed length	5600	–	B
BLOSUM62	BLOSUM62 row per residue, both chains padded	5600	[216]	B
Amino-acid composition	fraction of each amino acid, per chain	40	–	A
ESM-2	protein LM, last layer, mean over residues	1280/chain	[238]	B
ESM-2 (joined chains)	ESM-2 on heavy and light chain joined into one string	1280	[238]	B
ESM-2 early layers	ESM-2, mean over layers 0–16	1280/chain	[238]	B
ESM-2 middle layers	ESM-2, mean over layers 20–33	1280/chain	[238]	B
ESM-2 CDR	ESM-2, mean over CDR residues only	1280/chain	[238]	B
ESM-2 per CDR	ESM-2, CDR1/2/3 pooled separately then joined	3840/chain	[238]	B
ESM-2 variable positions	ESM-2, mean over the 20 highest-entropy positions	1280/chain	[238]	B
ESM-2 shift from parent	ESM-2 variant minus parent embedding	1280/chain	[238]	B
ESM-2 log-likelihood ratio	masked mutant vs parent log-probability: sum, mean, min	3	[239]	A
ESM3	protein LM	1536/chain	[240]	B
ESM3 CDR	ESM3, mean over CDR residues	1536/chain	[240]	B
ESM C	protein LM	1152/chain	[241]	B
ProtT5	protein LM	1024/chain	[242]	B
ProtT5 CDR	ProtT5, mean over CDR residues	1024/chain	[242]	B
SaProt	structure-aware protein LM	1280/chain	[243]	B
AbLang-2	paired antibody LM, both chains read together	480	[244]	B
IgT5	paired antibody LM, both chains read together	1024	[245]	B
AntiBERTy	antibody LM	512/chain	[246]	B
AntiBERTy CDR	AntiBERTy, mean over CDR residues	512/chain	[246]	B
Ab-RoBERTa	antibody LM	768/chain	[247]	B
Ab-RoBERTa CDR	Ab-RoBERTa, mean over CDR residues	768/chain	[247]	B
Mutation features	wild-type and mutant residue, substitution score, changes in hydrophobicity, weight and charge, position; averaged over mutations	48	–	B/A

Table 5.12: Representations used for nucleotide sequences. A dash in the Source column marks a simple summary built for this thesis, and Role is read as in Table 5.9.

Representation	What it encodes	Features	Source	Role
4-mer counts	normalized counts of every 4-letter word	256	[219]	B
5-mer counts	normalized counts of every 5-letter word	1024	[219]	B
6-mer counts	normalized counts of every 6-letter word	4096	[219]	B
Canonical TNF	4-mer counts merged with reverse complements	136	[220]	B
DNABERT-2	DNA LM	768	[221]	B
Nucleotide Transformer v2 250M	DNA LM	768	[201]	B
Nucleotide Transformer v2 500M	DNA LM	1024	[201]	B
HyenaDNA	long-context DNA LM	256	[222]	B
GROVER (DNA)	DNA LM (not the molecular GROVER)	768	[248]	B
Mistral-DNA	mixture-of-experts DNA LM	768	[249]	B
GC content	GC fraction, GC and AT skew, mean and spread of GC over 100-base windows	5	–	A
Dinucleotide frequency	frequency of each 2-letter word	16	[219]	A
Codon usage	relative usage of each of the 61 sense codons, read in frame from the first base	61	[250]	A
Sequence complexity	entropy of 1-, 2- and 3-letter words, linguistic complexity, fraction of alphabet used	5	–	A
DNA shape	minor groove width, propeller twist, helix twist and roll from a dinucleotide lookup table; mean and spread	8	–	A

Table 5.13: Representations used for cancer patients from TCGA. A dash in the Source column marks a representation computed for this thesis from the patient's own data, and Role is read as in Table 5.9.

Representation	What it encodes	Features	Source	Role
Geneformer (2,048 genes)	single-cell foundation model (V2-316M) on the 2,048 top-ranked genes, [CLS] vector	1152	[251, 252]	B
Geneformer (4,096 genes)	the same model on the 4,096 top-ranked genes, [CLS] vector	1152	[251, 252]	B
BulkRNABert	bulk RNA-seq foundation model	256	[253]	B
Top-variance methylation probes	beta values of the 5,000 most variable CpG probes	5000	[204]	B
Methylation PCA	50 principal components of the standardized CpG probes above median variance	50	[204]	B

Representation	What it encodes	Features	Source	Role
Copy number PCA	50 principal components of gene-level copy number	50	[254]	B
miRNA	200 most variable miRNAs, z-scored	200	[204]	B
Protein (RPPA)	reverse-phase protein array, z-scored	258	[255]	B
Chromosome-arm expression	mean expression of the genes on each chromosome arm	42	–	B/A
TITAN	whole-slide image foundation model	768	[256]	B
Prov-GigaPath	whole-slide image foundation model	512	[257]	B
CTransPath	histopathology image encoder	512	[258]	B
Phikon	histopathology image encoder	512	[259]	B
H-optimus-0	histopathology image encoder	512	[260]	B
HRD score	homologous recombination deficiency: LOH, TAI, LST and their sum	4	[261]	A
Tumor mutation burden	non-silent, missense and truncating mutation counts, log count, rate per megabase	5	[262]	A
Tumor purity	purity, ploidy, genome doublings, subclonal genome fraction	4	[263]	A
Expression statistics	summaries of the patient's expression distribution	15	–	A
Immune signatures	10 CIBERSORT cell fractions and Th1 and Th2 signature scores	12	[264]	A
CIBERSORT	immune cell fractions deconvolved from bulk expression	21	[264, 265]	A
Stemness index	stemness indices from expression and from epigenetically regulated expression	2	[266]	A
Hypermethylation summary	fraction of all methylation probes with beta above 0.3, and mean beta; a genome-wide stand-in for CIMP status [267]	2	–	A

Table 5.14: Base and augmentation representations of each dataset, grouped by domain. The hybrid search selects features from the base representations, and augmentation features are added after the hybrid is built. A dagger marks datasets searched at the fast tier.

Dataset	Task	Base representations	Augmentation
Small molecules			
Caco-2 efflux [109]	efflux ratio (log), regression	WL graph kernel, MHG-GNN, ECFP4, PDV, GROVER, Avalon, Topological torsions, SMI-TED, Atom pairs, MoLFormer	MACCS keys, 3D pharmacophore, USRCAT, Coulomb matrix, USR

Dataset	Task	Base representations	Augmentation
LogD† [109]	LogD, regression	MHG-GNN, Mordred, GROVER, ECFP4, SMI-TED, MoLFormer, Avalon, Topological torsions, ChemBERT, GraphMVP	MACCS keys, 3D pharmacophore, USRCAT
hERG K_i [110]	hERG inhibition, pK_i, regression	MHG-GNN, Mordred, WL graph kernel, Avalon, GROVER, PDV, ECFP4, Atom pairs, ChemBERT, SMI-TED	MACCS keys, 3D pharmacophore, Coulomb matrix, USR, USRCAT
MDR1–MDCK efflux [192]	efflux ratio (log), regression	Mordred, PDV, MHG-GNN, Avalon, GROVER, SMI-TED, Atom pairs, WL graph kernel, MACCS keys, ECFP4	3D pharmacophore, Coulomb matrix
Rat liver microsome clearance [192]	intrinsic clearance (log), regression	PDV, MHG-GNN, SMI-TED, Mordred, MoLFormer, GROVER, WL graph kernel, Atom pairs, Avalon, ECFP4	3D pharmacophore, MACCS keys, USRCAT, Coulomb matrix, USR
QM9† [107]	HOMO–LUMO gap, regression	Mordred, Avalon, SMI-TED, ECFP4, SELFormer, ChemBERTa-2, Mol2vec, ChemBERT, GraphMVP, MoLFormer	Topological torsions, 3D pharmacophore, USRCAT, Coulomb matrix
Chemical reactions			
Buchwald–Hartwig yield [195]	yield (%), regression	CGR fingerprint, GROVER (reactants), Avalon (difference), Mordred (reactants), MHG-GNN (difference), ReactionT5, Component one-hot, Mol2vec (reactants), rxnfp, MACCS (difference)	MACCS (product), MACCS (reactants), 3D pharmacophore (product), 3D pharmacophore (reactants)
Buchwald–Hartwig plates [195]	yield (%), regression	ECFP4 (reactants), GROVER (reactants), MHG-GNN (difference), CGR fingerprint, Component one-hot, Avalon (difference), ReactionT5, Mol2vec (difference), Mordred (reactants), rxnfp	MACCS (product), MACCS (reactants), 3D pharmacophore (product), 3D pharmacophore (reactants)
Antibodies			
Koenig expression† [196]	expression enrichment of variants, regression	ProtT5, ESM3 CDR, SaProt, IgT5, ESM-2 early layers, ESM C, AbLang-2, ESM-2 (joined chains), ESM-2, ESM-2 middle layers	Mutation features, ESM-2 log-likelihood ratio, Amino-acid composition
Nanobody thermostability [198]	melting temperature, regression	SaProt, ESM3, ESM C, ESM-2 variable positions, ESM-2 CDR, ESM-2 early layers, ESM-2 shift from parent, ProtT5 CDR, IgT5, One-hot	Mutation features

Dataset	Task	Base representations	Augmentation
Humanness [199]	human vs non-human, binary	IgT5, AntiBERTy CDR, Ab-RoBERTa, AbLang-2, BLOSUM62, ESM C, ESM-2 per CDR, ESM-2 early layers, ProtT5, ESM-2 middle layers	none
Trastuzumab–HER2 binding† [200]	binder vs non-binder, binary	BLOSUM62, ESM-2 per CDR, ESM-2 middle layers, ProtT5, ESM3, ESM C, Ab-RoBERTa CDR, AntiBERTy, Mutation features, AbLang-2	none
Nucleotide sequences			
H3K4me3† [201]	histone mark present vs absent, binary	GROVER (DNA), Mistral-DNA, Canonical TNF, Nucleotide Transformer v2 250M, Nucleotide Transformer v2 500M, HyenaDNA, 6-mer counts, DNABERT-2	Codon usage, Sequence complexity, Dinucleotide frequency, DNA shape, GC content
TATA promoter† [201]	promoter vs non-promoter, binary	Mistral-DNA, GROVER (DNA), HyenaDNA, 6-mer counts, 5-mer counts, Canonical TNF, 4-mer counts, Nucleotide Transformer v2 500M, Nucleotide Transformer v2 250M	Codon usage, Sequence complexity, Dinucleotide frequency, DNA shape, GC content
Cancer patients (TCGA)			
BRCA PAM50 subtype [205]	breast cancer subtype, 4 classes	Geneformer (4,096 genes), BulkRNABert, Protein (RPPA), Copy number PCA, TITAN, H-optimus-0, Prov-GigaPath, CTransPath, Phikon, miRNA	HRD score, Tumor mutation burden, Tumor purity, Expression statistics, Chromosome-arm expression, Immune signatures
LUAD vs LUSC [202]	lung cancer type, binary	Geneformer (2,048 genes), Top-variance methylation probes, BulkRNABert, TITAN, Methylation PCA, Chromosome-arm expression, Copy number PCA, miRNA, CTransPath, Protein (RPPA)	Immune signatures, Expression statistics, Stemness index, CIBERSORT, Hypermethylation summary, Tumor mutation burden, HRD score, Tumor purity

5.3.3 Models and Metrics

The downstream models range from methods that degrade in high dimensions (SVMs and Gaussian processes) to tree ensembles that tolerate irrelevant features.

The five downstream models are random forest, XGBoost, LightGBM, an SVM

and a DNN. Several are used because the best set of features depends on the model, and a hybrid that helped only one model family would reflect that model rather than a better input. Tree ensembles are the baseline to beat, because a hybrid is a tabular feature matrix and tree-based models still lead on medium-sized tabular data of about 10,000 samples [268]. A Gaussian process [269] is added on datasets under 1,000 molecules. Of the three such datasets, it runs on humanness only, because its run time under augmentation excluded PAM50 subtype and nanobody thermostability, and a dataset that drops it under any setting drops it under all.

By default, every model uses the same settings on every dataset, so that score differences come from the input. An optional per-fold random search tries 20 draws of a model's settings on 3 group-aware inner folds of the outer training fold; the DNN relies on early stopping instead. **[RESULT NEEDED: whether tuning pays on base representations]**. The Gaussian process is skipped above 1,000 molecules, because it factorizes the full kernel matrix, and the SVM above 10,000 molecules (5,000 under augmentation beyond the budget), because a fit no longer finishes in useful time. The caps count the molecules the search is given, so on LogD and TATA promoter the SVM is kept inside outer folds but dropped for the search on the whole dataset under augmentation beyond the budget. Other models left out on named datasets are listed in Additional file 13, and all model settings are in Table S.12.

The kNN model, which is not a default downstream model, takes $k = 5$ neighbors by default, lowered to the square root of the number of training molecules when a fold has fewer than 25. It is used to compare importance-weighted with unweighted distances at 5, 10 and 20 neighbors.

Each dataset is scored in the metric its field normally uses, so that it can be compared with published work (Table 5.8; definitions in Chapter 2, "Evaluation methodology"). Regression on small molecules, QM9 and reaction yields uses RMSE, in the units of the label. Antibody regression uses Spearman ρ, because these assays rank candidates and their scales differ between laboratories. Binary classification uses AUROC, which does not depend on a decision threshold. The two nucleotide datasets are the exception and use MCC, which is high only when all four entries of the confusion matrix are good [270]. The four PAM50 subtypes, of 67 to 421 tumors each, are scored with support-weighted F1. Metrics are never averaged across datasets.

Each hybrid is paired by fold with the best single representation, and each com-

parison is reported as its per-fold differences on outer test folds and their spread across the 5 folds. We use no significance tests and no confidence intervals, because 5 fold scores are correlated through shared training molecules. No unbiased estimator of the variance of k-fold cross-validation exists, and estimates that ignore the overlap underestimate it [271].

A full search takes **[RESULT NEEDED: hours per dataset]** on **[RESULT NEEDED: hardware]**.

5.3.4 Comparators, Controls and Ablations

The hybrid is the best of many configurations, so its gain may be no more than the reward for a large search. Each experiment below asks one question about this or a related issue.

Several experiments use leave-one-fold-out reselection. For held-out fold f, we pick the hybrid configuration with the best mean score on the other folds,

$$c^*_{-f} = \arg\max_c \frac{1}{F-1} \sum_{g \neq f} s_{c,g},$$

where $s_{c,g}$ is the score of configuration c on fold g and F is the number of folds (arg min for RMSE). The best single representation, k^*_{-f}, is picked in the same way, and we report

$$\Delta_f = s_{c^*_{-f},f} - s_{k^*_{-f},f},$$

signed so that $\Delta_f > 0$ means the hybrid is better. Importance and feature selection are always recomputed on each fold's training molecules. Some experiments, however, choose the configuration by a search over all molecules and redo only the feature selection per fold, so their scores are not fully held out, and we say so where this applies.

The first experiment uses nested cross-validation to ask whether the search overfits as the budget grows. At every budget B, the winning configuration is rebuilt on the outer training molecules and scored on the outer held-out fold. The selection bias is the inner-search score minus the held-out score, positive when the search promised more than the held-out fold gave. The budgets are the standard list from 10 to 2,000 features, plus 4,000 and 8,000, plus 0.5, 0.75, 1, 1.25, 1.5 and 2 times the smallest outer training set in molecules, which place a budget where the number of features equals the number of molecules. For the five downstream models, the bud-

get does not set the number of fitted parameters, so their curves show whether the search's choice of budget overfits. We add a nearly unpenalized linear model (ridge with penalty 10^{-6}, or logistic regression with $C = 10^6$ for classification), whose parameter count equals the budget. Per model, we report the budget picked in each outer fold and the budget with the best held-out score. A pick far above the latter indicates overfitting. **[RESULT NEEDED: held-out score and selection bias against budget]**.

The size of a search is the number of configurations it scores: 480 per pair of downstream model and internal evaluator at the full tier and 48 at the fast tier (Additional file 14). We ask whether the hybrid's lead over each comparator grows with this number. The comparators receive no search, so part of any growing gap reflects the hybrid taking the best of more draws, and we report numbers after leave-one-fold-out reselection, which charges the hybrid for this. This comparison needs one dataset searched at two sizes, whereas the full runs search each dataset at one tier only. **[RESULT NEEDED: hybrid minus each comparator at each search size]**.

The ensemble fits one model per representation and pools their predictions (late fusion), whereas the hybrid fits one model on one fused feature vector (early fusion) [148]. Every choice for the ensemble is made on an inner 3-fold group split of the outer training molecules. The ensemble uses the same model on every representation or each representation's best model, and its members are the top 3 representations or every representation with at least 50 features. Predictions (probabilities, for classification) are pooled in one of seven ways, from the mean and median to trimmed, outlier-removed and score-weighted means. Three controls are added. They are the best single representation, all representations joined and reduced by PCA to the hybrid's width, and the best pair of model and representation read off the held-out fold, an upper bound rather than a fair comparator. We also report the number of representations and models needed at prediction time, and the fit and prediction time. The hybrid saves cost only if it draws on fewer representations than the ensemble. **[RESULT NEEDED: held-out score and cost per method]**.

The next experiment asks whether the gain comes from drawing on several representations or only from having fewer features. On regression datasets, every method receives B features from all base representations, fitted on the fold's training molecules. The methods are the hybrid (its stored configuration at budget B, with features re-selected per fold), PCA on the joined, z-scored representations, mRMR

[165], recursive feature elimination [168] and B random features. mRMR stops at 200 features for run time. One downstream model, a random forest, scores every method, so only the features differ. This comparison is not fully held out, because the hybrid's configuration is chosen using all molecules. **[RESULT NEEDED: held-out score against B per method]**.

This experiment asks whether the internal evaluator must match the downstream model, using nested cross-validation with one search per outer fold for each evaluator setting. The matched evaluator belongs to the downstream model's family. A near-family evaluator pairs tree models with each other, the SVM with a Gaussian process, and the DNN with a narrower DNN. A far-family evaluator pairs tree models with an SVM, the SVM with LightGBM, and the DNN with k-nearest neighbors. A fourth setting scores the DNN with a wider DNN, and a fifth keeps the best of random forest, LightGBM and XGBoost. We report each setting minus the matched evaluator, paired per outer fold, at the same number of features. The answer determines whether one cheap evaluator is enough or whether the search must be rerun for each model. **[RESULT NEEDED: each setting minus matched, per downstream model]**.

Selection on each of the M folds gives a binary vector $\mathbf{z}_m \in \{0,1\}^d$, where d is the number of features in all base representations joined, not the size of the union of the selected features. We measure the stability of the selection with the index of Nogueira et al. [272],

$$\hat{\Phi} = 1 - \frac{\frac{1}{d}\sum_{f=1}^{d} s_f^2}{\frac{\bar{k}}{d}\left(1 - \frac{\bar{k}}{d}\right)}, \qquad s_f^2 = \frac{M}{M-1}\hat{p}_f(1-\hat{p}_f).$$

Here $\hat{p}_f$ is the fraction of folds that selected feature f, and $\bar{k}$ is the mean number of features selected. $\hat{\Phi} = 1$ means the same features on every fold, and $\hat{\Phi} \approx 0$ is what random selection gives. Unstable feature sets would weaken any chemical reading of the blueprints. **[RESULT NEEDED: $\hat{\Phi}$ per dataset and model]**.

The learning curves draw subsamples of 10, 25, 50, 75 and 100% of the molecules as whole groups, never fewer than 50 molecules, and above 5,000 molecules use 250, 500, 1,000 and 2,000 molecules instead. Each size below 100% is drawn 3 times. A full search runs on each subsample, dropping budgets above the number of training molecules, so part of any rise in gain with size may come from searching more configurations. The gain is computed after leave-one-fold-out reselection. **[RESULT**

NEEDED: gain per dataset, model and size].

This experiment asks whether importance also helps during training. It uses the hybrid at 100 features with random forest, LightGBM and XGBoost, the three models that accept per-molecule weights, with importance computed on the training molecules by the method the search picked. The five conditions are:

1. No weighting: the hybrid as selected.

2. Feature weighting: standardized features multiplied by the square root of their importance, with the weights normalized to average 1.

3. Molecule weighting: molecules weighted by how well their label agrees with their 10 nearest neighbors in the rescaled space.

4. Combined weighting: feature and molecule weighting together.

5. Above-median selection: only the features strictly above the median importance.

The weight of molecule i is $1-|y_i-\bar{y}_{N(i)}|/(\max y-\min y)$, clipped to lie between 0.1 and 1, where $\bar{y}_{N(i)}$ is the mean label of its 10 nearest neighbors by Manhattan distance. The configuration comes from a search over all molecules, so the comparison is not fully held out. Tree splits depend only on the order of values [273], so for these models feature weighting differs from no weighting only in that zero-importance features drop out. The rescaling does change which molecules are nearest neighbors, and so changes molecule weighting. Molecule weighting down-weights rather than removes a suspect label, unlike neighbor-based filtering (Section 4.4.2). We found no QSAR study that does this, and learned per-sample weights for noisy labels come from general machine learning [274]. **[RESULT NEEDED: one result line]**.

This experiment asks whether a few extra features help one strong single representation, and whether they help it more than the hybrid of several representations. Two kinds of decoy are planted among the candidates: pure Gaussian noise as wide as the real candidates combined, and near-copies of 64 base features with noise at 5% of each feature's standard deviation. A working rule takes no decoys, and we count how many are taken on each fold. **[RESULT NEEDED: one result line, with decoys taken per fold]**.

This experiment asks whether hybrids are more robust to label noise than their base representations, measured by $\mathrm{AUC}_{\mathrm{norm}}$ (Equation 2.7). Unlike Chapter 3, the retention curve uses each dataset's own metric, inverted for an error metric so that

1 still means no loss. The seven noise levels are 0, 0.2, 0.3, 0.5, 0.75, 1.0 and 1.5 times the spread of the fold's clean training labels, and the fraction of labels censored is 0, 0.10, 0.20, 0.25, 0.30 or 0.40. With a random forest, the hybrid at each budget is compared with each single representation and with the best one, re-picked per fold. The hybrid is searched again at every noise type and level on the noisy training labels, and held-out labels stay clean. The search uses the first fold to drop configurations, so that fold is left out of both sides and four folds are reported.

The last experiment asks whether joining two whole representations scores above the better of the two or between them. For each dataset, its top three single representations and their three pairwise concatenations are each scored with the same model and folds. A concatenation that lands between its two parts supports the case for combining and filtering at once. **[RESULT NEEDED: score of each representation and each concatenation]**.

5.3.5 Transfer, Mutational Scans and Separate Sites

The first study asks whether search settings (importance method, allocation method, budget B and internal evaluator) found on a large dataset can be reused on a small one instead of a new search. The source is LogD (lipophilicity), with 5,028 molecules, and the target is hERG pK_i, with 1,412 molecules. Both searches use the representations listed for both datasets at matching width. Before the width check, the current lists share seven: Avalon, ECFP4, Mordred, GROVER, MHG-GNN, SMI-TED and ChemBERT **[RESULT NEEDED: the shared set in the full run]**. If settings transfer, the expensive search becomes a one-off cost. The closest prior work, Meta-QSAR [275], chose the learning method and representation for a new QSAR dataset from results on over 2,700 others. It beat random forest on a fingerprint by up to 13% on average and, like this study, reused decisions rather than selected features. We have not found a paper that applies feature indices selected on one QSAR dataset unchanged to another, although our literature search was not complete.

Four comparisons run inside every held-out fold of the target. They are the best single representation, a hybrid searched on the target's training molecules as the upper reference, the source settings with features re-selected on the target, and the source's selected features applied unchanged. The last two ask separately whether

the search decisions and the chosen features transfer. We use five scaffold-group folds, RMSE in pK_i units and the five downstream models. **[RESULT NEEDED: hybrid minus best single representation per comparison and model]**.

A deep mutational scan measures a property, here expression or binding, for many single- or few-point mutants of one antibody. We use four such scans from FLAb [197]: the expression and binding scans of the anti-VEGF antibody G6 [196] and of the anti-fluorescein antibody 4-4-20. The candidates are the representations of "Representations by domain" plus two representations of the change from the wild type. The first sums, over the mutations, the changes in five amino-acid properties (hydrophobicity, charge, isoelectric point, molecular weight and aromaticity), each z-scored over the 20 amino acids; the second places the same five features at the mutated position. The binding scan of 4-4-20 is binarized at p$K_d > 5.0$ and scored by AUROC, and the others by Spearman correlation. We report the hybrid minus the best single representation per scan. This difference says nothing about the wild-type-difference representations, whose number of features in the hybrid is stored per fold but not reported here. **[RESULT NEEDED: features taken from the two wild-type-difference representations, per scan and fold]**. **[RESULT NEEDED: hybrid minus best single representation per scan and model]**.

Federated learning trains one model across data held by several parties without moving the data [276], and it has been run across pharmaceutical companies [277], whose structures and assay results are commercial assets that cannot be pooled [18, 19]. Our collaborator, Lhasa Limited, uses the teacher-student process of Chapter 2, "Noise in molecular data" [137]. We use their code for the fingerprints, site models and site clustering, and re-implement from it the step that combines the site models' predictions.

The site models' predictions are combined in one of two ways. Under distance weighting, each site's vote on a molecule is weighted by how close the molecule is to that site's training molecules, so a site counts most on chemistry it has seen. Distances are Manhattan by default, whereas the collaborator's code uses Euclidean. Under score weighting, each site has one weight, its own cross-validated MCC. Only predictions on shared molecules cross a site boundary. The published process then trains a student model on the most confident combined labels. In the hERG comparison below, no student is trained, and the combined predictions score the held-out molecules directly.

Our addition is that feature selection, as well as model fitting, takes place at

the sites. Each site runs the hybrid search on its own molecules and reports only feature indices, importance values and one score. A coordinator splits the total budget B across sites in proportion to their scores, with at least one feature per site. Only this proportional split is run, so we cannot say what the site scores buy. A feature already taken by another site is replaced by the site's next-ranked feature, so all features in the hybrid are distinct. Indices and importances are smaller, and harder to invert, than model weights or gradients. In related work, each client runs mutual-information feature selection on its own data and a server aggregates the results [278].

hERG channel blockade is a cardiotoxicity liability screened early in drug discovery, with public data from many laboratories [279]. Three real sources act as sites: the public ChEMBL hERG K_i set used elsewhere in this thesis, and the collaborator's training set (7,772 molecules) and test set (4,383 molecules). The two collaborator sets were labeled by different curation rules, so the sites differ in curation as well as in chemistry, whereas the process is usually simulated by clustering one dataset. Molecules held by several sources keep only the public copy. A public molecule is a blocker if $\mathrm{p}K_i \geq 6$. We use five scaffold-group folds over the pooled molecules. The search is scored by AUROC, and the reported number is MCC, which does not reward predicting the majority class.

The comparison crosses two choices: molecules pooled or kept at their sites, and all features or a selected set. Pooled molecules with all features are the upper reference, not possible in practice. Pooled molecules with selected features measure what the selection is worth on its own. Site-held molecules with all features are federated training as usually done, and site-held molecules with selected features are the only setting in which nothing but indices, importances, one score and predictions crosses between sites. Selected minus all features is what selection buys, and pooled minus site-held is what pooling buys. Four context comparisons are added: the single best source picked on training molecules, both ways of weighting the site models, the same search on one fingerprint alone, and one site model per representation. The budget B is swept over 50, 100, 250, 500 and 1,000 features. The all-features settings fit one 2,048-bit Morgan fingerprint of radius 2 with pharmacophoric atom types, whereas the selected settings draw on the hERG K_i candidate representations. Selected minus all features therefore differs in two ways at once, width and source. The closest control for source is the same search run on ECFP4 only.

Four further separate-site experiments are run. In the first, the sites carry differ-

ent amounts of label noise. Selection at the sites is compared, at several noise levels, with the ensembles of site models and with one model on the pooled molecules, in one version after each site removes the molecules it flags as noisy. In the second, every site holds the same molecules as a different representation, and fusing features is compared with pooling predictions. In the third, the sites hold different families of representations (substructure, physicochemical and pretrained). In the fourth, the sites are three antibody sources (a binding-energy-change set, a deep mutational scan of binding and a thermostability set). It skips the ensemble comparisons, because each site's features are reduced separately and the labels are in incompatible units. **[RESULT NEEDED: outcome of each of the four]**.

The separate-site work has four limitations. One source is proprietary, so the comparison cannot be reproduced from public data alone (Additional file S.6). It depends on the collaborator's implementation. The published process trains a student on a large separate public set; where a student is trained here, the training molecules take that role as a compromise, and held-out molecules are excluded from it. Finally, the sites differ in size and in how their labels were derived, so a site's score reflects its curation as well as its chemistry.

5.4 Results: Comparison with Single Representations and Ensembles

5.5 Results: Sources of the Gain

5.6 Results: Blueprints on Specific Datasets

5.7 Results: Transfer across Datasets and Antibody Mutational Scans

5.8 Results: Data Held at Separate Sites

5.9 Discussion

Ambroise and McLachlan [176] showed the extreme case of overfitting through fea-

ture selection. An SVM reached near-zero cross-validated error on random labels when genes were selected on the full dataset first. With selection nested inside each fold, the median error was 11 out of 40, close to chance. The bias from preselection can exceed that from hyperparameter tuning [177]. Cawley and Talbot [179] argued that feature selection is itself a form of model selection whose complexity cost must be accounted for. The hybrid search computes all importance scores within each fold, on training data only, but searching over many feature subsets still adds complexity even when each individual selection is unbiased [280]. The budget acts as a hard capacity constraint: selecting 100 features from 5,000 constrains the possible selections far more tightly than L_1 regularization, because the feasible set grows polynomially in the budget while a regularized model's parameter space is unbounded [281]. The selected features are also frozen embeddings rather than fitted parameters, and we consider choosing among them less expressive than learning new features. In support, frozen embeddings with a linear probe can outperform full fine-tuning by up to 10% in out-of-distribution accuracy [282]. Finally, overfit selection would give feature sets that differ widely between folds, which the Nogueira index measures ("Comparators, controls and ablations").

5.10 Limitations

The adaptive hybrid representation has several limitations. Below about 200 samples, the search is not expected to beat the best single source reliably **[RESULT NEEDED: sample size below which the hybrid stops beating the best single representation]**. We suspect this is because the internal validation set in each fold shrinks to about 20 samples, too few for the evaluator to tell real improvements from noise, and tuning cannot fix this. Frozen embeddings also have a ceiling, since no signal can be added beyond what the inputs contain. Integrating 20 to 50 foundation models, each with its own dependencies and GPU requirements, is difficult engineering, and the burden scales with the number of models. A downstream model supplied by a user from outside the fixed list of model families gets no matched evaluator, and no warning. Finally, on endpoints with high experimental noise such as public K_i data (Chapter 1), even a well-constructed hybrid may not improve on simpler baselines, because the noise sets an accuracy ceiling. The budget constraint plausibly guards against overshoot, but it cannot push past this noise floor.

5.11 Conclusions

Chapter 6

Hybrid Representations for Similarity and Label Noise

6.1 Introduction

Chapter 4 showed where the similarity assumption fails on standard representations. Any method that trusts a molecule's neighbors inherits this error, including noise detection that compares each label with its neighbors' labels. Chapter 5 introduced the adaptive hybrid representation.

The original idea behind the hybrid was to select the features relevant to the task first and then to compute distances on those features alone.

We test three methods that use neighbors or held-out predictions to handle noisy labels: nearest-neighbor editing [103], filtering by classifiers trained on other folds [143] and confident learning [52]. Each depends on the feature space it works in. We ask three research questions, each answered in one results section:

1. Is predicting each molecule's activity from its five nearest neighbors more accurate in a hybrid than in the single representations from which the hybrid is built?

2. Are noisy labels easier to find, or the size of their errors easier to estimate, in a hybrid?

3. Does a hybrid change how well noise mitigation, through label correction and cleaning, and noise-robust training perform?

This chapter is planned as a second publication on hybrids; its methods are final

and its results preliminary. **[RESULT NEEDED: one-sentence preview per research question]**

6.2 Methods

Datasets, scaffold-group-aware folds and metrics are those of Chapter 5. Each downstream model is compared with its own top two single representations, ranked by its own score, not with a fixed list or one chosen fingerprint.

In every experiment, the hybrid's features are rebuilt in each fold from that fold's training labels, whether noisy, clipped or uncleaned. The search settings (representations, budget and allocation method) come from a different search in each experiment. In the survey of twelve methods for learning with noisy labels, the search is repeated in each fold and at each noise level on the noisy labels, scored by the neural network the methods use, over budgets of 50, 100 and 250 features. In the censoring experiment, the settings come from the search on the clean, unclipped labels, and only the feature ranking and budget split in each fold see the clipped labels. Settings searched on clean labels give the hybrid information a real user would not have, an advantage that the survey avoids and the clipped-label experiment does not. When real labels are cleaned, the settings come from a search on those same labels.

We keep apart three tasks often treated as one: finding noisy labels, estimating the size of the noise and improving the model's held-out score. A method can succeed at the first and fail at the third. **[FIGURE NEEDED: flow from noisy data, to per-fold hybrid and single representations, to scores]**

6.2.1 Similarity in a Hybrid

We use hybrids for similarity in two ways. In an ordinary hybrid, the search of Chapter 5 selects features that help a downstream model, and distances are computed on those features. A similarity hybrid comes from the same search, but each candidate is ranked by how accurately each molecule's activity is predicted from its five nearest neighbors in the hybrid, and no downstream model is fitted. The two searches share importance methods, allocation methods, budgets and folds, and differ only in the score that ranks candidates. The neighbor score, computed on a validation set from the fold's training molecules, also replaces the internal evaluator

in splitting the budget. All of the dataset's representations are candidates, and the number of representations is not searched. Chapter 4 showed that the representation determines which molecules count as neighbors. The similarity hybrid tests whether choosing features for the quality of the neighbors addresses this problem more directly than choosing them for prediction. **[FIGURE NEEDED: D6.1, one search, two scores]**

The distance is the Manhattan distance of Chapter 4 on features z-scored as in Chapter 5. Each z-scored feature adds a term of similar size to this distance, so a representation's weight in the distance, set by the allocation method, is roughly its number of features divided by the budget. The neighbor score applies no importance weighting. **[RESULT NEEDED: confirm Manhattan distance and inverse-distance weighting]**

To rank whole candidates, the score is computed on each outer fold's held-out molecules, which also serve as each other's neighbors. Their scaffold groups are split into up to 5 parts, and each molecule's 5 nearest neighbors are drawn from the other parts, because a neighbor with the same scaffold is an easy match that would inflate the score. Each neighbor is weighted by the inverse of its distance:

$$\hat{y}_i = \frac{\sum_{j \in N_5(i)} y_j / d_{ij}}{\sum_{j \in N_5(i)} 1 / d_{ij}}. \tag{6.1}$$

A neighbor at distance 0 receives all of the weight. On regression data, the score is the Spearman correlation $\rho(\hat{\mathbf{y}}, \mathbf{y})$. On binary data, $\hat{y}_i$ is the distance-weighted fraction of neighbors in class 1, and the score is $2(\text{AUROC} - 0.5)$, where AUROC is the area under the receiver operating characteristic curve. Both scores put chance at 0, which the budget split needs because it divides by the score. The modelability index of Golbraikh et al. [118] (Chapter 4) is a related published measure, but it uses a single nearest neighbor and a hard class match, whereas our score retains the graded fraction of neighbors. The score thus measures how far the similar property principle (Chapter 2) holds in the hybrid.

A fold with fewer than 10 held-out molecules, fewer than 2 scaffold groups, or constant labels or predictions is left out of the mean rather than scored 0. The similarity hybrid is not searched on three datasets. AUROC needs two classes, which excludes the PAM50 breast cancer subtypes (4 classes) and trastuzumab HER2 multiclass (3 classes). The folds of trastuzumab HER2 binary follow edit-distance shells around the wild-type CDR-H3, so a held-out block can hold a single shell,

with no neighbor from another group. All other datasets are searched, including the antibody and other non-molecule datasets.

Feature importance weighting. Beyond simply using the feature importance scores for allocation, we investigate whether feature weighting by importance during distance computation improves noise detection. For a feature importance vector w, weighted distances are computed by scaling the standardized features: $x' = x \odot w$ under Manhattan distance ($x \odot \sqrt{w}$ under Euclidean), with negative importances set to 0 and w rescaled to average 1.

6.2.2 Noise Conditions

The noise conditions are defined in Section 2.5.1 (Figure 2.3). Censoring, Gaussian noise and real labels drive the three experiments on learning with noisy labels. Outlier and grouped-wider noise grade detection, and grouped-shifted noise grades estimation only, because it moves every training label. Noise is added only to each outer fold's training labels. Its size, the censoring limit and the scaffold families that grouped-wider noise widens come from that fold's clean training labels. The simulation of several sources per molecule with correlated errors is the exception. It sizes its noise from the label standard deviation of all molecules, held-out included, but its held-out labels stay clean.

The censoring levels, the fraction of training labels clipped, are 0, 0.10, 0.20, 0.25, 0.30 and 0.40. The survey and the detection results use 0, 0.10, 0.25 and 0.40, with 0 as the negative control. The clip involves no random draw, so each level is run once. The error is one-sided, a bias rather than a scatter, and its direction and location (the limit) are known. Censoring applies to regression only, because a class label has no value to clip.

Under Gaussian noise, the levels are 0, 0.2, 0.3, 0.5, 0.75, 1.0 and 1.5 times the standard deviation of the fold's clean training labels, with three noise draws per level. On classification data, the level is instead the fraction of training labels flipped to the wrong class: 0, 0.05, 0.10, 0.20, 0.30 or 0.40. Gaussian noise, which represents the random error of a repeated measurement and is also the standard assumption in studies of noise robustness, is used in the survey of twelve methods.

Outlier noise represents annotation and transcription errors [14, 98]. We apply it to 1%, 5% and 10% of training molecules, where Chapter 3 used 10% only. The

grouped conditions need a scaffold per molecule, so they run only on datasets with SMILES strings.

With real labels, nothing is injected and no list of wrong labels exists, so success can only be measured as a better held-out score after cleaning. That shows cleaning helped the model, not that it found errors; it may have removed difficult molecules instead. QM9 is a control: its labels come from a quantum-chemical calculation with no experimental error [107], so a cleaning method that helps on QM9 is not removing measurement error. Table 6.1 summarizes each condition. **[RESULT NEEDED: datasets per condition, for Table 6.1]**

6.2.3 Noise Mitigation Methods

Sample removal has limits when Gaussian noise affects every label, so we also use sample weighting and label smoothing, which act on a continuous estimate of noise size instead of a noisy-or-clean decision (Section 4.4.2).

Sample Removal Methods

Distance-based approaches rely on the similar property principle (Chapter 2). Presumably, noisy data would break this pattern, making it a method for detecting anomalous data. k-Nearest neighbor filtering examines the variance of a sample's neighborhood. For sample x_i with k nearest neighbors $\mathcal{N}_k(x_i)$, we compute the standard deviation of neighbor labels:

$$\sigma_i = \text{std}(\{y_j : x_j \in \mathcal{N}_k(x_i)\}) \tag{6.2}$$

Samples with σ_i exceeding the 90th percentile are marked as potentially noisy. We test $k \in \{5, 10, 20\}$.

Local Outlier Factor (LOF) identifies anomalous samples by detecting local density estimates between a sample and its neighbors [283]. These density estimates are based on the distance to its k nearest neighbors. If a point's density is lower than that of its neighbors, its LOF is high. We flag the 10% of training molecules with the highest LOF, using 20 neighbors.

We also tested the following noise removal methods: distance-weighted prediction filtering, neighborhood consensus filtering, AC detection, Mahalanobis outlier detection, cross-validation disagreement, bootstrap ensemble filtering, co-teaching

[140], and DivideMix [284].

Noise Magnitude Estimation

The central part of the noise mitigation work is estimating the size of the noise, which lets each label be handled in proportion to its error instead of being removed outright. Table 6.1 lists three families of estimators. Neighbor-based estimators compare a molecule's label with those of its k nearest training neighbors, model-based estimators use the disagreement between predictions made without the molecule, and probabilistic estimators use a model's predicted uncertainty.

Estimator	Family	Score of molecule i
Neighbor residual	Neighbors	$\lvert y_i - \bar{y}_{\mathcal{N}} \rvert / \sigma_y$
Local variance ratio	Neighbors	$\lvert y_i - \bar{y}_{\mathcal{N}} \rvert / \max(s_{\mathcal{N}}, c)$, with $c = \max(0.01\,\sigma_y, 10^{-3})$ so that a neighborhood sharing one label gives a finite score
AC-aware	Neighbors	$\frac{\lvert y_i - \bar{y}_{\mathcal{N}} \rvert}{\sigma_y} \cdot \frac{1}{1 + s_{\mathcal{N}}/\sigma_y}$: a large deviation among neighbors that agree suggests noise; among neighbors that disagree, an activity cliff [73]
Cross-validation residual	Model	Absolute residual of the out-of-fold prediction
Ensemble disagreement	Model	Standard deviation of several models' out-of-fold predictions, plus the absolute residual of their mean
Gaussian process	Probabilistic	Posterior variance, with a learned noise level [66]
Heteroscedastic network	Probabilistic	Predicted variance $\sigma^2(x)$, trained with the β-NLL loss of Equation 6.3 [285]
Conformal prediction	Probabilistic	Width of the split-conformal prediction interval [286]
Deep ensemble	Probabilistic	M networks: variance of their means (epistemic) plus mean of their variances (aleatoric) [69]
Bayesian random forest	Probabilistic	Standard deviation of the trees' predictions, plus half the absolute out-of-fold residual [287]
Quantile regression	Probabilistic	Width between the fitted quantiles
NGBoost	Probabilistic	Predicted standard deviation σ [58]

Table 6.1: Noise magnitude estimators by family. The last column gives the noise score of molecule i, where $\bar{y}_{\mathcal{N}}$ and $s_{\mathcal{N}}$ are the mean and standard deviation of the labels of its k nearest training neighbors, σ_y is the standard deviation of all training labels and M is the number of networks in the deep ensemble. A higher score marks a label as more likely to be noisy.

The heteroscedastic network predicts a mean $\mu(x)$ and a variance $\sigma^2(x)$ and is

trained with

$$\mathcal{L} = \frac{1}{2}\left(\frac{(y-\mu)^2}{\sigma^2} + \log\sigma^2\right)\cdot\sigma^{2\beta}, \tag{6.3}$$

where $\beta = 0.5$ prevents the variance from hiding the mean errors.

NGBoost in particular was selected due to its promising uncertainty values with respect to the correlation between noise and prediction error, as discussed in Section 3.7.

Label Smoothing and Sample Weighting

Besides modifying labels, we gave more weight to trustworthy samples during training. Both approaches use the noise score min-max normalized to $[0, 1]$, written $\tilde{s}_i$ (Table 6.2).

Rule	Changes	New label or weight of molecule i
Proportional	Label	$y_i + \alpha\,\tilde{s}_i\,(\text{target} - y_i)$
Threshold	Label	Only above the $100(1-\alpha)$th percentile of $\tilde{s}$: $(1-\alpha)\,y_i + \alpha\cdot\text{target}$, so α sets both how many labels move and how far
Soft threshold	Label	Correction strength $\alpha\cdot\text{sigmoid}\big(5(\tilde{s}_i - 0.5)\big)$
Confidence	Label	$(1-c_i)\,y_i + c_i\,\mu_i$, with $c_i = \alpha\sigma_i/(1+\alpha\sigma_i)$; for estimators that output σ, such as NGBoost
Inverse	Weight	$1/(1+\alpha\,\tilde{s}_i)$
Exponential	Weight	$\exp(-\alpha\,\tilde{s}_i)$
Soft	Weight	$1-\alpha\,\tilde{s}_i^2$, clipped to $[0.1, 1]$

Table 6.2: Label smoothing and sample weighting rules. $\tilde{s}_i$ is the noise score of molecule i scaled to $[0, 1]$, α sets the strength, the target is the mean label of the molecule's nearest neighbors, and μ_i and σ_i are the mean and standard deviation predicted by a probabilistic model. Weights are rescaled to a mean of 1.

Recovery Rate

For noise detection accuracy, we compute precision, recall, and F1 score, treating noise injection as ground truth.

Each downstream model m is trained on the clean, noisy and corrected training labels, giving held-out scores s^m_{clean}, s^m_{noisy} and s^m_{denoised} in the dataset's own metric, averaged over folds. The recovery rate is the share of the lost performance that correction recovers,

$$\text{Recovery}_m = 100 \times \frac{s^m_{\text{denoised}} - s^m_{\text{noisy}}}{s^m_{\text{clean}} - s^m_{\text{noisy}}}\ \%, \tag{6.4}$$

which reads the same whether higher or lower values of the metric are better. When a model's gap between clean and noisy score is no more than 2% of the clean score, the noise did too little damage to be recovered, and the recovery rate is left undefined. A recovery rate of 100% fully restores clean performance, and a negative value means the corrected labels scored worse than the noisy ones.

6.3 Results: Similarity

6.4 Results: Noise Detection and Estimation

6.5 Results: Noise Mitigation by Noise Type

6.5.1 Censoring: Correction of Clipped Labels

6.5.2 Gaussian Noise: Methods for Learning with Noisy Labels

6.5.3 Real Labels: Cleaning without Injected Noise

6.5.4 Comparison across Noise Types

6.6 Conclusions

Chapter 7

Discussion and Conclusions

Real assay labels are not noisy in the way most studies assume. Measurements pooled across laboratories share an offset per laboratory [80], and labels near an assay's detection limit are clipped to it [81]. Gaussian noise, the usual assumption, is the hardest case to correct, because every label is off by an unknown amount in an unknown direction, and cleaning can leave a model worse than no cleaning ("Comparison across noise types", Chapter 6).

7.1 Summary

Chapter 3 found that, on the QM9 HOMO–LUMO gap, the model and the representation explained comparable shares of the variance in R^2, but the model dominated robustness (AUC_{norm}; "Conclusions", Chapter 3). Changing the shape of the noise barely changed performance, although grouped-shifted and censored noise cost the most (censoring was run on five pairings of model and representation only). Mean predicted uncertainty rose with noise, but no model's per-molecule uncertainty identified the corrupted labels, and under censoring the models were most confident on the clipped labels.

Chapter 4 showed that on any one dataset many fingerprint features do not vary or do not relate to the property, yet a distance sums over all of them, and that some representations give different molecules identical feature vectors. In the exploratory study of learning with noisy labels, configurations that used a distance did worst, and judging a label by its neighbors usually made the model worse. A better representation would have few features, chosen for the task from several representations and traceable to their source.

Chapter 5 introduced the adaptive hybrid representation, which chooses features for one task from several representations under a budget in features, inside each fold. It beat the best single representation on **[RESULT NEEDED: datasets, score pairs]** and not on **[RESULT NEEDED: datasets, score pairs]**. Preferred search settings **[RESULT NEEDED: differed across datasets?]**. **[RESULT NEEDED: transfer.]** **[RESULT NEEDED: antibody.]** **[RESULT NEEDED: separate sites.]** Importance weighting **[RESULT NEEDED: helped?]**. Its limits are set out in the Limitations section of Chapter 5.

Chapter 6 tested whether a hybrid chosen for the task repairs nearest-neighbor prediction, noise detection and learning with noisy labels. Noise detection: **[RESULT NEEDED]**. Learning with noisy labels: **[RESULT NEEDED: and noise types where any LNL method helped]**.

Chapter 3 tells a reader which model to choose when labels are noisy, and Chapters 4 to 6 tell them which features to give that model. The contributions of this thesis are the NoiseInject package, the adaptive hybrid representation and its blueprints, and evidence that distance-based noise handling on standard representations usually made models worse. The resulting papers and software are listed in the Summary of Research Contributions.

7.2 Dataset Dependence of the Best Method

No value of any of the six settings of the hybrid search (Table 5.6) was best on every dataset **[RESULT NEEDED: confirm on the full runs]**, and which LNL method helps, if any, depends on the data **[RESULT NEEDED: confirm on the full runs]**. Noise mitigation, the LNL methods that change the labels themselves, is the harder part (Chapter 6). We therefore report no overall winner averaged across datasets.

The best choice depends on at least five factors: the type and amount of label noise, the number of molecules, whether any feature describes the property, and the downstream model. A small dataset leaves few molecules in a fold to score each setting on, so the chosen setting is less stable (size threshold: Limitations, Chapter 5). The search cannot add signal its inputs lack. The internal evaluator is searched per downstream model, since the best hybrid for a tree ensemble need not suit a neural network.

In Chapter 3, the most robust representation for a model changed between Gaus-

sian and grouped-shifted noise for five of the thirteen base models on QM9. In Chapter 5, the winning allocation method, number of base representations, importance method and budget were **[RESULT NEEDED: per dataset]**, and the winning internal evaluator **[RESULT NEEDED: per dataset and model]**. A fixed share of features per representation won on **[RESULT NEEDED: datasets]**, one pooled ranking on **[RESULT NEEDED: datasets]**. In Chapter 6, raising clipped labels **[RESULT NEEDED: helped?]**, and under Gaussian noise the number of the twelve LNL methods beating the plain network **[RESULT NEEDED: changed with noise amount and input?]**. The LNL methods that helped were **[RESULT NEEDED: per noise type]**.

The thesis therefore offers no fixed recipe or new architecture tuned to one problem, but a set of methods that might work and a procedure that chooses the features for each supervised task. A practitioner has no default to trust and should ask which settings to try and how to compare them fairly.

7.2.1 Simple Models and Small Feature Sets

Under label noise, the simpler choice was tree ensembles and SVM rather than plain neural networks (Chapter 3). On QM9 under Gaussian noise, the forests, boosted trees, SVM and the GP were accurate on clean labels and robust. The four plain and Bayesian networks were among the most accurate but the four least robust, and their $\mathrm{AUC}_{\mathrm{norm}}$ varied most across representations; the VBLL networks sat in the middle. Robustness is not accuracy: NGBoost had the highest $\mathrm{AUC}_{\mathrm{norm}}$ on all six representations, yet the lowest clean R^2 on four. On the assay datasets, random forest stayed in the top seven on every assay dataset and representation and rose as noise grew.

The hybrid search sweeps the budget rather than assuming that more features are better. The best hybrid used fewer features than the largest budget in **[RESULT NEEDED: how often]**. An ensemble of single-representation models **[RESULT NEEDED: matched the hybrid at lower cost?]**. When predicting each molecule's activity from its five nearest neighbors, a small hybrid chosen for the task scored **[RESULT NEEDED: score pairs]** against each downstream model's top two single representations. The model stays simple, so any gain comes from the input.

Several kinds of LNL method did not help, beginning with those of Chapter 4

(Sections 4.4.1 and 4.4.2). Of twelve LNL methods compared with a plain neural network under Gaussian noise at 0.5 and 1.5 times the label standard deviation ("Gaussian noise: methods for learning with noisy labels", Chapter 6), **[RESULT NEEDED: how many beat it]**. On real labels, **[RESULT NEEDED: whether any cleaning beat no cleaning]** ("Real labels: cleaning without injected noise"). For labels clipped at a detection limit, correction helped **[RESULT NEEDED: confirm]**. At separate sites with different noise levels, removing flagged molecules **[RESULT NEEDED: beat pooling?]**.

LNL methods from other fields carry over poorly to assay data. Most were built for images with flipped class labels [100] (Section 4.4.1), whereas the three assay datasets of Chapter 3 have 1,415 to 5,039 molecules and continuous labels, all with some error. Many LNL methods rely on a sample's neighbors or on how easily it is learned; both need a space where close molecules have similar activity, which standard representations lack (Chapter 4). Tree models also still lead on medium-sized tabular data [268] ("Models and Metrics", Chapter 5).

A method tuned closely to one dataset loses generality. The tasks in this thesis span binding affinity, ADME, quantum-chemical properties and antibodies, and this range is the reason learned models are used at all. Simple methods do not generalize automatically either: feature selection needs tuning for each dataset, which is why the search sweeps its settings, and the search itself can overfit (Discussion, Chapter 5). The balance is to fix the simple, robust parts, such as a tree ensemble or nearest neighbors on a small feature set, and search only what varies by dataset.

7.3 Limitations

The limits of the hybrid search itself are in the Limitations section of Chapter 5. This thesis set out to mitigate label noise, and we conclude that correcting it without ground truth is a hard problem with no clear general solution. Every experimental label carries an unknown amount of noise, so there is no clean copy of the labels to check a correction against, and finding a noisy label is easier than correcting it ("Comparison across noise types", Chapter 6). Outliers and systematic errors of known structure, such as a shift shared by one source or clipping at a detection limit, can still be detected. In the assay experiments, "clean" means no noise was added, not that the labels are true; only QM9 is close to noise-free. Real data mix the injected noise conditions in unknown amounts, so a method that works under

one condition may fail on real data. **[RESULT NEEDED: per noise condition, methods beating the uncorrected model; negative recovery.]**

Noise mitigation writes an estimated correction into the labels before training, so a wrong estimate stays for every later model; it is cheaper, since it is done once and reused by any model. LNL methods that change training instead adapt throughout training and use all the data [100]. Correcting labels directly is justified when the noise has a known direction, as in censoring, or affects a clearly separate subset, such as gross outliers or an offset tied to one source. Otherwise, we recommend LNL methods that change training. Either approach can be paired with uncertainty estimation where it is reliable. Predicted uncertainty rises with noise (Chapter 3), but per-molecule uncertainty does not identify noisy labels, so it should be used to judge how noisy a dataset is, not which labels are wrong.

The label-noise experiments are almost all regression; only the survey of twelve LNL methods flips class labels, uniformly at random. The controlled label-noise experiments use QM9 and three small assay datasets, LogD, Caco-2 and hERG K_i. No dataset here has repeated measurements with a known true value for every molecule, as a direct test of label correction would need. Compute limited how many noise conditions, noise levels, folds and datasets could be crossed. Results are per-fold scores and their spread over 3 to 5 folds, without significance tests, so some patterns show only a direction. **[RESULT NEEDED: final folds and datasets per chapter.]**

7.4 Future Directions

The first next step is classification, where a noisy label is a flipped class. Many QSAR class labels come from cutting a continuous value at a threshold, so flips concentrate near the cut-off and depend on the molecule. NoiseInject provides six flipping strategies (Section 2.6.1) and the search handles classification, but molecule-dependent flips have not been studied at the scale of the regression study. Most LNL methods that change training were designed for flipped labels, so whether the regression findings carry over is open.

The search does not yet include transformed features (normalized or binarized continuous features), features derived from distances in the molecular graph, or importance methods that treat the model as a black box, such as KernelSHAP and permutation importance. No search objective yet rewards robustness to label noise

as well as accuracy.

Chapter 6 uses outliers and grouped offsets, where all labels from one laboratory or source document are shifted together, only to grade noise detection. It remains open whether neighbor-based cleaning, which fails under Gaussian noise, does better on outliers, and whether label smoothing and sample weighting still do best on these noise types.

Hybrid features could also serve as inputs to generative models, such as the model that scores molecules for a generator. What the search should optimize there is unclear, since held-out accuracy on known molecules may not reward the right properties in new ones. The hybrid features of every generated molecule must also be computed.

The full search is too expensive for routine use. The fast search tier (Table 5.6) and adaptive pruning, which drops configurations early, already cut its cost. The next step is a warm start near the blueprint of a similar dataset; Chapter 5 already reuses one dataset's search settings on another without search. A faster search must still reach every configuration, only visiting fewer. **[RESULT NEEDED: hours and model fits, full search against fast tier.]**

Three smaller directions remain open: whether the hybrid improves a weak new embedding as well as absorbing a strong one; transfer of blueprints across domains, which would have to work at the level of representation families, since features differ between domains; and a fuller federated-learning deployment with Lhasa, in which real sites run feature selection on their own data [276].

External benchmarks not run here could be added later: activity-cliff prediction [73], antibody developability with a public competition's cluster splits [288], and antibody–antigen binding free-energy change, comparing sequence representations with a structure-based graph network [289].

7.5 Conclusions

Both halves of this thesis lead to the same advice. Before choosing a model, representation or LNL method, a practitioner should ask where the labels came from and what shape their noise is likely to have, then choose features for the task rather than take one representation whole. Since no single choice was best on every dataset, the choices that vary by dataset should be searched and compared fairly.

Simple choices were often enough. Tree ensembles and SVM held up better under

label noise than neural networks (Chapter 3), and a small feature set chosen for the task **[RESULT NEEDED: matched or beat larger inputs? (Chapters 5 and 6)]**.

A new foundation model appears every few months in molecular, protein and antibody modeling. With the hybrid search, a researcher need not replace their representation or retrain their pipeline: they add the new embeddings to the representations searched and re-run the search, which decides whether the new embedding contributes features and, if so, how many and which. A choice often set by familiarity or laboratory convention becomes one measured on each task, repeatable as new representations appear.

Summary of Research Contributions

Research Papers

In preparation: Punt, A., Morris, G., Werner, S., & Hanser, T. (2026). Improving noise prediction robustness and mitigation with feature-importance hybrid representations in multi-modal contexts
I am finishing a draft and will extend the experiments in Chapters 5 and 6 within the next few months.

Conference Presentations

Punt, A., Morris, G., Werner, S., & Hanser, T. (2024). Investigating the impact and mitigation of noise in QSAR models. Presented at the Autumn UK-QSAR 2024 Meeting.

Blogposts

I have contributed to the *Blopig* blog of the OPIG:

- Are you addicted to dopamine?
- Demystifying Git and merge conflicts
- AI in Academic Writing: Ethical Considerations and Best Practices
- Memory-mapped files for efficient data processing
- Exploring multilingual programming

Bibliography

[1] Markus A. Lill. Multi-dimensional QSAR in drug discovery. *Drug Discovery Today*, 12(23):1013–1017, 2007. ISSN 1359-6446.

[2] Bruno J. Neves, Rodolpho C. Braga, and Cleber C. Melo-Filho et al. QSAR-based virtual screening: Advances and applications in drug discovery. *Frontiers in Pharmacology*, 9:1275, 2018. ISSN 1663-9812.

[3] María Virginia Sabando, Ignacio Ponzoni, Evangelos E Milios, and Axel J Soto. Using molecular embeddings in QSAR modeling: Does it make a difference? *Briefings in Bioinformatics*, 23(1):bbab365, 2022. ISSN 1477-4054.

[4] David Rogers and Mathew Hahn. Extended-connectivity fingerprints. *Journal of Chemical Information and Modeling*, 50(5):742–754, 2010. doi: 10.1021/ci100050t. PMID: 20426451.

[5] Luís David, Abhijit Thakkar, Raul Mercado, et al. Molecular representations in AI-driven drug discovery: a review and practical guide. *Journal of Cheminformatics*, 12(1):56, 2020.

[6] Markus Dablander, Thierry Hanser, Renaud Lambiotte, and Garrett M. Morris. Sort & slice: a simple and superior alternative to hash-based folding for extended-connectivity fingerprints. *Journal of Cheminformatics*, 16(1), December 2024. ISSN 1758-2946. doi: 10.1186/s13321-024-00932-y.

[7] Peter Gedeck, Bernhard Rohde, and Christian Bartels. QSAR — how good is it in practice? Comparison of descriptor sets on an unbiased cross section of corporate data sets. *Journal of Chemical Information and Modeling*, 46(5): 1924–1936, 2006. doi: 10.1021/ci050413p.

[8] Akihiro Kishimoto, Hiroshi Kajino, Masataka Hirose, Junta Fuchiwaki, Indra Priyadarsini, Lisa Hamada, Hajime Shinohara, Daiju Nakano, and Seiji

Takeda. Mhg-gnn: Combination of molecular hypergraph grammar with graph neural network, 2023.

[9] Scott S. Kolmar and Christopher M. Grulke. The effect of noise on the predictive limit of QSAR models. *Journal of Cheminformatics*, 13(1):92, 2021.

[10] Kjell Jorner, Tore Brinck, Per-Ola Norrby, and David Buttar. Machine learning meets mechanistic modelling for accurate prediction of experimental activation energies. *Chem. Sci.*, 12:1163–1175, 2021. doi: 10.1039/D0SC04896H.

[11] Denis Fourches, Eugene Muratov, and Alexander Tropsha. Trust, but verify: On the importance of chemical structure curation in cheminformatics and qsar modeling research. *Journal of Chemical Information and Modeling*, 50 (7):1189–1204, 2010. doi: 10.1021/ci100176x. PMID: 20572635.

[12] Xiuwen Fang and Mang Ye. Robust federated learning with noisy and heterogeneous clients. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10062–10071, 2022.

[13] W. Patrick Walters. We need better benchmarks for machine learning in drug discovery. Practical Cheminformatics Blog, 2023. URL http://practicalcheminformatics.blogspot.com/2023/08/we-need-better-benchmarks-for-machine.html. Accessed: 2026-04-12.

[14] Tuomo Kalliokoski, Christian Kramer, Anna Vulpetti, and Peter Gedeck. Comparability of mixed IC50 data – a statistical analysis. *PLoS ONE*, 8 (4):e61007, 2013. doi: 10.1371/journal.pone.0061007.

[15] Christian Kramer, Tuomo Kalliokoski, Peter Gedeck, and Anna Vulpetti. The experimental uncertainty of heterogeneous public Ki data. *Journal of Medicinal Chemistry*, 55(11):5165–5173, 2012. doi: 10.1021/jm300131x.

[16] Gregory A. Landrum and Sereina Riniker. Combining IC50 or Ki values from different sources is a source of significant noise. *Journal of Chemical Information and Modeling*, 64(5):1560–1567, 2024. doi: 10.1021/acs.jcim.4c00049. PMID: 38394344.

[17] Isidro Cortes-Ciriano, Andreas Bender, and Thérèse E. Malliavin. Comparing the influence of simulated experimental errors on 12 machine learning algorithms in bioactivity modeling using 12 diverse data sets. *Journal of Chemical Information and Modeling*, 55(7):1413–1425, 2015. doi: 10.1021/acs.jcim.5b00101. PMID: 26038978.

[18] Davide Bassani, Alessandro Brigo, and Andrea Andrews-Morger. Federated learning in computational toxicology: An industrial perspective on the effiris hackathon. *Chemical Research in Toxicology*, 36(9):1503–1517, 2023.

[19] Thierry Hanser. Federated learning for molecular discovery. *Current Opinion in Structural Biology*, 79:102545, 2023. ISSN 0959-440X.

[20] David Weininger. SMILES, a chemical language and information system. 1. Introduction to methodology and encoding rules. *Journal of Chemical Information and Computer Sciences*, 28(1):31–36, 1988.

[21] Michael Backenköhler, Paula Linh Kramer, and Joschka et al. Groß. TeachOpenCADD goes Deep Learning: Open-source Teaching Platform Exploring Molecular DL Applications. *ChemRxiv preprint*, 2023. doi: 10.26434/chemrxiv-2023-kz1pb.

[22] Alexandre V. Fassio, Laura Shub, and Luca Ponzoni et al. Prioritizing virtual screening with interpretable interaction fingerprints. *Journal of Chemical Information and Modeling*, 62(18):4300–4318, 2022.

[23] Liangxu Xie, Lei Xu, Ren Kong, Shan Chang, and Xiaojun Xu. Improvement of prediction performance with conjoint molecular fingerprint in deep learning. *Frontiers in Pharmacology*, 11, 2020. doi: 10.3389/fphar.2020.606668.

[24] Isidro Cortés-Ciriano, Ctibor Škuta, Andreas Bender, and Daniel Svozil. QSAR-derived affinity fingerprints (part 2): Modeling performance for potency prediction. *Journal of Cheminformatics*, 12(1):41, 2020.

[25] Sabrina Jaeger, Simone Fulle, and Samo Turk. Mol2vec: Unsupervised machine learning approach with chemical intuition. *Journal of Chemical Information and Modeling*, 58(1):27–35, 2018. doi: 10.1021/acs.jcim.7b00616.

[26] Seyone Chithrananda, Gabriel Grand, and Bharath Ramsundar. Chemberta: Large-scale self-supervised pretraining for molecular property prediction. *arXiv preprint arXiv:2010.09885*, 2020.

[27] Benedek Fabian, Thomas Edlich, and Héléna et al. Gaspar. Molecular representation learning with language models and domain-relevant auxiliary tasks. *arXiv preprint arXiv:2011.13230*, 2020.

[28] Walid Ahmad, Elana Simon, Seyone Chithrananda, Gabriel Grand, and Bharath Ramsundar. Chemberta-2: Towards chemical foundation models. *arXiv preprint arXiv:2209.01712*, 2022.

[29] Roberto Todeschini and Viviana Consonni. *Molecular Descriptors for Chemoinformatics*. Wiley-VCH, Weinheim, 2nd edition, 2009. doi: 10.1002/9783527628766.

[30] Andrea Mauri, Viviana Consonni, Manuela Pavan, and Roberto Todeschini. DRAGON software: An easy approach to molecular descriptor calculations. *MATCH Communications in Mathematical and in Computer Chemistry*, 56 (2):237–248, 2006.

[31] Artem Cherkasov, Eugene N. Muratov, and Denis et al. Fourches. QSAR modeling: Where have you been? where are you going to? *Journal of Medicinal Chemistry*, 57(12):4977–5010, 2014. doi: 10.1021/jm4004285.

[32] Greg Landrum. Rdkit: Open-source cheminformatics. https://www.rdkit.org. Accessed: 2026-01-09.

[33] Zhichun Guo, Kehan Guo, and Bozhao Nan et al. Graph-based molecular representation learning, 2023. Accessed: 2024-11-30.

[34] Stefan M.N. Hönig, Carsten Lemmen, and Matthias Rarey. Small molecule superposition: A comprehensive overview on pose scoring of the latest methods. *WIREs Computational Molecular Science*, 13(2):e1640, 2023.

[35] Ge Yan, Huaijin Wu, and Junchi Yan. Quantum 3D graph learning with applications to molecule embedding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 39126–39137. PMLR, 2023.

[36] Ziduo Yang, Weihe Zhong, Qiujie Lv, Tiejun Dong, and Calvin Yu-Chian Chen. Geometric interaction graph neural network for predicting protein–ligand binding affinities from 3D structures (GIGN). *The Journal of Physical Chemistry Letters*, 14(8):2020–2033, 2023. doi: 10.1021/acs.jpclett.2c03906.

[37] Wenjie Du, Xiaoting Yang, and Di et al. Wu. Fusing 2D and 3D molecular graphs as unambiguous molecular descriptors for conformational and chiral stereoisomers. *Briefings in Bioinformatics*, 24(1):bbac560, 2022. ISSN 1477-4054.

[38] Denis Kuzminykh, Daniil Polykovskiy, and Artur Kadurin et al. 3D molecular representations based on the wave transform for convolutional neural networks. *Molecular Pharmaceutics*, 15(10):4378–4385, 2018.

[39] Giorgos Bouritsas, Fabrizio Frasca, Stefanos Zafeiriou, and Michael M. Bronstein. Improving graph neural network expressivity via subgraph isomorphism counting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(1):657–668, 2023.

[40] Jerret Ross, Brian Belgodere, Vijil Chenthamarakshan, Inkit Padhi, Youssef Mroueh, and Payel Das. Large-scale chemical language representations capture molecular structure and properties, 2022.

[41] Esther Heid, Charles J. McGill, Florence H. Vermeire, and William H. Green. Characterizing uncertainty in machine learning for chemistry. *Journal of Chemical Information and Modeling*, 63(13):4012–4029, 2023. doi: 10.1021/acs.jcim.3c00373. PMID: 37338239.

[42] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Wadsworth & Brooks/Cole, Monterey, CA, 1984. ISBN 978-0-412-04841-8.

[43] Vladimir Svetnik, Andy Liaw, Christopher Tong, John Culberson, Robert Sheridan, and Bradley Feuston. Random forest: A classification and regression tool for compound classification and qsar modeling. *Journal of chemical information and computer sciences*, 43:1947–58, 11 2003. doi: 10.1021/ci034160g.

[44] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/A:1010933404324.

[45] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. doi: 10.1145/2939672.2939785.

[46] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989. doi: 10.1016/0893-6080(89)90020-8.

[47] Junshui Ma, Robert P Sheridan, Andy Liaw, George E Dahl, and Vladimir Svetnik. Deep neural nets as a method for quantitative structure-activity relationships. *Journal of Chemical Information and Modeling*, 55(2):263–274, 2015. doi: 10.1021/ci500747n.

[48] Zhenqin Wu, Bharath Ramsundar, Evan N. Feinberg, Joseph Gomes, Caleb Geniesse, Aneesh S. Pappu, Karl Leswing, and Vijay Pande. Moleculenet: A benchmark for molecular machine learning, 2018.

[49] Igor I. Baskin, Vladimir A. Palyulin, and Nikolai S. Zefirov. Neural networks in building QSAR models. In *Methods in Molecular Biology*, volume 458, pages 137–158. Humana Press, 2008. doi: 10.1007/978-1-60327-101-1_8.

[50] Alexios Koutsoukas, Keith J. Monaghan, Xiaoli Li, and Jun Huan. Deep-learning: investigating deep neural networks hyper-parameters and comparison of performance to shallow methods for modeling bioactivity data. *Journal of Cheminformatics*, 9:42, 2017. doi: 10.1186/s13321-017-0226-y.

[51] Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization. In *International Conference on Learning Representations (ICLR)*, 2017.

[52] Curtis G Northcutt, Lu Jiang, and Isaac L Chuang. Confident learning: Estimating uncertainty in dataset labels. *Journal of Artificial Intelligence Research*, 70:1373–1411, 2021. doi: 10.1613/jair.1.12125.

[53] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks?, 2019.

[54] Nicolai Meinshausen. Quantile regression forests. *Journal of Machine Learning Research*, 7:983–999, 06 2006.

[55] Ismail Babajide Mustapha and Faisal Saeed. Bioactive molecule prediction using extreme gradient boosting. *Molecules*, 21(8), 2016. ISSN 1420-3049. doi: 10.3390/molecules21080983.

[56] Hao Tian, Rajas Ketkar, and Peng Tao. Accurate admet prediction with xgboost, 2022.

[57] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 3146–3154, 2017.

[58] Tony Duan, Anand Avati, Daisy Yi Ding, Khanh K. Thai, Sanjay Basu, Andrew Y. Ng, and Alejandro Schuler. Ngboost: Natural gradient boosting for probabilistic prediction, 2020.

[59] Vladimir N. Vapnik. *The Nature of Statistical Learning Theory*. Springer, New York, 1995. doi: 10.1007/978-1-4757-2440-0.

[60] Alex J. Smola and Bernhard Schölkopf. A tutorial on support vector regression. *Statistics and Computing*, 14(3):199–222, 2004. doi: 10.1023/B:STCO.0000035301.49549.88.

[61] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995. doi: 10.1007/BF00994018.

[62] Chih-Chung Chang and Chih-Jen Lin. LIBSVM: A library for support vector machines. *ACM Transactions on Intelligent Systems and Technology*, 2(3): 1–27, 2011. doi: 10.1145/1961189.1961199.

[63] Olga Obrezanova, Joelle M.R. Gola, and Matthew D. Segall. Gaussian processes: A method for automatic qsar and adme modelling. Presentation at ACS Boston, 22 August 2007, 2007.

[64] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. The MIT Press, 2005. ISBN 9780262256834.

[65] Joaquin Quiñonero-Candela and Carl Edward Rasmussen. A unifying view of sparse approximate gaussian process regression. *Journal of Machine Learning Research*, 6(65):1939–1959, 2005.

[66] Ryan-Rhys Griffiths and Leo Klarner et al. Gauche: A library for gaussian processes in chemistry, 2022. Accessed: 2024-08-21.

[67] L. Ralaivola, S. Joshua Swamidass, Hiroto Saigo, and Pierre Baldi. Graph kernels for chemical informatics. *Neural networks : the official journal of the International Neural Network Society*, 18:1093–110, 11 2005. doi: 10.1016/j.neunet.2005.07.009.

[68] Henry Moss and Ryan-Rhys Griffiths. Gaussian process molecule property prediction with flowmo, 10 2020.

[69] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles, 2017.

[70] Glenn Palmer, Siqi Du, Alexander Politowicz, Joshua Emory, Xiyu Yang, Anupraas Gautam, Grishma Gupta, Zhelong Li, Ryan Jacobs, and Dane Morgan. Calibration after bootstrap for accurate uncertainty quantification in regression models. *npj Computational Materials*, 8:115, 05 2022. doi: 10.1038/s41524-022-00794-8.

[71] Dávid Bajusz, Anita Rácz, and Károly Héberger. Why is tanimoto index an appropriate choice for fingerprint-based similarity calculations? *Journal of Cheminformatics*, 7(1), May 2015. ISSN 1758-2946. doi: 10.1186/s13321-015-0069-3.

[72] Dagmar Stumpfe and Jürgen Bajorath. Exploring activity cliffs in medicinal chemistry. *Journal of Medicinal Chemistry*, 55(7):2932–2942, 2012. doi: 10.1021/jm201706b.

[73] Markus Dablander, Thierry Hanser, Renaud Lambiotte, and Garrett M. Morris. Exploring QSAR models for activity-cliff prediction. *Journal of Cheminformatics*, 15(1):47, 2023. doi: 10.1186/s13321-023-00708-w.

[74] Mark A. Johnson and Gerald M. Maggiora, editors. *Concepts and Applications of Molecular Similarity*. Wiley, New York, 1990.

[75] Derek van Tilborg, Alisa Alenicheva, and Francesca Grisoni. Exposing the limitations of molecular machine learning with activity cliffs. *Journal of Chemical Information and Modeling*, 62(23):5938–5951, 2022. doi: 10.1021/acs.jcim.2c01073.

[76] Gerald Maggiora, Martin Vogt, Dagmar Stumpfe, and Jürgen Bajorath. Molecular similarity in medicinal chemistry. *Journal of Medicinal Chemistry*, 57(8):3186–3204, 2014. doi: 10.1021/jm401411z.

[77] Gerald M. Maggiora. On outliers and activity cliffs—why QSAR often disappoints. *Journal of Chemical Information and Modeling*, 46(4):1535, 2006. doi: 10.1021/ci060117s.

[78] Charu C. Aggarwal, Alexander Hinneburg, and Daniel A. Keim. On the surprising behavior of distance metrics in high dimensional space. In *Database Theory – ICDT 2001*, volume 1973 of *Lecture Notes in Computer Science*, pages 420–434, Berlin, Heidelberg, 2001. Springer. doi: 10.1007/3-540-44503-X_27.

[79] Felix A. Krüger and John P. Overington. Global analysis of small molecule binding to related protein targets. *PLoS Computational Biology*, 8(1): e1002333, 2012. doi: 10.1371/journal.pcbi.1002333.

[80] Joe Bentz, Michael P. O'Connor, Dallas Bednarczyk, JoAnn Coleman, Caroline Lee, Johan Palm, Y. Anne Pak, Elke S. Perloff, Eric Reyner, Praveen Balimane, et al. Variability in P-glycoprotein inhibitory potency (IC50) using various in vitro experimental systems: Implications for universal digoxin drug-drug interaction risk assessment decision criteria. *Drug Metabolism and Disposition*, 41(7):1347–1366, 2013. doi: 10.1124/dmd.112.050500.

[81] Emma Svensson, Hannah Rosa Friesacher, Susanne Winiwarter, Lewis Mervin, Adam Arany, and Ola Engkvist. Enhancing uncertainty quantification in drug discovery with censored regression labels. *Artificial Intelligence in the Life Sciences*, 7:100128, 2025. doi: 10.1016/j.ailsci.2025.100128.

[82] Ricardo Macarron, Martyn N. Banks, and Dejan et al. Bojanic. Impact of high-throughput screening in biomedical research. *Nature Reviews Drug Discovery*, 10(3):188–195, 2011. doi: 10.1038/nrd3368.

[83] James Inglese, Douglas S. Auld, and Ajit et a.. Jadhav. Quantitative high-throughput screening: A titration-based approach that efficiently identifies biological activities in large chemical libraries. *Proceedings of the National Academy of Sciences*, 103(31):11473–11478, 2006. doi: 10.1073/pnas.0604348103.

[84] Christine Brideau, Bert Gunter, Bill Pikounis, and Andy Liaw. Improved statistical methods for hit selection in high-throughput screening. *Journal of Biomolecular Screening*, 8(6):634–647, 2003. doi: 10.1177/1087057103258285.

[85] Ji-Hu Zhang, Thomas D. Y. Chung, and Kevin R. Oldenburg. A simple statistical parameter for use in evaluation and validation of high throughput screening assays. *Journal of Biomolecular Screening*, 4(2):67–73, 1999. doi: 10.1177/108705719900400206.

[86] Anton Simeonov, Ajit Jadhav, and Craig J. et al. Thomas. Fluorescence spectroscopic profiling of compound libraries. *Journal of Medicinal Chemistry*, 51(8):2363–2371, 2008. doi: 10.1021/jm701301m.

[87] Christos Hatzis, Philippe L. Bedard, Nicolai J. Birkbak, Andrew H. Beck, Hugo J.W.L. Aerts, David F. Stern, Leming Shi, Robert Clarke, John Quackenbush, and Benjamin Haibe-Kains. Enhancing reproducibility in cancer drug screening: How do we move forward? *Cancer Research*, 74(15):4016–4023, July 2014. ISSN 1538-7445. doi: 10.1158/0008-5472.can-14-0725.

[88] Elena Postnikova, Yu Cong, Lisa Evans DeWald, and Julie et al. Dyall. Testing therapeutics in cell-based assays: Factors that influence the apparent potency of drugs. *PLOS ONE*, 13(3):e0194880, March 2018. ISSN 1932-6203. doi: 10.1371/journal.pone.0194880.

[89] Anna Gaulton, Anne Hersey, and Michał et al. Nowotka. The ChEMBL database in 2017. *Nucleic Acids Research*, 45(D1):D945–D954, 2017. doi: 10.1093/nar/gkw1074.

[90] Michael K Gilson, Tiqing Liu, Michael Baitaluk, and et al. BindingDB in 2015: A public database for medicinal chemistry, computational chemistry and systems pharmacology. *Nucleic Acids Research*, 44(D1):D1045–D1053, 2016. doi: 10.1093/nar/gkv1072.

[91] Denis Fourches, Eugene Muratov, and Alexander Tropsha. Trust, but verify II: A practical guide to chemogenomics data curation. *Journal of Chemical Information and Modeling*, 56(7):1243–1252, 2016. doi: 10.1021/acs.jcim.6b00129.

[92] Antony J. Williams, Christopher M. Grulke, and Jeff et al. Edwards. The comptox chemistry dashboard: a community data resource for environmental chemistry. *Journal of Cheminformatics*, 9:61, 2017. doi: 10.1186/s13321-017-0247-6.

[93] Douglas Young, Todd Martin, Raghuraman Venkatapathy, and Phil Harten. Are the chemical structures in your QSAR correct? *QSAR & Combinatorial Science*, 27(11-12):1337–1345, 2008. doi: 10.1002/qsar.200810084.

[94] A Patrícia Bento, Anne Hersey, Eloy Félix, and Greg et al. Landrum. An open source chemical structure curation pipeline using RDKit. *Journal of Cheminformatics*, 12(1):51, 2020. doi: 10.1186/s13321-020-00456-1.

[95] Insha Ullah and A. H. Welsh. On the effect of noise on fitting linear regression models, 2024.

[96] Alexander Camuto, Matthew Willetts, Umut Şimşekli, Stephen Roberts, and Chris Holmes. Explicit regularisation in gaussian noise injections, 2021.

[97] Peter J. Huber. Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1):73–101, 1964. doi: 10.1214/aoms/1177703732.

[98] Frank R. Hampel. Robust statistics: a brief introduction and overview. Research Report 94, Seminar für Statistik, Eidgenössische Technische Hochschule (ETH) Zürich, 2001.

[99] Alex Avdeef. Multi-lab intrinsic solubility measurement reproducibility in CheqSol and shake-flask methods. *ADMET and DMPK*, 7(3):210–219, 2019. doi: 10.5599/admet.698.

[100] Hwanjun Song, Minseok Kim, Dongmin Park, Yooju Shin, and Jae-Gil Lee. Learning from noisy labels with deep neural networks: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 34(11):8135–8153, 2023. doi: 10.1109/TNNLS.2022.3152527.

[101] David H. Wolpert. The lack of a priori distinctions between learning algorithms. *Neural Computation*, 8(7):1341–1390, 1996. doi: 10.1162/neco.1996.8.7.1341.

[102] D.H. Wolpert and W.G. Macready. No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1):67–82, 1997. doi: 10.1109/4235.585893.

[103] Dennis L. Wilson. Asymptotic properties of nearest neighbor rules using edited data. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-2(3):408–421, 1972. doi: 10.1109/TSMC.1972.4309137.

[104] Adelaide Punt. NoiseInject: A Python package for label-noise robustness benchmarking in QSAR models. https://github.com/adpunt/noise_inject, 2026. Accessed: 2026-06-01.

[105] Yarin Gal and Zoubin Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning, 2016.

[106] Jianyuan Deng, Zhibo Yang, and Hehe Wang et al. Unraveling key elements underlying molecular property prediction: A systematic study, 2023.

[107] Raghunathan Ramakrishnan, Pavlo O. Dral, Matthias Rupp, et al. Quantum chemistry structures and properties of 134 kilo molecules. *Scientific Data*, 1:140022, 2014.

[108] Artem Fediai et al. Accurate GW frontier orbital energies of 134 kilo molecules. *Scientific Data*, 10:581, 2023. doi: 10.1038/s41597-023-02486-4.

[109] Open ADMET Consortium. OpenADMET-ExpansionRx challenge dataset, 2026. URL https://huggingface.co/datasets/openadmet/openadmet-expansionrx-challenge-data. CC-BY-4.0. ADMET data from drug discovery campaigns by Expansion Therapeutics targeting RNA-mediated diseases. Challenge closed January 19, 2026.

[110] Barbara Zdrazil, Eloy Felix, Fiona Hunter, et al. The ChEMBL database in 2023: a drug discovery platform spanning multiple bioactivity data types and time periods. *Nucleic Acids Research*, 52(D1):D1180–D1192, 11 2023. ISSN 0305-1048. doi: 10.1093/nar/gkad1004.

[111] Jacob R. Gardner, Geoff Pleiss, Kilian Q. Weinberger, David Bindel, and Andrew Gordon Wilson. GPyTorch: Blackbox matrix-matrix Gaussian process inference with GPU acceleration. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

[112] Matthias Fey and Jan E. Lenssen. Fast graph representation learning with pytorch geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[113] Alex Kendall and Yarin Gal. What uncertainties do we need in bayesian deep learning for computer vision? *CoRR*, abs/1703.04977, 2017.

[114] James Harrison, John Willes, and Jasper Snoek. Variational bayesian last layers, 2024.

[115] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, pages 2623–2631, 2019.

[116] Markus Orsi and Jean-Louis Reymond. One chiral fingerprint to find them all. *Journal of Cheminformatics*, 16:53, 2024. doi: 10.1186/s13321-024-00849-6.

[117] Lien Ai Nguyen, Hua He, and Chuong Pham-Huy. Chiral drugs: an overview. *International Journal of Biomedical Science*, 2(2):85–100, 2006. PMID: 23674971; PMCID: PMC3614593.

[118] Alexander Golbraikh, Eugene Muratov, Denis Fourches, and Alexander Tropsha. Data set modelability by QSAR. *Journal of Chemical Information and Modeling*, 54(1):1–4, 2014. doi: 10.1021/ci400572x.

[119] Robert P. Sheridan, Prabha Karnachi, Matthew Tudor, Yuting Xu, Andy Liaw, Falgun Shah, Alan C. Cheng, Elizabeth Joshi, Meir Glick, and Juan Alvarez. Experimental error, kurtosis, activity cliffs, and methodology: What limits the predictivity of quantitative structure–activity relationship models? *Journal of Chemical Information and Modeling*, 60(4):1969–1982, 2020. doi: 10.1021/acs.jcim.9b01067.

[120] José L. Medina-Franco. Activity cliffs: Facts or artifacts? *Chemical Biology & Drug Design*, 81(5):553–556, 2013. doi: 10.1111/cbdd.12115.

[121] Sereina Riniker and Gregory A. Landrum. Open-source platform to benchmark fingerprints for ligand-based virtual screening. *Journal of Cheminformatics*, 5:26, 2013. doi: 10.1186/1758-2946-5-26.

[122] Mateusz Praski, Jakub Adamczyk, and Wojciech Czech. Benchmarking pretrained molecular embedding models for molecular representation learning, 2025.

[123] Florian Huber and Julian Pollmann. Count your bits: fingerprint benchmarking to assess broad chemical space representation. *Journal of Cheminformatics*, 18:83, 2026. doi: 10.1186/s13321-026-01245-y.

[124] Pawel Dabrowski-Tumanski, Bartosz Topolski, Dariusz Plewczynski, and Tomasz Jetka. The geometry of activity cliffs: Representation dependence and multi-scale characterization of activity landscapes, 2026.

[125] Rajarshi Guha and John H. Van Drie. Structure–activity landscape index: Identifying and quantifying activity cliffs. *Journal of Chemical Information and Modeling*, 48(3):646–658, 2008. doi: 10.1021/ci7004093.

[126] Kenneth López-Pérez, Juan F. Avellaneda-Tamayo, Lexin Chen, Edgar López-López, K. Eurídice Juárez-Mercado, José L. Medina-Franco, and Ramón Alain Miranda-Quintana. Molecular similarity: Theory, applications, and perspectives. *Artificial Intelligence Chemistry*, 2(2):100077, 2024. doi: 10.1016/j.aichem.2024.100077.

[127] Alice Capecchi, Daniel Probst, and Jean-Louis Reymond. One molecular fingerprint to rule them all: drugs, biomolecules, and the metabolome. *Journal of Cheminformatics*, 12(1):43, 2020. doi: 10.1186/s13321-020-00445-4.

[128] Noel M. O'Boyle and Roger A. Sayle. Comparing structural fingerprints using a literature-based similarity benchmark. *Journal of Cheminformatics*, 8:36, 2016. doi: 10.1186/s13321-016-0148-0.

[129] Miloš Radovanović, Alexandros Nanopoulos, and Mirjana Ivanović. Hubs in space: Popular nearest neighbors in high-dimensional data. *Journal of Machine Learning Research*, 11:2487–2531, 2010. URL https://jmlr.org/papers/v11/radovanovic10a.html.

[130] David E. Patterson, Richard D. Cramer, Allen M. Ferguson, Robert D. Clark, and Laurence E. Weinberger. Neighborhood behavior: A useful concept for validation of "molecular diversity" descriptors. *Journal of Medicinal Chemistry*, 39(16):3049–3059, 1996. doi: 10.1021/jm960290n.

[131] David E. Coupry and Péter Pogány. Application of deep metric learning to molecular graph similarity. *Journal of Cheminformatics*, 14(1):11, 2022.

[132] Weifan Zheng and Alexander Tropsha. Novel variable selection quantitative structure–property relationship approach based on the k-nearest-neighbor principle. *Journal of Chemical Information and Computer Sciences*, 40(1): 185–194, 2000. doi: 10.1021/ci980033m.

[133] Peter Itskowitz and Alexander Tropsha. k nearest neighbors QSAR modeling as a variational problem: Theory and applications. *Journal of Chemical Information and Modeling*, 45(3):777–785, 2005. doi: 10.1021/ci049628+.

[134] Prasanta Chandra Mahalanobis. On the generalized distance in statistics. *Proceedings of the National Institute of Sciences of India*, 2(1):49–55, 1936.

[135] Alexander Tropsha. Best practices for QSAR model development, validation, and exploitation. *Molecular Informatics*, 29(6-7):476–488, 2010. doi: 10.1002/minf.201000061.

[136] Darko Butina. Unsupervised data base clustering based on daylight's fingerprint and tanimoto similarity: A fast and automated way to cluster small and large data sets. *Journal of Chemical Information and Computer Sciences*, 39 (4):747–750, 1999. doi: 10.1021/ci9803381.

[137] Thierry Hanser, Ernst Ahlberg, Alexander Amberg, Lennart T. Anger, Chris Barber, Richard J. Brennan, Alessandro Brigo, Annie Delaunois, Susanne Glowienke, Nigel Greene, Laura Johnston, Daniel Kuhn, Lara Kuhnke, Jean-François Marchaland, Wolfgang Muster, Jeffrey Plante, Friedrich Rippmann, Yogesh Sabnis, Friedemann Schmidt, Ruud van Deursen, Stéphane Werner, Angela White, Joerg Wichard, and Tomoya Yukawa. Data-driven federated learning in drug discovery with knowledge distillation. *Nature Machine Intelligence*, 7(3):423–436, 2025. doi: 10.1038/s42256-025-00991-2.

[138] Michael F. Dablander. *Investigating Graph Neural Networks and Classical Feature-Extraction Techniques in Activity-Cliff and Molecular Property Prediction.* PhD thesis, University of Oxford, 2023.

[139] Junnan Li, Richard Socher, and Steven C.H. Hoi. Dividemix: Learning with noisy labels as semi-supervised learning. In *International Conference on Learning Representations*, 2020.

[140] Bo Han, Quanming Yao, and Xingrui Yu et al. Co-teaching: Robust training of deep neural networks with extremely noisy labels. 2018. Accessed: 2024-09-01.

[141] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning*, ICML '09, pages 41–48. ACM, 2009.

[142] Haitao Liu, Yew-Soon Ong, and Jianfei Cai. Large-scale heteroscedastic regression via gaussian process, 2020.

[143] Carla E. Brodley and Mark A. Friedl. Identifying mislabeled training data. *Journal of Artificial Intelligence Research*, 11:131–167, 1999. doi: 10.1613/jair.606.

[144] Benoît Frénay and Michel Verleysen. Classification in the presence of label noise: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 25(5):845–869, 2014. doi: 10.1109/TNNLS.2013.2292894.

[145] Juan Martín, José A. Sáez, and Emilio Corchado. On the regressand noise problem: Model robustness and synergy with regression-adapted noise filters. *IEEE Access*, 9:145800–145816, 2021. doi: 10.1109/ACCESS.2021.3123151.

[146] Oliver Laufkötter, Noé Sturm, Jürgen Bajorath, Hongming Chen, and Ola Engkvist. Combining structural and bioactivity-based fingerprints improves prediction performance and scaffold hopping capability. *Journal of Cheminformatics*, 11(1), August 2019. ISSN 1758-2946. doi: 10.1186/s13321-019-0376-1.

[147] Viktoriia Turkina, Melanie R.W. Messih, Etienne Kant, and Jelle T. et al. Gringhuis. Enhancing predictive modeling with molecular fingerprint fusion strategies. July 2025. doi: 10.26434/chemrxiv-2025-b3lfq.

[148] Sören Richard Stahlschmidt, Benjamin Ulfenborg, and Jane Synnergren. Multimodal deep learning for biomedical data fusion: a review. *Briefings in Bioinformatics*, 23(2):bbab569, 2022. doi: 10.1093/bib/bbab569.

[149] Sandra Steyaert, Marija Pizurica, Divya Nagaraj, Priya Khandelwal, Tina Hernandez-Boussard, Andrew J. Gentles, and Olivier Gevaert. Multimodal data fusion for cancer biomarker discovery with deep learning. *Nature Machine Intelligence*, 5(4):351–362, 2023. doi: 10.1038/s42256-023-00633-5.

[150] Tongxin Wang, Wei Shao, Zhi Huang, Haixu Tang, Jie Zhang, Zhengming Ding, and Kun Huang. MOGONET integrates multi-omics data using graph convolutional networks allowing patient classification and biomarker identification. *Nature Communications*, 12(1):3445, 2021. doi: 10.1038/s41467-021-23774-w.

[151] Aurélien Beaude, Franck Augé, Farida Zehraoui, and Blaise Hanczar. CrossAttOmics: multiomics data integration with cross-attention. *Bioinformatics*, 41 (6):btaf302, 2025. doi: 10.1093/bioinformatics/btaf302.

[152] Mehmet Gönen and Ethem Alpaydın. Multiple kernel learning algorithms. *Journal of Machine Learning Research*, 12:2211–2268, 2011. URL https://www.jmlr.org/papers/v12/gonen11a.html.

[153] Anna Cichonska, Tapio Pahikkala, Sandor Szedmak, Heli Julkunen, Antti Airola, Markus Heinonen, Tero Aittokallio, and Juho Rousu. Learning with multiple pairwise kernels for drug bioactivity prediction. *Bioinformatics*, 34 (13):i509–i518, 2018. doi: 10.1093/bioinformatics/bty277.

[154] Gordon F. Hughes. On the mean accuracy of statistical pattern recognizers. *IEEE Transactions on Information Theory*, 14(1):55–63, 1968. doi: 10.1109/TIT.1968.1054102.

[155] Richard E. Bellman. *Adaptive Control Processes: A Guided Tour*. Princeton University Press, Princeton, NJ, 1961. doi: 10.1515/9781400874668.

[156] Dejun Jiang, Zhenxing Wu, Chang-Yu Hsieh, Guangyong Chen, Ben Liao, Zhe Wang, Chao Shen, Dongsheng Cao, Jian Wu, and Tingjun Hou. Could graph neural networks learn better molecular representation for drug discovery? A

comparison study of descriptor-based and graph-based models. *Journal of Cheminformatics*, 13(1):12, 2021. doi: 10.1186/s13321-020-00479-8.

[157] Andreas Mayr, Günter Klambauer, Thomas Unterthiner, Marvin Steijaert, Jörg K. Wegner, Hugo Ceulemans, Djork-Arné Clevert, and Sepp Hochreiter. Large-scale comparison of machine learning methods for drug target prediction on ChEMBL. *Chemical Science*, 9(24):5441–5451, 2018. doi: 10.1039/C8SC00148K.

[158] Henry Kenlay, Frédéric A. Dreyer, Aleksandr Kovaltsuk, Dom Miketa, Douglas Pires, and Charlotte M. Deane. Large scale paired antibody language models. *PLOS Computational Biology*, 20(12):e1012646, 2024. doi: 10.1371/journal.pcbi.1012646.

[159] Robert Schmirler, Michael Heinzinger, and Burkhard Rost. Fine-tuning protein language models boosts predictions across diverse tasks. *Nature Communications*, 15(1):7407, 2024. doi: 10.1038/s41467-024-51844-2.

[160] Peter Neidlinger, Omar S. M. El Nahhas, Hannah Sophie Muti, Tim Lenz, Michael Hoffmeister, Hermann Brenner, Marko van Treeck, Rupert Langer, Bastian Dislich, Hans Michael Behrens, Christoph Röcken, Sebastian Foersch, Daniel Truhn, Antonio Marra, Oliver Lester Saldanha, and Jakob Nikolas Kather. Benchmarking foundation models as feature extractors for weakly supervised computational pathology. *Nature Biomedical Engineering*, 10(6): 1113–1123, 2025. doi: 10.1038/s41551-025-01516-3.

[161] Bosheng Song, Jiayi Zhang, Ying Liu, Yuansheng Liu, Jing Jiang, Sisi Yuan, Xia Zhen, and Yiping Liu. A systematic review of molecular representation learning foundation models. *Briefings in Bioinformatics*, 27(1):bbaf703, 2026. doi: 10.1093/bib/bbaf703.

[162] Chris Thornton, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. Auto-WEKA: Combined selection and hyperparameter optimization of classification algorithms. In *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 847–855. ACM, 2013. doi: 10.1145/2487575.2487629.

[163] Cecile Valsecchi, Francesca Grisoni, Viviana Consonni, and Davide Ballabio. Consensus versus individual QSARs in classification: Comparison on a large-

scale case study. *Journal of Chemical Information and Modeling*, 60(3):1215–1223, 2020. doi: 10.1021/acs.jcim.9b01057.

[164] Isabelle Guyon and André Elisseeff. An introduction to variable and feature selection. *Journal of Machine Learning Research*, 3:1157–1182, 2003. URL https://www.jmlr.org/papers/v3/guyon03a.html.

[165] Hanchuan Peng, Fuhui Long, and Chris Ding. Feature selection based on mutual information criteria of max-dependency, max-relevance, and min-redundancy. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 27(8):1226–1238, 2005. doi: 10.1109/TPAMI.2005.159.

[166] Ron Kohavi and George H. John. Wrappers for feature subset selection. *Artificial Intelligence*, 97(1–2):273–324, 1997. doi: 10.1016/S0004-3702(97)00043-X.

[167] Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1):267–288, 1996. doi: 10.1111/j.2517-6161.1996.tb02080.x.

[168] Isabelle Guyon, Jason Weston, Stephen Barnhill, and Vladimir Vapnik. Gene selection for cancer classification using support vector machines. *Machine Learning*, 46:389–422, 2002. doi: 10.1023/A:1012487302797.

[169] Ming Yuan and Yi Lin. Model selection and estimation in regression with grouped variables. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 68(1):49–67, 2006. doi: 10.1111/j.1467-9868.2005.00532.x.

[170] Ian T. Jolliffe and Jorge Cadima. Principal component analysis: a review and recent developments. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 374(2065):20150202, 2016. doi: 10.1098/rsta.2015.0202.

[171] Ian T. Jolliffe. A note on the use of principal components in regression. *Journal of the Royal Statistical Society Series C: Applied Statistics*, 31(3):300–303, 1982. doi: 10.2307/2348005.

[172] Svante Wold, Michael Sjöström, and Lennart Eriksson. PLS-regression: a basic tool of chemometrics. *Chemometrics and Intelligent Laboratory Systems*, 58 (2):109–130, 2001. doi: 10.1016/S0169-7439(01)00155-1.

[173] José Jiménez-Luna, Francesca Grisoni, and Gisbert Schneider. Drug discovery with explainable artificial intelligence. *Nature Machine Intelligence*, 2(10): 573–584, 2020. doi: 10.1038/s42256-020-00236-4.

[174] Cynthia Rudin. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1(5):206–215, 2019. doi: 10.1038/s42256-019-0048-x.

[175] Marcus Olivecrona, Thomas Blaschke, Ola Engkvist, and Hongming Chen. Molecular de-novo design through deep reinforcement learning. *Journal of Cheminformatics*, 9(1):48, 2017. doi: 10.1186/s13321-017-0235-x.

[176] Christophe Ambroise and Geoffrey J. McLachlan. Selection bias in gene extraction on the basis of microarray gene-expression data. *Proceedings of the National Academy of Sciences*, 99(10):6562–6566, 2002. doi: 10.1073/pnas.102102699.

[177] Andrius Vabalas, Emma Gowen, Ellen Poliakoff, and Alexander J. Casson. Machine learning algorithm validation with a limited sample size. *PLOS ONE*, 14(11):e0224365, 2019. doi: 10.1371/journal.pone.0224365.

[178] Sudhir Varma and Richard Simon. Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics*, 7:91, 2006. doi: 10.1186/1471-2105-7-91.

[179] Gavin C. Cawley and Nicola L. C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11(70):2079–2107, 2010. URL https://www.jmlr.org/papers/v11/cawley10a.html.

[180] Jiayun Dong and Cynthia Rudin. Exploring the cloud of variable importance for the set of all good models. *Nature Machine Intelligence*, 2(12):810–824, 2020. doi: 10.1038/s42256-020-00264-0.

[181] Carolin Strobl, Anne-Laure Boulesteix, Achim Zeileis, and Torsten Hothorn. Bias in random forest variable importance measures: Illustrations, sources and a solution. *BMC Bioinformatics*, 8:25, 2007. doi: 10.1186/1471-2105-8-25.

[182] Avanti Shrikumar, Peyton Greenside, and Anshul Kundaje. Learning important features through propagating activation differences. In *Proceedings*

of the 34th International Conference on Machine Learning, volume 70 of *Proceedings of Machine Learning Research*, pages 3145–3153, 2017. URL https://proceedings.mlr.press/v70/shrikumar17a.html.

[183] Scott M. Lundberg, Gabriel Erion, Hugh Chen, Alex DeGrave, Jordan M. Prutkin, Bala Nair, Ronit Katz, Jonathan Himmelfarb, Nisha Bansal, and Su-In Lee. From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2(1):56–67, 2020. doi: 10.1038/s42256-019-0138-9.

[184] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations (ICLR)*, 2015. URL https://arxiv.org/abs/1412.6980.

[185] Narine Kokhlikyan, Vivek Miglani, Miguel Martin, Edward Wang, Bilal Alsallakh, Jonathan Reynolds, Alexander Melnikov, Natalia Kliushkina, Carlos Araya, Siqi Yan, and Orion Reblitz-Richardson. Captum: A unified and generic model interpretability library for PyTorch. *arXiv preprint arXiv:2009.07896*, 2020. URL https://arxiv.org/abs/2009.07896.

[186] Marco Ancona, Enea Ceolini, Cengiz Öztireli, and Markus Gross. Towards better understanding of gradient-based attribution methods for deep neural networks. In *International Conference on Learning Representations (ICLR)*, 2018. URL https://arxiv.org/abs/1711.06104.

[187] Malkeet Singh Bahia, Omer Kaspi, Meir Touitou, Idan Binayev, Seema Dhail, Jacob Spiegel, Netaly Khazanov, Abraham Yosipof, and Hanoch Senderowitz. A comparison between 2D and 3D descriptors in QSAR modeling based on bio-active conformations. *Molecular Informatics*, 42(4):e2200186, 2023. doi: 10.1002/minf.202200186.

[188] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001. doi: 10.1214/aos/1013203451.

[189] Bradley Efron, Trevor Hastie, Iain Johnstone, and Robert Tibshirani. Least angle regression. *The Annals of Statistics*, 32(2):407–499, 2004. doi: 10.1214/009053604000000067.

[190] Kevin Jamieson and Ameet Talwalkar. Non-stochastic best arm identification and hyperparameter optimization. In *Proceedings of the 19th International Conference on Artificial Intelligence and Statistics*, volume 51 of *Proceedings of Machine Learning Research*, pages 240–248, 2016. URL https://proceedings.mlr.press/v51/jamieson16.html.

[191] Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185):1–52, 2018. URL https://jmlr.org/papers/v18/16-558.html.

[192] Cheng Fang, Ye Wang, and Richard et al. Grater. Prospective validation of machine learning algorithms for absorption, distribution, metabolism, and excretion prediction: An industrial perspective. *Journal of Chemical Information and Modeling*, 63(11):3263–3274, 2023. doi: 10.1021/acs.jcim.3c00160. PMID: 37216672.

[193] Cas Wognum et al. Polaris: A benchmarking platform for drug discovery, 2024. URL https://doi.org/10.5281/zenodo.13652587.

[194] Raghunathan Ramakrishnan, Pavlo O. Dral, Matthias Rupp, and O. Anatole von Lilienfeld. Quantum chemistry structures and properties of 134 kilo molecules (QM9 dataset). https://doi.org/10.6084/m9.figshare.978904, 2014. Accessed: 2026-06-02.

[195] Derek T. Ahneman, Jesús G. Estrada, Shishi Lin, Spencer D. Dreher, and Abigail G. Doyle. Predicting reaction performance in C–N cross-coupling using machine learning. *Science*, 360(6385):186–190, 2018. doi: 10.1126/science.aar5169.

[196] Patrick Koenig, Chingwei V. Lee, Benjamin T. Walters, Vasantharajan Janakiraman, Jeremy Stinson, Thomas W. Patapoff, and Germaine Fuh. Mutational landscape of antibody variable domains reveals a switch modulating the interdomain conformational dynamics and antigen binding. *Proceedings of the National Academy of Sciences*, 114(4):E486–E495, 2017. doi: 10.1073/pnas.1613231114.

[197] Michael Chungyoun, Jeffrey Ruffolo, and Jeffrey Gray. FLAb: Benchmarking

deep learning methods for antibody fitness prediction. *bioRxiv*, 2024. doi: 10.1101/2024.01.13.575504.

[198] Mario S. Valdés-Tresanco, Mario E. Valdés-Tresanco, Esteban Molina-Abad, and Ernesto Moreno. NbThermo: a new thermostability database for nanobodies. *Database*, 2023:baad021, 2023. doi: 10.1093/database/baad021.

[199] David Prihoda, Jad Maamary, Andrew Waight, Veronica Juan, Laurence Fayadat-Dilman, Daniel Svozil, and Danny A. Bitton. BioPhi: A platform for antibody design, humanization, and humanness evaluation based on natural antibody repertoires and deep learning. *mAbs*, 14(1):2020203, 2022. doi: 10.1080/19420862.2021.2020203.

[200] Lewis Chinery, Alissa M. Hummer, Brij Bhushan Mehta, Rahmad Akbar, Puneet Rawat, Andrei Slabodkin, Khang Le Quy, Fridtjof Lund-Johansen, Victor Greiff, Jeliazko R. Jeliazkov, and Charlotte M. Deane. Baselining the buzz. Trastuzumab-HER2 affinity, and beyond. *bioRxiv*, 2024. doi: 10.1101/2024.03.26.586756. Version 1; later versions retitled "Simple computational methods can outperform deep learning in designing diverse, binder-enriched antibody libraries".

[201] Hugo Dalla-Torre, Liam Gonzalez, Javier Mendoza-Revilla, Nicolas Lopez Carranza, Adam Henryk Grzywaczewski, Francesco Oteri, Christian Dallago, Evan Trop, Bernardo P. de Almeida, Hassan Sirelkhatim, Guillaume Richard, Marcin Skwark, Karim Beguir, Marie Lopez, and Thomas Pierrot. Nucleotide transformer: building and evaluating robust foundation models for human genomics. *Nature Methods*, 22(2):287–297, 2025. doi: 10.1038/s41592-024-02523-z.

[202] The Cancer Genome Atlas Research Network. Comprehensive molecular profiling of lung adenocarcinoma. *Nature*, 511(7511):543–550, 2014. doi: 10.1038/nature13385.

[203] The Cancer Genome Atlas Research Network. Comprehensive genomic characterization of squamous cell lung cancers. *Nature*, 489(7417):519–525, 2012. doi: 10.1038/nature11404.

[204] Mary J. Goldman, Brian Craft, Mim Hastie, Kristupas Repečka, Fran McDade, Akhil Kamath, Ayan Banerjee, Yunhai Luo, Dave Rogers, Angela N.

Brooks, Jingchun Zhu, and David Haussler. Visualizing and interpreting cancer genomics data via the Xena platform. *Nature Biotechnology*, 38(6):675–678, 2020. doi: 10.1038/s41587-020-0546-8.

[205] The Cancer Genome Atlas Network. Comprehensive molecular portraits of human breast tumours. *Nature*, 490(7418):61–70, 2012. doi: 10.1038/nature11412.

[206] Joel S. Parker, Michael Mullins, Maggie C. U. Cheang, Samuel Leung, David Voduc, Tammi Vickery, Sherri Davies, Christiane Fauron, Xiaping He, Zhiyuan Hu, John F. Quackenbush, Inge J. Stijleman, Juan Palazzo, J. S. Marron, Andrew B. Nobel, Elaine Mardis, Torsten O. Nielsen, Matthew J. Ellis, Charles M. Perou, and Philip S. Bernard. Supervised risk predictor of breast cancer based on intrinsic subtypes. *Journal of Clinical Oncology*, 27(8):1160–1167, 2009. doi: 10.1200/JCO.2008.18.1370.

[207] Kevin Yang, Kyle Swanson, Wengong Jin, Connor Coley, Philipp Eiden, Hua Gao, Angel Guzman-Perez, Timothy Hopper, Brian Kelley, Miriam Mathea, Andrew Palmer, Volker Settels, et al. Analyzing learned molecular representations for property prediction. *Journal of Chemical Information and Modeling*, 59(8):3370–3388, 2019. doi: 10.1021/acs.jcim.9b00237.

[208] Izhar Wallach and Abraham Heifets. Most ligand-based classification benchmarks reward memorization rather than generalization. *Journal of Chemical Information and Modeling*, 58(5):916–932, 2018. doi: 10.1021/acs.jcim.7b00403.

[209] Pascal Notin, Aaron W. Kollasch, Daniel Ritter, Lood van Niekerk, Steffanie Paul, Hansen Spinner, Nathan Rollins, Ada Shaw, Ruben Weitzman, Jonathan Frazer, Mafalda Dias, Dinko Franceschi, et al. ProteinGym: Large-scale benchmarks for protein design and fitness prediction. *bioRxiv*, 2023. doi: 10.1101/2023.12.07.570727.

[210] Luiz C. Vieira, Sophia Lin, and Claus O. Wilke. Intrinsic dataset features drive mutational effect prediction by protein language models. *bioRxiv*, 2026. doi: 10.64898/2026.03.08.710389. Preprint.

[211] Frank Hoonakker, Nicolas Lachiche, Alexandre Varnek, and Alain Wagner. A representation to apply usual data mining techniques to chemical reactions

– illustration on the rate constant of SN2 reactions in water. *International Journal on Artificial Intelligence Tools*, 20(2):253–270, 2011. doi: 10.1142/S0218213011000140.

[212] Philippe Schwaller, Benjamin Hoover, Jean-Louis Reymond, Hendrik Strobelt, and Teodoro Laino. Extraction of organic chemistry grammar from unsupervised learning of chemical reactions. *Science Advances*, 7(15):eabe4166, 2021. doi: 10.1126/sciadv.abe4166.

[213] Philippe Schwaller, Daniel Probst, Alain C. Vaucher, Vishnu H. Nair, David Kreutter, Teodoro Laino, and Jean-Louis Reymond. Mapping the space of chemical reactions using attention-based neural networks. *Nature Machine Intelligence*, 3(2):144–152, 2021. doi: 10.1038/s42256-020-00284-w.

[214] Tatsuya Sagawa and Ryosuke Kojima. Reactiont5: a large-scale pre-trained model towards application of limited reaction data, 2023. URL https://arxiv.org/abs/2311.06708.

[215] Kangway V. Chuang and Michael J. Keiser. Comment on "predicting reaction performance in C–N cross-coupling using machine learning". *Science*, 362 (6416):eaat8603, 2018. doi: 10.1126/science.aat8603.

[216] S. Henikoff and J. G. Henikoff. Amino acid substitution matrices from protein blocks. *Proceedings of the National Academy of Sciences*, 89(22):10915–10919, 1992. doi: 10.1073/pnas.89.22.10915.

[217] Marie-Paule Lefranc, Christelle Pommié, Manuel Ruiz, Véronique Giudicelli, Elodie Foulquier, Lisa Truong, Valérie Thouvenin-Contet, and Gérard Lefranc. Imgt unique numbering for immunoglobulin and t cell receptor variable domains and ig superfamily v-like domains. *Developmental & Comparative Immunology*, 27(1):55–77, 2003. doi: 10.1016/S0145-305X(02)00039-3.

[218] James Dunbar and Charlotte M. Deane. Anarci: antigen receptor numbering and receptor classification. *Bioinformatics*, 32(2):298–300, 2016. doi: 10.1093/bioinformatics/btv552.

[219] Susana Vinga and Jonas Almeida. Alignment-free sequence comparison—a review. *Bioinformatics*, 19(4):513–523, 2003. doi: 10.1093/bioinformatics/btg005.

[220] Hanno Teeling, Anke Meyerdierks, Margarete Bauer, Rudolf Amann, and Frank Oliver Glöckner. Application of tetranucleotide frequencies for the assignment of genomic fragments. *Environmental Microbiology*, 6(9):938–947, 2004. doi: 10.1111/j.1462-2920.2004.00624.x.

[221] Zhihan Zhou, Yanrong Ji, Weijian Li, Pratik Dutta, Ramana Davuluri, and Han Liu. Dnabert-2: Efficient foundation model and benchmark for multi-species genome, 2023. URL https://arxiv.org/abs/2306.15006.

[222] Eric Nguyen, Michael Poli, Marjan Faizi, Armin Thomas, Callum Birch-Sykes, Michael Wornow, Aman Patel, Clayton Rabideau, Stefano Massaroli, Yoshua Bengio, Stefano Ermon, Stephen A. Baccus, and Chris Ré. Hyenadna: Long-range genomic sequence modeling at single nucleotide resolution, 2023. URL https://arxiv.org/abs/2306.15794.

[223] The Cancer Genome Atlas Research Network, John N Weinstein, Eric A Collisson, Gordon B Mills, Kenna R Mills Shaw, Brad A Ozenberger, Kyle Ellrott, Ilya Shmulevich, Chris Sander, and Joshua M Stuart. The cancer genome atlas pan-cancer analysis project. *Nature Genetics*, 45(10):1113–1120, 2013. doi: 10.1038/ng.2764.

[224] Raymond E. Carhart, Dennis H. Smith, and R. Venkataraghavan. Atom pairs as molecular features in structure-activity studies: definition and applications. *Journal of Chemical Information and Computer Sciences*, 25(2):64–73, 1985. doi: 10.1021/ci00046a002.

[225] Ramaswamy Nilakantan, Norman Bauman, J. Scott Dixon, and R. Venkataraghavan. Topological torsion: a new molecular descriptor for sar applications. comparison with other descriptors. *Journal of Chemical Information and Computer Sciences*, 27(2):82–85, 1987. doi: 10.1021/ci00054a008.

[226] Peter Gedeck, Bernhard Rohde, and Christian Bartels. Qsar – how good is it in practice? comparison of descriptor sets on an unbiased cross section of corporate data sets. *Journal of Chemical Information and Modeling*, 46(5): 1924–1936, 2006. doi: 10.1021/ci050413p.

[227] Joseph L. Durant, Burton A. Leland, Douglas R. Henry, and James G. Nourse. Reoptimization of MDL keys for use in drug discovery. *Journal of Chemical*

Information and Computer Sciences, 42(6):1273–1280, 2002. doi: 10.1021/ci010132r.

[228] Nino Shervashidze, Pascal Schweitzer, Erik Jan van Leeuwen, Kurt Mehlhorn, and Karsten M. Borgwardt. Weisfeiler-lehman graph kernels. *Journal of Machine Learning Research*, 12(77):2539–2561, 2011. URL https://www.jmlr.org/papers/v12/shervashidze11a.html.

[229] Hirotomo Moriwaki, Yu-Shi Tian, Norihito Kawashita, and Tatsuya Takagi. Mordred: a molecular descriptor calculator. *Journal of Cheminformatics*, 10 (1):4, 2018. doi: 10.1186/s13321-018-0258-y.

[230] Yu Rong, Yatao Bian, Tingyang Xu, Weiyang Xie, Ying Wei, Wenbing Huang, and Junzhou Huang. Self-supervised graph transformer on large-scale molecular data, 2020. URL https://arxiv.org/abs/2007.02835.

[231] Shengchao Liu, Hanchen Wang, Weiyang Liu, Joan Lasenby, Hongyu Guo, and Jian Tang. Pre-training molecular graph representation with 3d geometry, 2022. URL https://arxiv.org/abs/2110.07728.

[232] Eduardo Soares, Emilio Vital Brazil, Victor Shirasuna, Dmitry Zubarev, Renato Cerqueira, and Kristin Schmidt. An open-source family of large encoder-decoder foundation models for chemistry. *Communications Chemistry*, 8, 2025. doi: 10.1038/s42004-025-01585-0.

[233] Xiao-Chen Zhang, Cheng-Kun Wu, Jia-Cai Yi, Xiang-Xiang Zeng, Can-Qun Yang, Ai-Ping Lu, Ting-Jun Hou, and Dong-Sheng Cao. Pushing the boundaries of molecular property prediction for drug discovery with multitask learning bert enhanced by smiles enumeration. *Research*, 2022, 2022. doi: 10.34133/research.0004.

[234] Atakan Yüksel, Erva Ulusoy, Atabey Ünlü, and Tunca Doğan. Selformer: molecular representation learning via selfies language models. *Machine Learning: Science and Technology*, 4(2):025035, 2023. doi: 10.1088/2632-2153/acdb30.

[235] Pedro J. Ballester and W. Graham Richards. Ultrafast shape recognition to search compound databases for similar molecular shapes. *Journal of Computational Chemistry*, 28(10):1711–1723, 2007. doi: 10.1002/jcc.20681.

[236] Adrian M. Schreyer and Tom Blundell. Usrcat: real-time ultrafast shape recognition with pharmacophoric constraints. *Journal of Cheminformatics*, 4 (1):27, 2012. doi: 10.1186/1758-2946-4-27.

[237] Matthias Rupp, Alexandre Tkatchenko, Klaus-Robert Müller, and O. Anatole von Lilienfeld. Fast and accurate modeling of molecular atomization energies with machine learning. *Physical Review Letters*, 108(5):058301, 2012. doi: 10.1103/PhysRevLett.108.058301.

[238] Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin, Robert Verkuil, Ori Kabeli, Yaniv Shmueli, Allan dos Santos Costa, Maryam Fazel-Zarandi, Tom Sercu, Salvatore Candido, and Alexander Rives. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, 2023. doi: 10.1126/science.ade2574.

[239] Joshua Meier, Roshan Rao, Robert Verkuil, Jason Liu, Tom Sercu, and Alexander Rives. Language models enable zero-shot prediction of the effects of mutations on protein function. *bioRxiv*, 2021. doi: 10.1101/2021.07.09.450648.

[240] Thomas Hayes, Roshan Rao, Halil Akin, Nicholas J. Sofroniew, Deniz Oktay, Zeming Lin, Robert Verkuil, Vincent Q. Tran, Jonathan Deaton, Marius Wiggert, Rohil Badkundri, Irhum Shafkat, Jun Gong, Alexander Derry, Raul S. Molina, Neil Thomas, Yousuf A. Khan, Chetan Mishra, Carolyn Kim, Liam J. Bartie, Matthew Nemeth, Patrick D. Hsu, Tom Sercu, Salvatore Candido, and Alexander Rives. Simulating 500 million years of evolution with a language model. *Science*, 387(6736):850–858, 2025. doi: 10.1126/science.ads0018.

[241] Salvatore Candido, Thomas Hayes, Alexander Derry, Roshan Rao, Zeming Lin, Robert Verkuil, Bryan Z. Wu, Jin Sub Lee, Elise S. Bruguera, Jehan A. Keval, Mykhailo Kopylov, John E. Pak, Wesley Wu, Neil Thomas, Samson Mataraso, Alvin Hsu, Ashton C. Trotman-Grant, Kilian Fatras, Allan dos Santos Costa, Rohil Badkundri, Halil Akın, Deniz Oktay, Jonathan Deaton, Elizabeth Montabana, Hrishita Sitwala, Yue Yu, Marius Wiggert, Dylan Alexander Carlin, Anthony W. Goering, Tomasz Blazejewski, McCullen Sandora, Michael Hla, Tina Z. Jia, Leon H. Kloker, Nicholas J. Sofroniew, Masatoshi Uehara, Jassi Pannu, Sharrol Bachas, Daniel S. Liu, Tom Sercu, and Alexander Rives.

Language modeling materializes a world model of protein biology. *bioRxiv*, 2026. doi: 10.64898/2026.06.03.729735. Preprint, posted 4 June 2026.

[242] Ahmed Elnaggar, Michael Heinzinger, Christian Dallago, Ghalia Rehawi, Yu Wang, Llion Jones, Tom Gibbs, Tamas Feher, Christoph Angerer, Martin Steinegger, Debsindhu Bhowmik, and Burkhard Rost. Prottrans: Toward understanding the language of life through self-supervised learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(10):7112–7127, 2022. doi: 10.1109/TPAMI.2021.3095381.

[243] Jin Su, Chenchen Han, Yuyang Zhou, Junjie Shan, Xibin Zhou, and Fajie Yuan. Saprot: Protein language modeling with structure-aware vocabulary. *bioRxiv*, 2023. doi: 10.1101/2023.10.01.560349.

[244] Tobias H. Olsen, Iain H. Moal, and Charlotte M. Deane. Addressing the antibody germline bias and its effect on language models for improved antibody design. *Bioinformatics*, 40(11), 2024. doi: 10.1093/bioinformatics/btae618.

[245] Henry Kenlay, Frédéric A. Dreyer, Aleksandr Kovaltsuk, Dom Miketa, Douglas Pires, and Charlotte M. Deane. Large scale paired antibody language models. *PLOS Computational Biology*, 20(12):e1012646, 2024. doi: 10.1371/journal.pcbi.1012646.

[246] Jeffrey A. Ruffolo, Jeffrey J. Gray, and Jeremias Sulam. Deciphering antibody affinity maturation with language models and weakly supervised learning, 2021. URL https://arxiv.org/abs/2112.07782.

[247] Eunna Huh, Hyeonsu Lee, and Hyunjin Shin. Antibody foundational model: Ab-RoBERTa, 2025.

[248] Melissa Sanabria, Jonas Hirsch, Pierre M. Joubert, and Anna R. Poetsch. Dna language model grover learns sequence context in the human genome. *Nature Machine Intelligence*, 6(8):911–923, 2024. doi: 10.1038/s42256-024-00872-0.

[249] Raphael Mourad. Mistral-DNA: Large language model Mistral for DNA. https://github.com/raphaelmourad/Mistral-DNA, 2024. Code and model release, no paper; model RaphaelMourad/Mistral-DNA-v1-422M-hg38 on Hugging Face.

[250] Paul M. Sharp, Therese M. F. Tuohy, and Krzysztof R. Mosurski. Codon usage in yeast: cluster analysis clearly differentiates highly and lowly expressed genes. *Nucleic Acids Research*, 14(13):5125–5143, 1986. doi: 10.1093/nar/14.13.5125.

[251] Christina V. Theodoris, Ling Xiao, Anant Chopra, Mark D. Chaffin, Zeina R. Al Sayed, Matthew C. Hill, Helene Mantineo, Elizabeth M. Brydon, Zexian Zeng, X. Shirley Liu, and Patrick T. Ellinor. Transfer learning enables predictions in network biology. *Nature*, 618(7965):616–624, 2023. doi: 10.1038/s41586-023-06139-9.

[252] Christina V. Theodoris. Geneformer model repository: Geneformer-V2-316M. https://huggingface.co/ctheodoris/Geneformer, 2024. Model card: V2 trained December 2024 on about 104 million human single-cell transcriptomes, input size 4096; default model in the main directory is Geneformer-V2-316M (18 layers, 1152 hidden units).

[253] Maxence Gélard, Guillaume Richard, Thomas Pierrot, and Paul-Henry Cournède. Bulkrnabert: Cancer prognosis from bulk rna-seq based language models. *bioRxiv*, 2024. doi: 10.1101/2024.06.18.599483.

[254] Craig H Mermel, Steven E Schumacher, Barbara Hill, Matthew L Meyerson, Rameen Beroukhim, and Gad Getz. Gistic2.0 facilitates sensitive and confident localization of the targets of focal somatic copy-number alteration in human cancers. *Genome Biology*, 12(4):R41, 2011. doi: 10.1186/gb-2011-12-4-r41.

[255] Jun Li, Yiling Lu, Rehan Akbani, Zhenlin Ju, Paul L Roebuck, Wenbin Liu, Ji-Yeon Yang, Bradley M Broom, Roeland G W Verhaak, David W Kane, Chris Wakefield, John N Weinstein, Gordon B Mills, and Han Liang. Tcpa: a resource for cancer functional proteomics data. *Nature Methods*, 10(11): 1046–1047, 2013. doi: 10.1038/nmeth.2650.

[256] Tong Ding, Sophia J. Wagner, Andrew H. Song, Richard J. Chen, Ming Y. Lu, Andrew Zhang, Anurag J. Vaidya, Guillaume Jaume, Muhammad Shaban, Ahrong Kim, Drew F. K. Williamson, Bowen Chen, Cristina Almagro-Perez, Paul Doucet, Sharifa Sahai, Chengkuan Chen, Daisuke Komura, Akihiro Kawabe, Shumpei Ishikawa, Georg Gerber, Tingying Peng, Long Phi Le,

and Faisal Mahmood. Multimodal whole slide foundation model for pathology, 2024. URL https://arxiv.org/abs/2411.19666.

[257] Hanwen Xu, Naoto Usuyama, Jaspreet Bagga, Sheng Zhang, Rajesh Rao, Tristan Naumann, Cliff Wong, Zelalem Gero, Javier González, Yu Gu, Yanbo Xu, Mu Wei, Wenhui Wang, Shuming Ma, Furu Wei, Jianwei Yang, Chunyuan Li, Jianfeng Gao, Jaylen Rosemon, Tucker Bower, Soohee Lee, Roshanthi Weerasinghe, Bill J. Wright, Ari Robicsek, Brian Piening, Carlo Bifulco, Sheng Wang, and Hoifung Poon. A whole-slide foundation model for digital pathology from real-world data. *Nature*, 630(8015):181–188, 2024. doi: 10.1038/s41586-024-07441-w.

[258] Xiyue Wang, Sen Yang, Jun Zhang, Minghui Wang, Jing Zhang, Wei Yang, Junzhou Huang, and Xiao Han. Transformer-based unsupervised contrastive learning for histopathological image classification. *Medical Image Analysis*, 81: 102559, 2022. doi: 10.1016/j.media.2022.102559.

[259] Alexandre Filiot, Ridouane Ghermi, Antoine Olivier, Paul Jacob, Lucas Fidon, Alice Mac Kain, Charlie Saillard, and Jean-Baptiste Schiratti. Scaling self-supervised learning for histopathology with masked image modeling. *medRxiv*, 2023. doi: 10.1101/2023.07.21.23292757.

[260] Charlie Saillard, Rodolphe Jenatton, Felipe Llinares-López, Zelda Mariet, David Cahané, Eric Durand, and Jean-Philippe Vert. H-optimus-0, 2024. URL https://github.com/bioptimus/releases/tree/main/models/h-optimus/v0.

[261] Theo A. Knijnenburg, Linghua Wang, Michael T. Zimmermann, Nyasha Chambwe, Galen F. Gao, Andrew D. Cherniack, et al. Genomic and molecular landscape of DNA damage repair deficiency across The Cancer Genome Atlas. *Cell Reports*, 23(1):239–254.e6, 2018. doi: 10.1016/j.celrep.2018.03.076.

[262] Kyle Ellrott, Matthew H. Bailey, Gordon Saksena, Kyle R. Covington, Cyriac Kandoth, Chip Stewart, et al. Scalable open science approach for mutation calling of tumor exomes using multiple genomic pipelines. *Cell Systems*, 6(3): 271–281.e7, 2018. doi: 10.1016/j.cels.2018.03.002.

[263] Scott L. Carter, Kristian Cibulskis, Elena Helman, Aaron McKenna, Hui Shen, Travis Zack, Peter W. Laird, Robert C. Onofrio, Wendy Winckler, Barbara A.

Weir, Rameen Beroukhim, David Pellman, Douglas A. Levine, Eric S. Lander, Matthew Meyerson, and Gad Getz. Absolute quantification of somatic DNA alterations in human cancer. *Nature Biotechnology*, 30(5):413–421, 2012. doi: 10.1038/nbt.2203.

[264] Vésteinn Thorsson, David L. Gibbs, Scott D. Brown, Denise Wolf, Dante S. Bortone, Tai-Hsien Ou Yang, et al. The immune landscape of cancer. *Immunity*, 48(4):812–830.e14, 2018. doi: 10.1016/j.immuni.2018.03.023.

[265] Aaron M. Newman, Chih Long Liu, Michael R. Green, Andrew J. Gentles, Weiguo Feng, Yue Xu, Chuong D. Hoang, Maximilian Diehn, and Ash A. Alizadeh. Robust enumeration of cell subsets from tissue expression profiles. *Nature Methods*, 12(5):453–457, 2015. doi: 10.1038/nmeth.3337.

[266] Tathiane M. Malta, Artem Sokolov, Andrew J. Gentles, Tomasz Burzykowski, Laila Poisson, John N. Weinstein, et al. Machine learning identifies stemness features associated with oncogenic dedifferentiation. *Cell*, 173(2):338–354.e15, 2018. doi: 10.1016/j.cell.2018.03.034.

[267] Minoru Toyota, Nita Ahuja, Mutsumi Ohe-Toyota, James G. Herman, Stephen B. Baylin, and Jean-Pierre J. Issa. CpG island methylator phenotype in colorectal cancer. *Proceedings of the National Academy of Sciences*, 96(15):8681–8686, 1999. doi: 10.1073/pnas.96.15.8681.

[268] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022), Datasets and Benchmarks Track*, 2022. URL https://arxiv.org/abs/2207.08815.

[269] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, Cambridge, MA, 2006. ISBN 0-262-18253-X.

[270] Davide Chicco and Giuseppe Jurman. The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation. *BMC Genomics*, 21(1):6, 2020. doi: 10.1186/s12864-019-6413-7.

[271] Yoshua Bengio and Yves Grandvalet. No unbiased estimator of the variance of

K-fold cross-validation. *Journal of Machine Learning Research*, 5:1089–1105, 2004. URL https://www.jmlr.org/papers/v5/grandvalet04a.html.

[272] Sarah Nogueira, Konstantinos Sechidis, and Gavin Brown. On the stability of feature selection algorithms. *Journal of Machine Learning Research*, 18(174): 1–54, 2018. URL https://jmlr.org/papers/v18/17-514.html.

[273] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Series in Statistics. Springer, New York, 2 edition, 2009. doi: 10.1007/978-0-387-84858-7.

[274] Jun Shu, Qi Xie, Lixuan Yi, Qian Zhao, Sanping Zhou, Zongben Xu, and Deyu Meng. Meta-weight-net: Learning an explicit mapping for sample weighting. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[275] Ivan Olier, Noureddin Sadawi, G. Richard Bickerton, Joaquin Vanschoren, Crina Grosan, Larisa Soldatova, and Ross D. King. Meta-QSAR: a large-scale application of meta-learning to drug design and discovery. *Machine Learning*, 107(1):285–311, 2018. doi: 10.1007/s10994-017-5685-x.

[276] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, volume 54 of *Proceedings of Machine Learning Research*, pages 1273–1282, 2017. URL https://proceedings.mlr.press/v54/mcmahan17a.html.

[277] Wouter Heyndrickx, Lewis Mervin, Tobias Morawietz, Noé Sturm, Lukas Friedrich, Adam Zalewski, Anastasia Pentina, Lina Humbeck, Martijn Oldenhof, Ritsuya Niwayama, Peter Schmidtke, Nikolas Fechner, Jaak Simm, Adam Arany, Nicolas Drizard, Rama Jabal, Arina Afanasyeva, Regis Loeb, Shlok Verma, Simon Harnqvist, Matthew Holmes, Balazs Pejo, Maria Telenczuk, Nicholas Holway, Arne Dieckmann, Nicola Rieke, Friederike Zumsande, Djork-Arné Clevert, Michael Krug, Christopher Luscombe, Darren Green, Peter Ertl, Peter Antal, David Marcus, Nicolas Do Huu, Hideyoshi Fuji, Stephen Pickett, Gergely Acs, Eric Boniface, Bernd Beck, Yax Sun, Arnaud Gohier, Friedrich Rippmann, Ola Engkvist, Andreas H. Göller, Yves Moreau, Mathieu N.

Galtier, Ansgar Schuffenhauer, and Hugo Ceulemans. MELLODDY: Cross-pharma federated learning at unprecedented scale unlocks benefits in QSAR without compromising proprietary information. *Journal of Chemical Information and Modeling*, 64(7):2331–2344, 2024. doi: 10.1021/acs.jcim.3c00799.

[278] Pietro Cassarà, Alberto Gotta, and Lorenzo Valerio. Federated feature selection for cyber-physical systems of systems. *IEEE Transactions on Vehicular Technology*, 71(9):9937–9950, 2022. doi: 10.1109/TVT.2022.3178612.

[279] Paul Czodrowski. herg me out. *Journal of Chemical Information and Modeling*, 53(9):2240–2251, 2013.

[280] Ryan J. Tibshirani. Degrees of freedom and model search. *Statistica Sinica*, 25(3):1265–1296, 2015. doi: 10.5705/ss.2014.147.

[281] Andrew Y. Ng. Feature selection, L1 vs. L2 regularization, and rotational invariance. In *Proceedings of the Twenty-First International Conference on Machine Learning (ICML '04)*, page 78, 2004. doi: 10.1145/1015330.1015435.

[282] Ananya Kumar, Aditi Raghunathan, Robbie Matthew Jones, Tengyu Ma, and Percy Liang. Fine-tuning can distort pretrained features and underperform out-of-distribution. In *International Conference on Learning Representations (ICLR)*, 2022. URL https://openreview.net/forum?id=UYneFzXSJWh.

[283] Markus M. Breunig, Hans-Peter Kriegel, Raymond T. Ng, and Jörg Sander. Lof: identifying density-based local outliers. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, SIGMOD '00, page 93–104, New York, NY, USA, 2000. Association for Computing Machinery. ISBN 1581132174. doi: 10.1145/342009.335388.

[284] Junnan Li, Richard Socher, and Steven C. H. Hoi. Dividemix: Learning with noisy labels as semi-supervised learning, 2020.

[285] Maximilian Seitzer, Arash Tavakoli, Dimitrije Antic, and Georg Martius. On the pitfalls of heteroscedastic uncertainty estimation with probabilistic neural networks, 2022.

[286] Glenn Shafer and Vladimir Vovk. A tutorial on conformal prediction. *Journal of Machine Learning Research*, 9(12):371–421, 2008.

[287] Matt Taddy, Chun-Sheng Chen, Jun Yu, and Mitch Wyle. Bayesian and empirical bayesian forests, 2015.

[288] Lood van Niekerk, Joshua Moller, Seth Ritter, Porfirio Quintero-Cadena, Ariel Cohen, Alexander Channing, Michael Chungyoun, Ulfert Rand, Ian Smith, Nikhil Bhatt, Emmanuel Pierre, Casey Harris, Yi Ao, Vincent Grippo, Jasmin Schwenk, Amos Rosenbaum, Lucas Allen, Naif Asi, Jiaqi Zhu, Ritika Singh, Sonia Sammi, Vikram Jadhav, Jan Dušek, Sarath Chandra, Andreea Badea, Nels Thorsteinson, Marissa Blalock, Jared Kim, Oliver Turnbull, Shreyas Kulkarni, Vivek Kohar, Abiy Gebremedhin, Charlotte M. Deane, Peter M. Tessier, and Ammar Arsiwala. 2025 Ginkgo Datapoints antibody developability competition outcomes: limited model performance and a call for data standardization. *mAbs*, 18(1):2634216, 2026. doi: 10.1080/19420862.2026.2634216.

[289] Alissa M. Hummer, Constantin Schneider, Lewis Chinery, and Charlotte M. Deane. Investigating the volume and diversity of data needed for generalizable antibody–antigen $\delta\delta g$ prediction. *Nature Computational Science*, 5(8):635–647, 2025. doi: 10.1038/s43588-025-00823-8.